SE601 - Software Construction & Development

Final Week 7 to 16

13/01/2025

Software Construction and Development

Week-7

Efficiency of use and performance

- Different steps and concepts involve in defining the task accomplishment should contain the minimum set of requirements.
- Explaination:
- Smartphone should support the users to perform their respective tasks by providing refined and an abstracted interface to the user. System should explicitly provide basic application functionality, focused on users' objective. A task accomplishment requires system resources including processing capabilities, battery, number of steps, mental effort and others. Complex task requires comparatively more resources than a simple task.

Efficiency of use and performance

- Example
- If the user disconnects the call with “A”, after conversation. After some time, if the user dials to another person, let’s say “B”. Then, following steps are required to initiate the call. Press the dialed button, this results in opening up the previous caller interface (not necessary), from there, the user needs to go back with only “search” option. Another extra step is to cancel/delete the previously searched option, then find the contact number of “B” to call. Finally, press the dialed button. These extra steps further add up, if the dialer is busy in some other activity or moving.



Efficiency of use and performance

- Benefits
- Users comfort level goes up and intended to use the application in the future.
- Problems
- Possibly, one task accomplishment steps are minimum for an evaluator, while other raised question. So, it’s a subjective in nature.

Aesthetic and minimalist design

- Dialogues should not contain information which is irrelevant or rarely needed.
- Explanation
- The application should provide aesthetically pleasant interaction, controls, icons, contents and others to the user. So, they can avoid negative UX and feel joyness while performing the task.
- Example
- An auto movement of cursor to the next field rather than scrolling enhances user experience. Similarly, when the user closes application, system should support the user in task accomplishment. But the displayed dialogue box contains additional and irrelevant information in the form advertisement, which enhances negative UX

Aesthetic and minimalist design

- Benefits
- Minimum and aesthetically pleasant interface leads to better performance and increase comfort level.
- Problems
- Different evaluators may have different interpretations due to the subjective nature of this heuristic.



Flexibility and efficiency of use

- System should provide shortcuts and configuration options for naïve and expert users to speed up interaction.
- Explanation
- Experienced users may prefer to configure gestures as per their own requirements (e.g., double tapping on a particular area for direct zoom-in a specific part of the map). Similarly, short keys are available for different operations to speed up interaction. Users can also customize various operations according to requirements. System also provide shortcut to mute all calls, stop vibration of incoming calls, alarms and timers, when the device is flipped.

Flexibility and efficiency of use

- Example:

Enabling the Wi-Fi by just flip the screen from the top and select the Wi-Fi icon. In the second way, Wi-Fi is turned on with some additional steps including “open the setting menu and then turning on Wi-Fi.

- Benefits

Users can accomplish the task efficiently and establish a sense of ownership.

- Problems

Each user has its own requirements and wants to customize accordingly. So, it is a difficult task for designers to accommodate all such users with full capacity.



Handling varied context of use in mobile environments

- Smartphone application should provide a familiar, natural and effective way to interact in different context of use. Interface interaction in all such constraints should be highly acceptable for users.
- Explanation
- Possible constraints are: auditory due to noise, visually due to bad light, shaking hands due to motion, or less focused due to social interaction. An effective interaction in these contexts of use facilitates the user to provide large controls, multi-modal input/output and subtle animation to accomplish task. System should also provide a usable setting of important functions of mobile application especially. It is observed that sometimes, due to movement, text is not readable on UI. It may be due to the use of improper color scheme and/or brightness restricts clear visibility, etc. System should support by putting a dot on some event/part of an application to improve the readability.

Handling varied context of use in mobile environments

- Example

By default, audio sound settings of ring tone is not audible in motion, due to noisy environment. Ring tone should be audible and further support with vibration can enhance UX.

- Benefits

Users perform the task efficiently in varied context of use.

- Problems

Evaluators may get confuse this heuristic with “match between system and the real world”.



Fingertip size controls and Ergonomics

- Smartphone should provide layout of the UI controls with enough margins, located at some recognizable places and fit to use.
- Explanation

While in motion, user mistakenly presses icon/button because of congestion which may lead to another unwanted situation. The system should provide enough margins to get fingertip size control. Holding smartphone with one hand and using the same hand thumb to interact with control elements will provide more ease to the user.



Fingertip size controls and Ergonomics

- Example
- “Save” and “Add” controls for events are very far from the single hand. So it requires second hand to save the event as shown in Figure, which is less ergonomically usable.
- Benefits
- User’s performance increases, when buttons are placed at some recognizable places and definitely easy to use.

- Problems
- One problem when executing this heuristic in the case of users with special needs. Evaluators should analyze each such case separately.

Effective Design to lessen the user’s workload

- System should provide contextual information wherever necessary to the concerned application to provide usable interface to the user.
- Explanation
- As the user’s objective is difficult to determine directly so context cues are used, to help infer this information and to inform an application for how best to support the user. While in motion, it is observed that users engage in multiple activities simultaneously and perform context switching according to the need. This requires more cognitive load to perform the task. Gestalt continuation law provides support to design interface, where one object link up to next object and so on. It supports the user to read properly and help to accomplish the task. For example, send message task has several steps like: open the app, write up the text, and enter/find the contact (receiver) and press the "send" button. In mobility, if these steps link up with each other, users can comfortably accomplish the task.

Effective Design to lessen the user's workload

- Example
- The map application in Figure supports users in trav- eling from one location to other by providing contextual information about the traffic condition, current place and routing information to lessen the workload.
- Benefits
- User gets support in task accomplishment and better under- standing of the system.
- Problems
- Evaluators may get confuse in exercising this heuristic with match between system and the real world.



Recognition rather than recall

- The smartphone should offer visible actions, options and objects in order to avert operators to remember information from one part of the dialog to another.
- Explanation
- System should provide recognizable multi-touch gestures while interacting with mobile application. Users while using double tap, slide, flick, two finger rotation, pinch in/out and many others, feel that these gestures will help to accomplish task in an effective manner. Make it sure to eliminate gesture ambiguity for significant gestures to be more usable according to touch targets, size and placement. Similarly, due to limited short term memory of users, system should also provide support in other parts.

Recognition rather than recall

- Example
- Delete icon (bin) is visible and recognizable for users
- Benefits
- Reduce mental effort and fewer chances for error.
- d: Problems
- Evaluators may get confuse in exercising this heuristic with” effective design to lessen the user’s workload”.



User control and obviousness

- System should facilitate users to protect information by providing undo/redo options and provide clearly an “emergency exits” to leave unwanted positions. These options should be available preferably through a physical button or equivalent.
- Explanation
- System should ensure user’s control and obviousness especially in modification/deletion of any important information. Users can easily manage the application and resource consumption conveniently. System should support users to navigate comfortably across the application.
- Example
- Placing a pattern or fingerprint identification makes it possible that no unauthorized person can access any personal information

User control and obviousness

- Benefits
- Users feeling of ownership can enhance UX and they can perform the task efficiently.
- Problems
- This heuristic provides an insight to repair an error and provides control to users to let it work with their own needs. So, this heuristic may not overlap with “flexibility and efficiency of use”.



Consistency and standards

- The smartphone should follow the well-known conventions, allowing the user to do things in a familiar, standard and consistent way.
- Explanation
- Every application contains similar functions. These similar functions put together with same layout design. This is the reason; users feel comfort and efficiently perform the task. Same options can deploy at different UIs of mobile application. These should also have same designed mechanics to provide easy to use interaction. These conventions and standards should also shadow in different devices of same manufacturers.

Consistency and standards

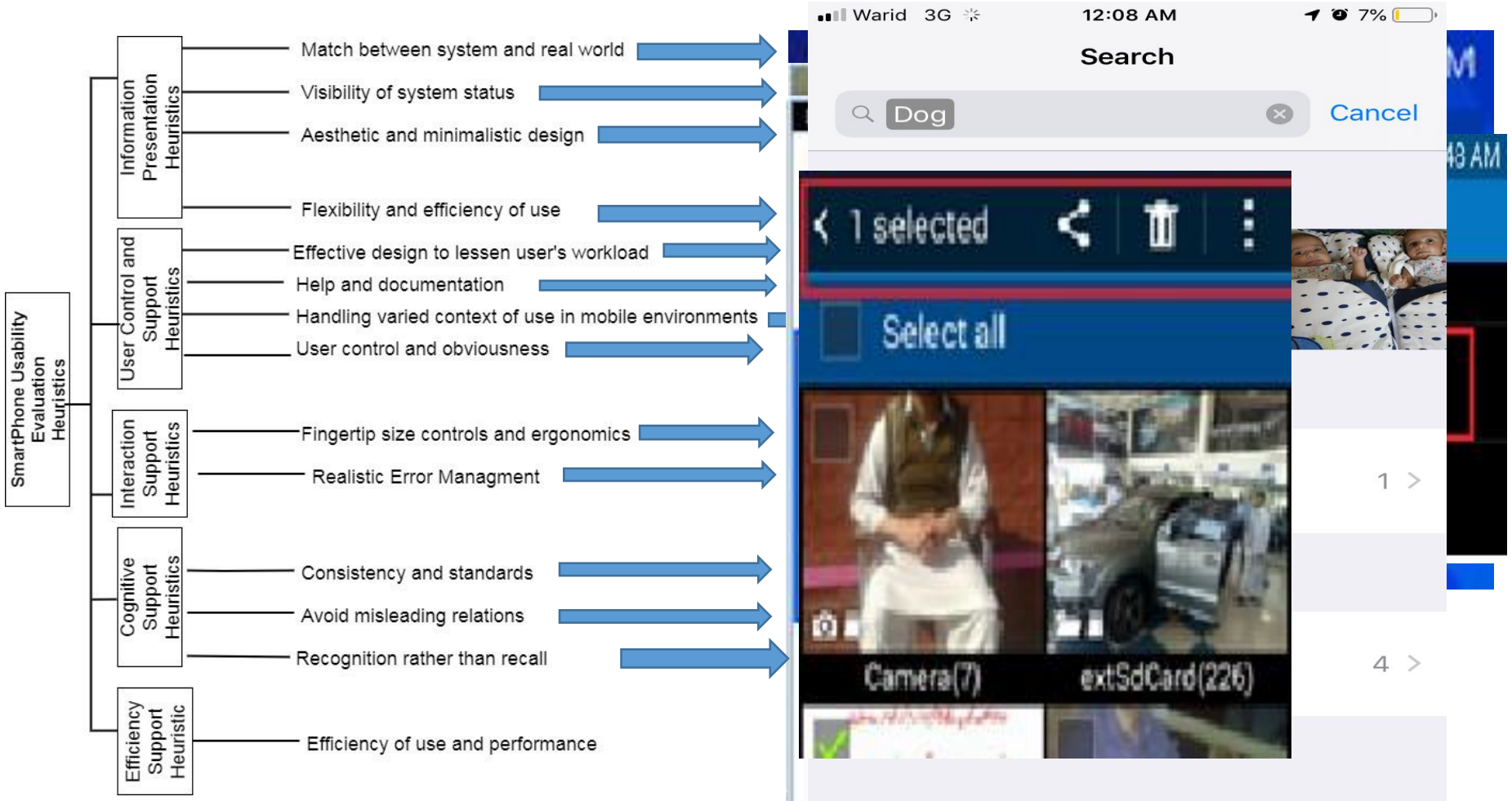
- Example
- For example, Gallery mobile application contains pictures, where we can edit/modify/insert/delete any image.
- Benefits
- Consistent options help users to get familiarity, easy to learn and memorization. This will certainly increase task efficiency, minimize errors and increase user satisfaction.
- Problems
- Evaluators may get confuse in exercising this heuristic with “match between system and the real world.



Framework for Classification of Usability heuristics

- Information Presentation features: This category contains features associated with the smartphone presentation and information.
- Interaction features: This category deals with the interaction between the user and smartphone.
- User's Cognitive features: Category associated with the user's mental model/perception to perform any specific task.
- User's Control and Support: This category contains features that provide control and support to the user during and after completion of any specific task.
- Efficiency related features: This category contains aspects associated with the efficiency of the task accomplishment.

Framework for Classification of Usability heuristics



Classification of smartphone usability Heuristics

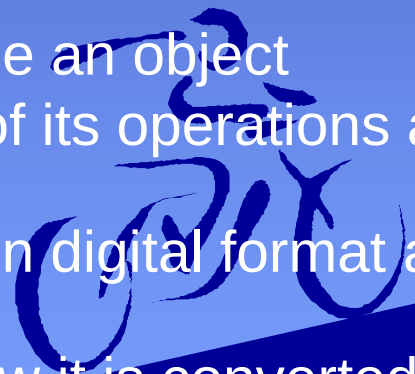
Software Construction and Development

Week-8



Object Oriented Design Principles

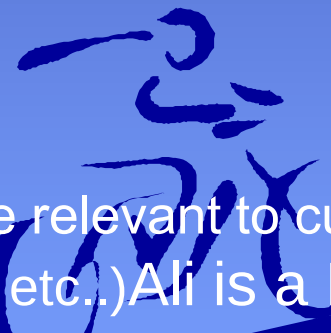
- Information Hiding
  - Information is stored within the object
  - It is hidden from the outside world
  - It can only be manipulated by the object itself
  - Example – Name of a Person, can’t read the numbers directly from the SIM
- Encapsulation
  - Data and behavior are tightly coupled inside an object
  - Information structure and implementation of its operations are hidden from the outer world
  - Example: A Phone stores phone numbers in digital format and knows how to convert it into human-readable characters
  - We don’t know, How the data is stored, How it is converted to human-readable characters



Object Oriented Design Principles

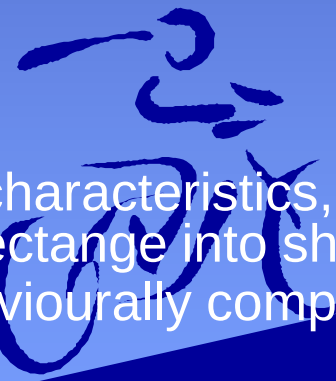
- Object has an Interface
  - An object encapsulates data and behaviour, So how objects interact with each other?
  - Each object provides an interface (operations), Other objects communicate through this interface
  - Example: Interface of a Car (Steer Wheels, Accelerate, Change Gear,Apply Brakes,Turn Lights On/Off)

- Abstraction
  - Abstraction is a way to cope with complexity.
    "Capture only those details about an object that are relevant to current perspective"
  - Example: Ahmad is a Student (StId, StName, GPA etc..)Ali is a PhD student
  - Ali is a PhD student and teaches BS students



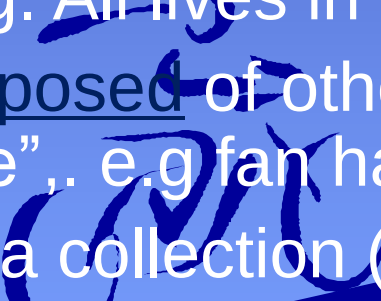
Object Oriented Design Principles

- Inheritance:
  - A child inherits characteristics of its parents, a child may have its own unique characteristics.
  - The parent class is called *base* class and the child class is called *derived* class,
  - Inheritance – “IS A” or “IS A KIND OF” Relationship
  - Example – Person is a parent class and student, teacher are derived classes
  - Reuse
  - Concepts Related with Inheritance
    - Generalization (classes may have common characteristics, extract features into a new class and inherit. Example line, circle, rectange into shape class
    - Subtyping (extension) Derived class is behaviourally compatible with the base class. e.g Shape and circle
    - Specialization (restriction) Derived class is behaviourally incompatible with the base class. e.g. Person and Adult class



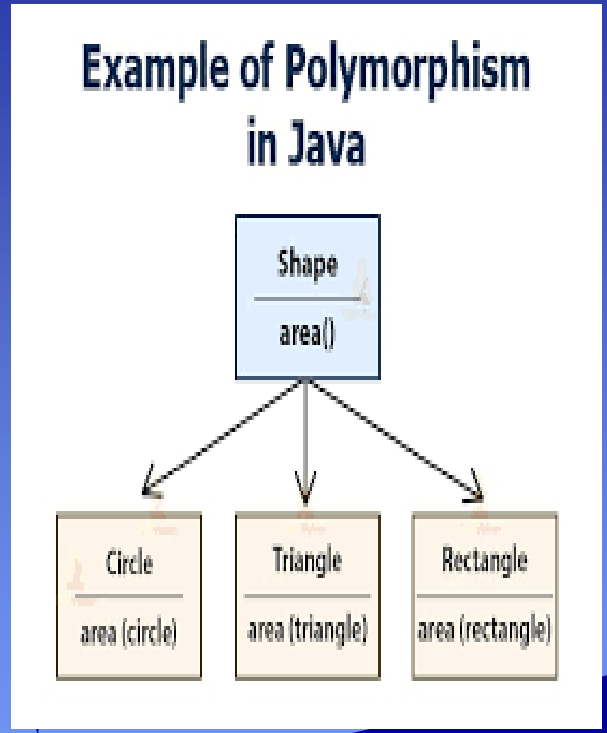
Object Oriented Design Principles

- Association
  - An object keeps associations with other objects to delegate tasks
- Class Association
  - Inheritance
- Object Association
  - Simple Association (weakest link, e.g. Ali lives in a House)
  - Composition (An object may be composed of other smaller objects, “part” objects and the “whole”,. e.g fan has wings
  - Aggregation (An object may contain a collection (aggregate) of other objects, weeker than composition, e.g. Furniture can be shifted to another room



Object Oriented Design Principles

- Polymorphism
  - Overriding (Area function in shape overrides in Circle, Triangle and Rectangle)
  - Abstract Class (Shape and Person)
  - Concrete Classes
  - Polymorphism refers to existence of different forms of a single entity
  - Example, Diamond and Coal are different forms of Carbon
  - polymorphism means that different objects can behave in different ways for the same message (stimulus)
  - Consequently, sender of a message does not need to know exact class of the receiver
  - Example: Area Function

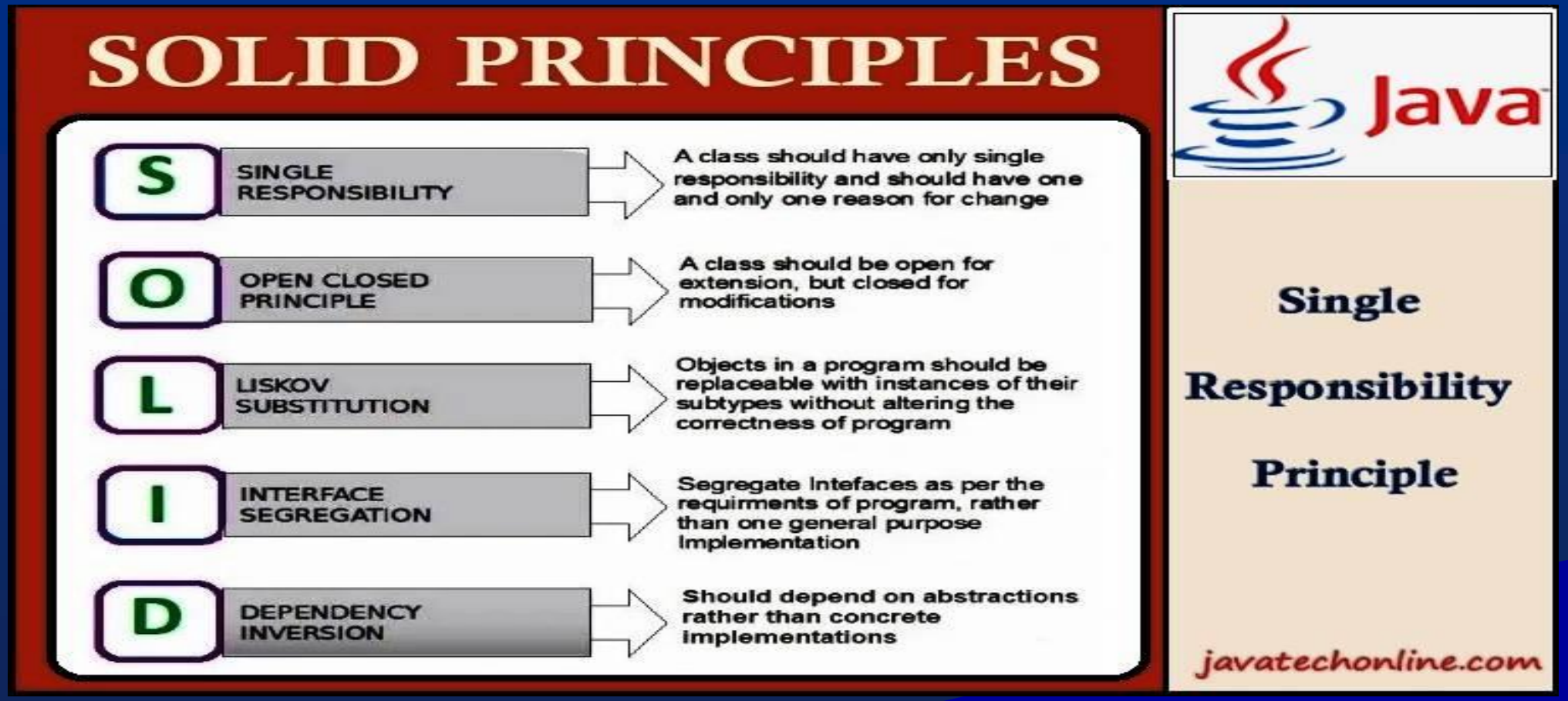


Why need of Software Design Principles?

- To avoid 3 important characteristics of a bad design
  - Rigidity - It is hard to change because every change affects too many other parts of the system.
  - Fragility - When you make a change, unexpected parts of the system break.
  - Immobility - It is hard to reuse in another application because it cannot be disentangled from the current application.

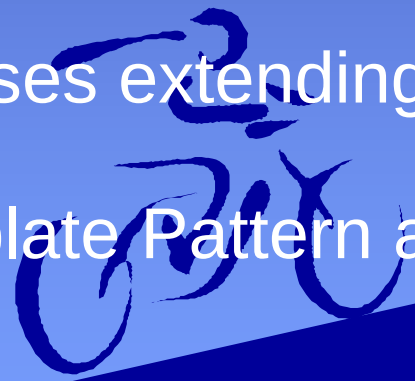


SOLID: Object Oriented Design Principles

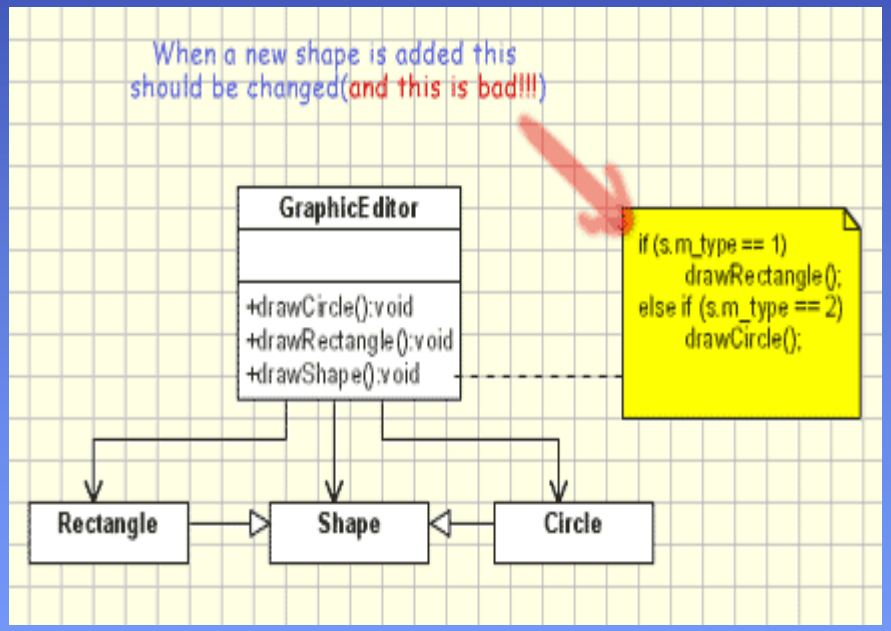


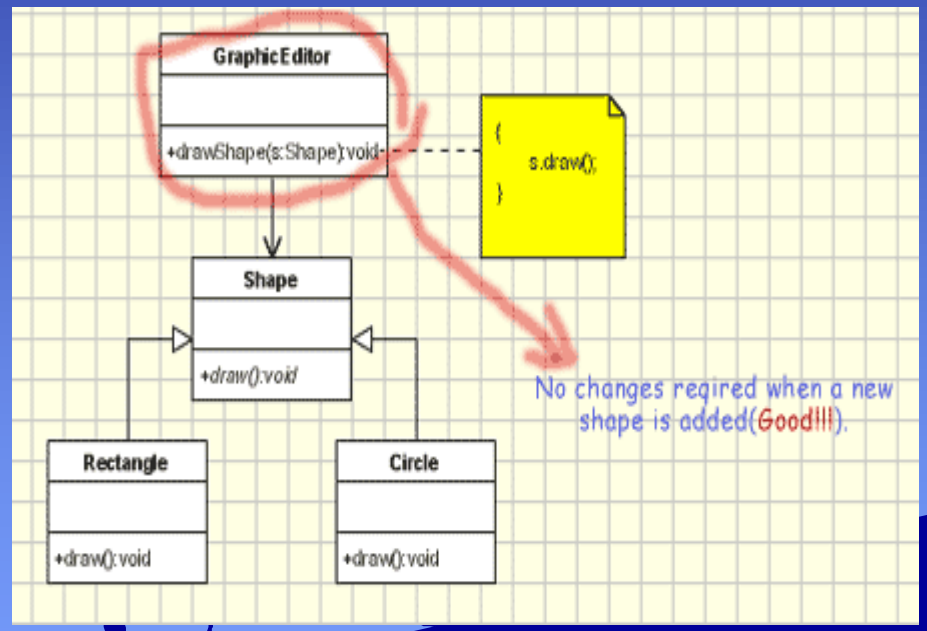
Open Close Principle

- Software entities like classes, modules and functions should be open for extension but closed for modifications.
- OC is a generic principle.
- The same principle can be applied for modules, packages, libraries.
- When referring to the classes Open Close Principle can be ensured by use of Abstract Classes and concrete classes for implementing their behavior.
- This will enforce having Concrete Classes extending Abstract Classes instead of changing them.
- Some particular cases of this are Template Pattern and Strategy Pattern.



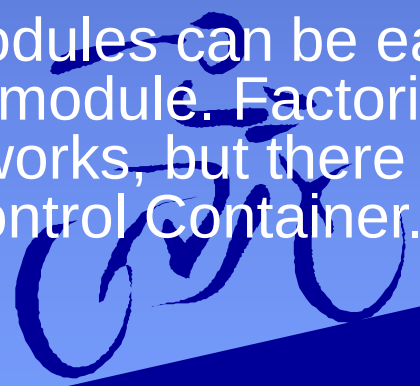
Open Close Principle



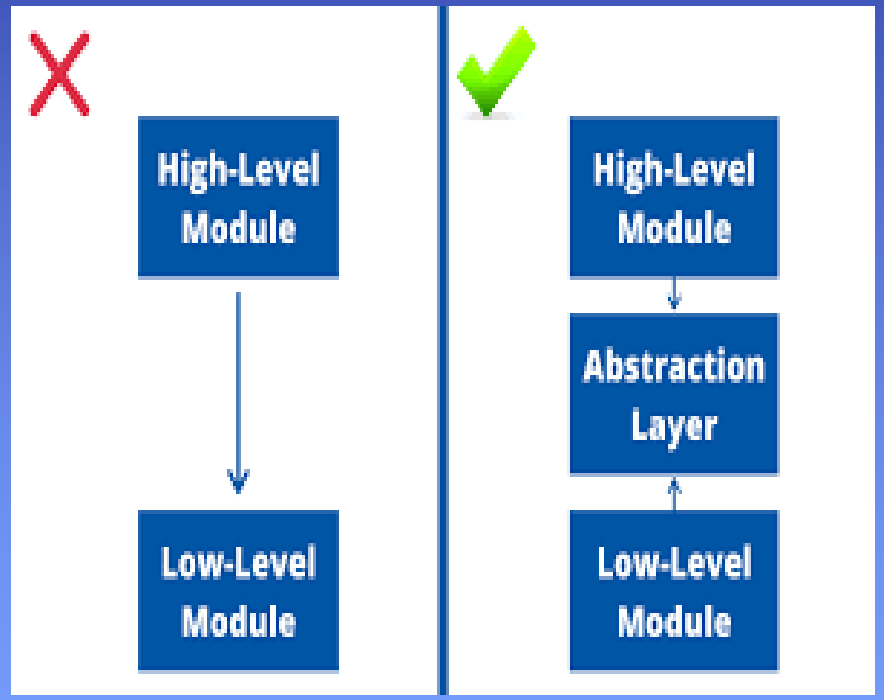


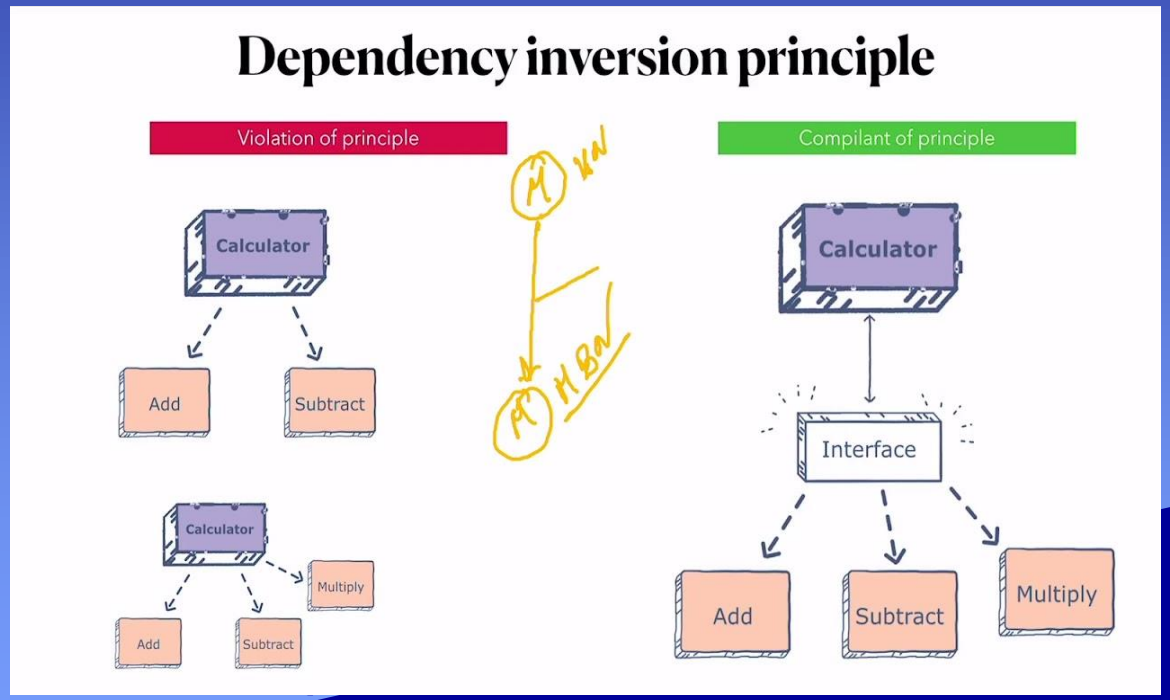
Dependency Inversion Principle

- High-level modules should not depend on low-level modules. Both should depend on abstractions. Abstractions should not depend on details. Details should depend on abstractions.
- Dependency Inversion Principle states that we should decouple high level modules from low level modules, introducing an abstraction layer between the high level classes and low level classes.
- Further more it inverts the dependency: instead of writing our abstractions based on details, the we should write the details based on abstractions.
- By applying the Dependency Inversion the modules can be easily changed by other modules just changing the dependency module. Factories and Abstract Factories can be used as dependency frameworks, but there are specialized frameworks for that, known as Inversion of Control Container.



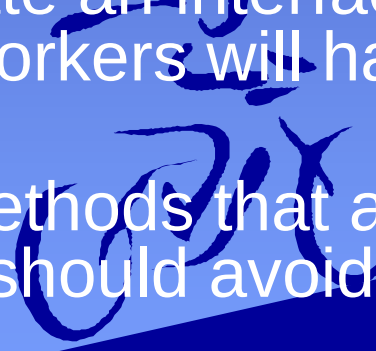
Dependency Inversion Principle



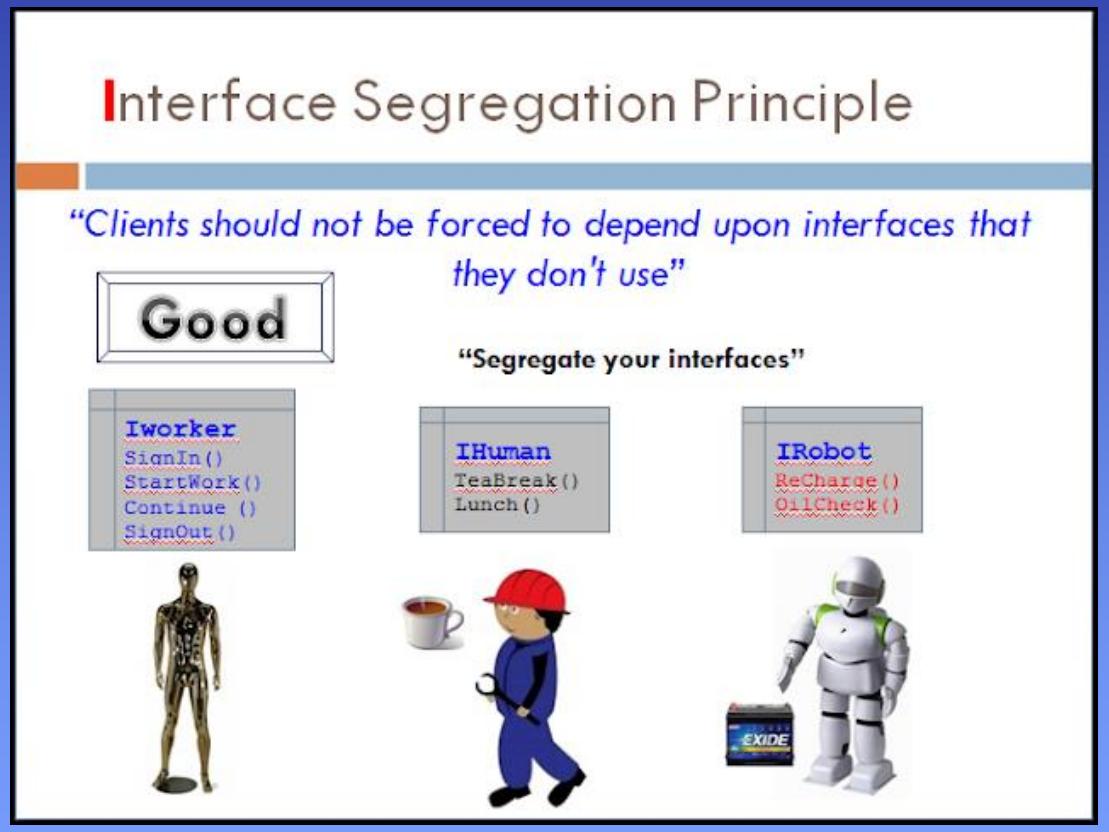


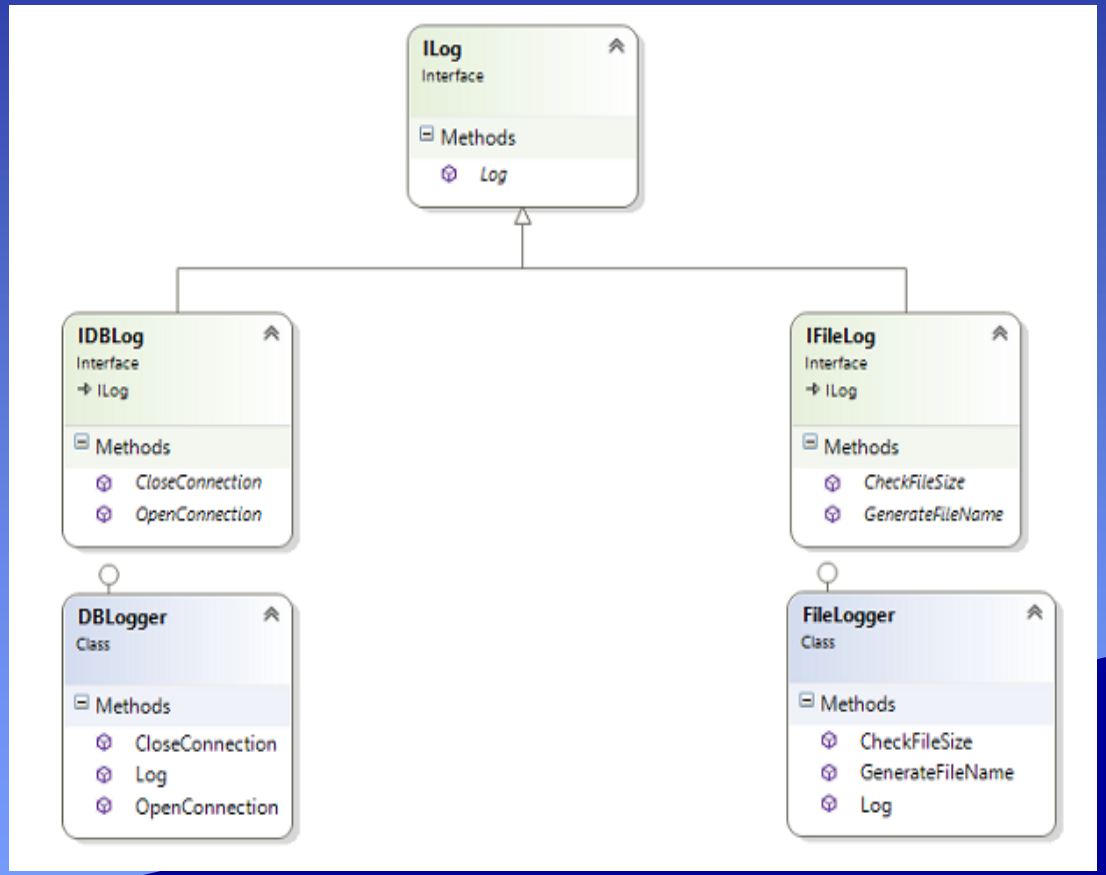
Interface Segregation Principle

- Clients should not be forced to depend upon interfaces that they don't use.
- This principle teaches us to take care how we write our interfaces. When we write our interfaces we should take care to add only methods that should be there. If we add methods that should not be there the classes implementing the interface will have to implement those methods as well. For example if we create an interface called Worker and add a method lunch break, all the workers will have to implement it. What if the worker is a robot?
- As a conclusion Interfaces containing methods that are not specific to it are called polluted or fat interfaces. We should avoid them.



Interface Segregation Principle



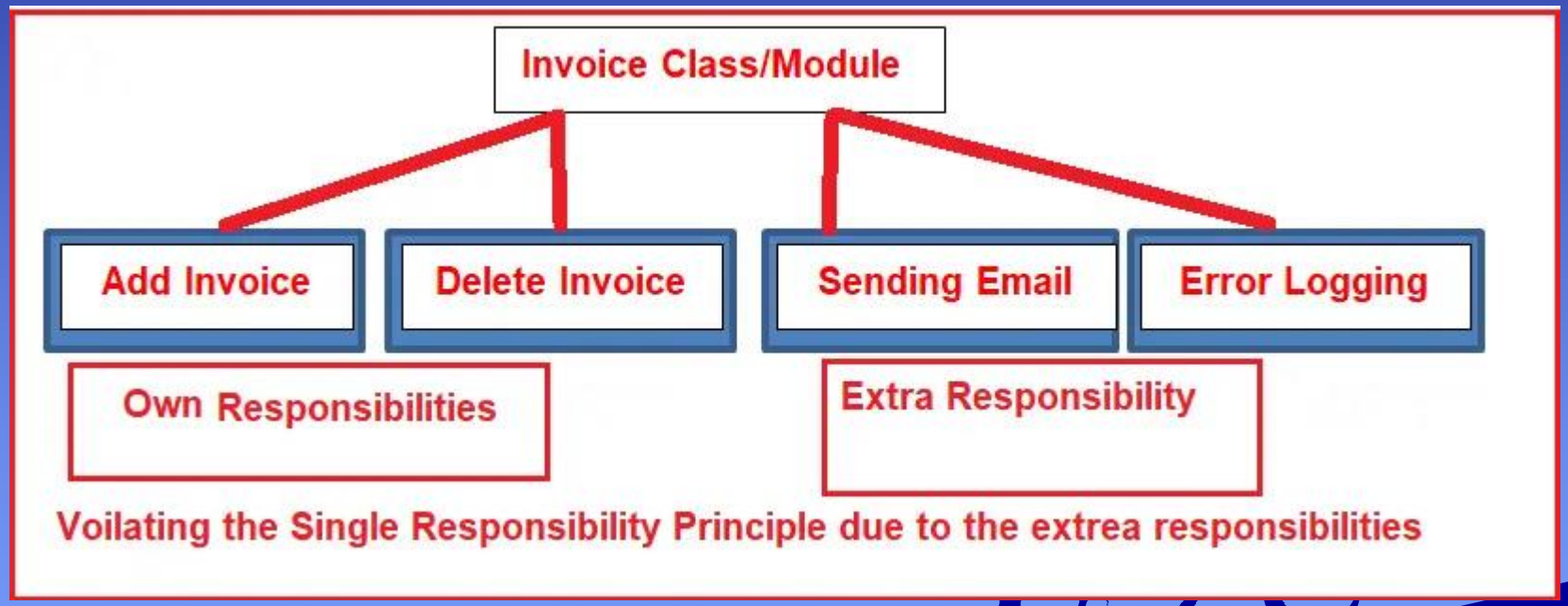


Single Responsibility Principle

- A class should have only one reason to change.
- In this context a responsibility is considered to be one reason to change. This principle states that if we have 2 reasons to change for a class, we have to split the functionality in two classes. Each class will handle only one responsibility and on future if we need to make one change we are going to make it in the class which handle it. When we need to make a change in a class having more responsibilities the change might affect the other functionality of the classes.
- Single Responsibility Principle was introduced Tom DeMarco in his book Structured Analysis and Systems Specification, 1979. Robert Martin reinterpreted the concept and defined the responsibility as a reason to change.

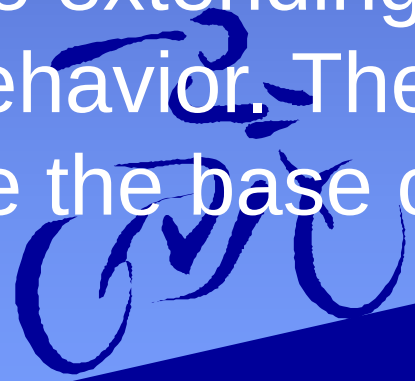


Single Responsibility Principle

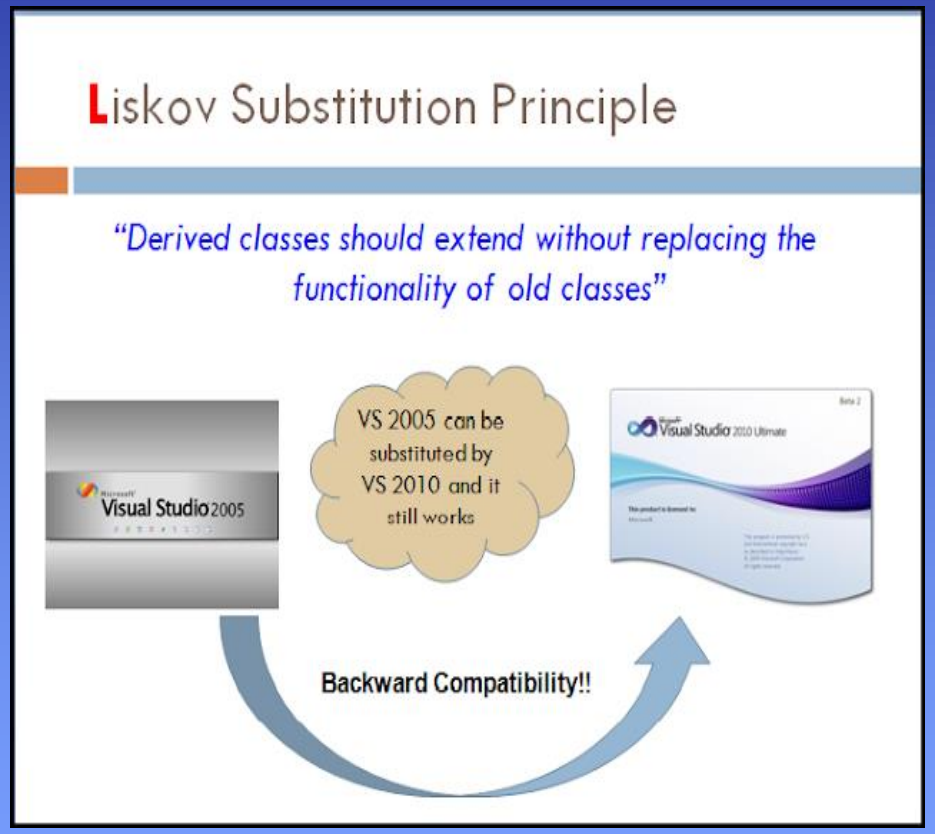


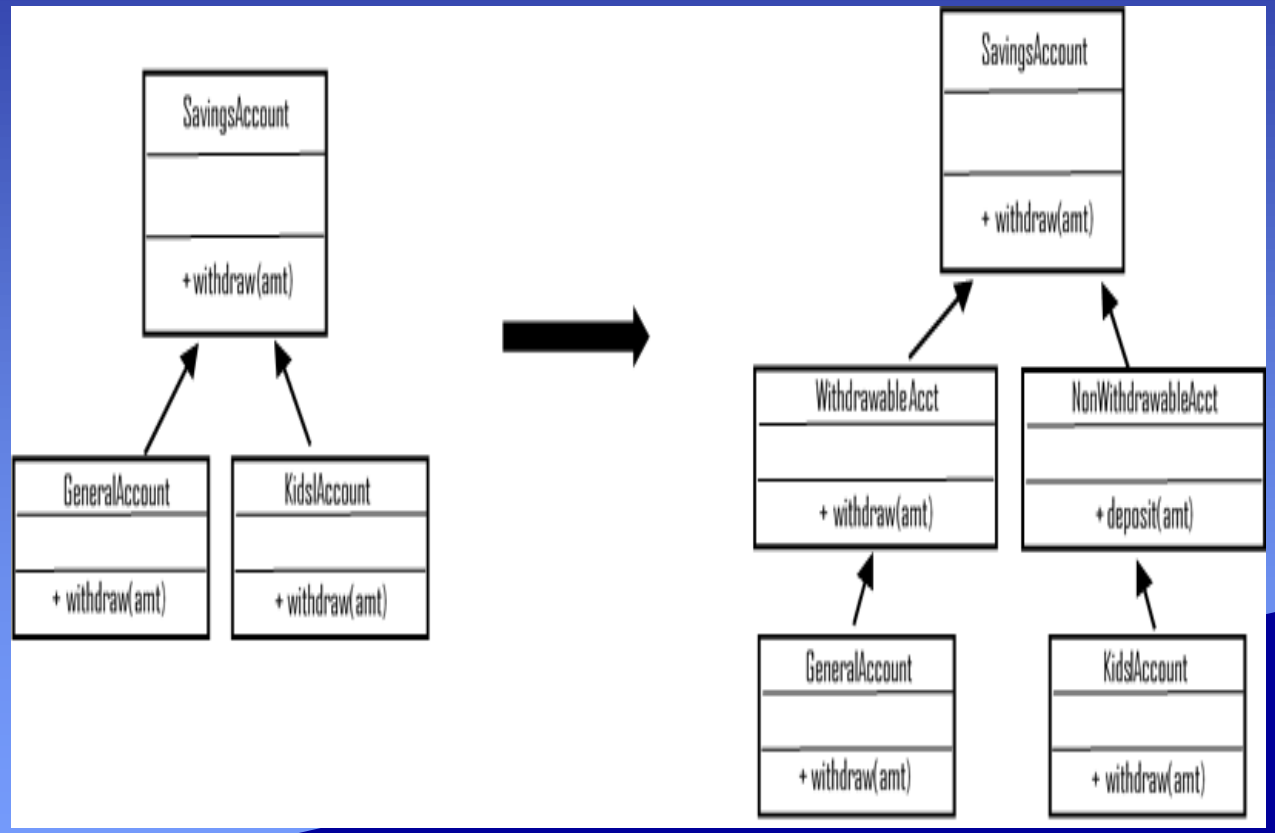
Liskov's Substitution Principle

- Derived types must be completely substitutable for their base types.
- This principle is just an extension of the Open Close Principle in terms of behavior meaning that we must make sure that new derived classes are extending the base classes without changing their behavior. The new derived classes should be able to replace the base classes without any change in the code.



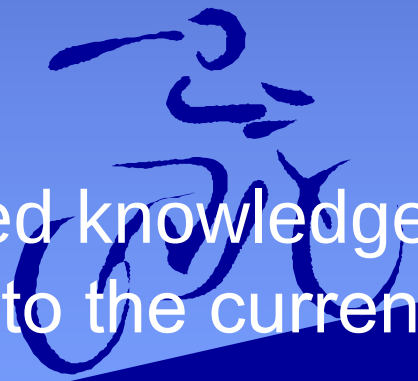
Liskov's Substitution Principle





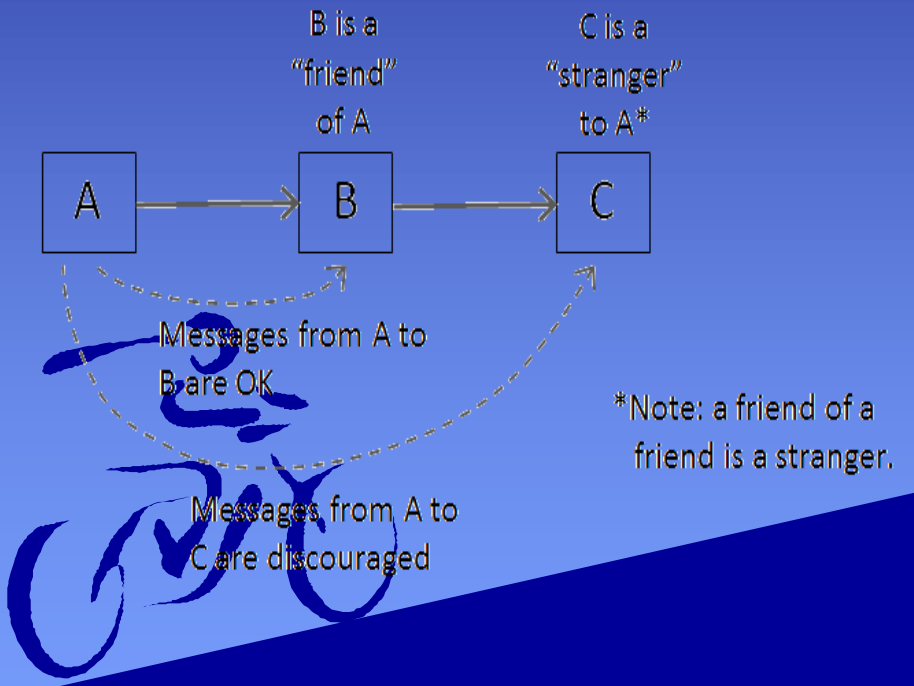
Principle of least knowledge

- The Law of Demeter (LoD) or principle of least knowledge is a design guideline for developing software, particularly object-oriented programs.
- Demeter's law is known as "don't talk to strangers" because:
  - Each unit should have only limited knowledge about other units — only units "closely" related to the current unit.
  - Each unit should only talk to its friends — don't talk to strangers.



Principle of least knowledge

- More formally, the Law of Demeter requires that a method m of an object O may only invoke the methods of the following kinds of objects:
- O itself.
- m's parameters.
- Any objects instantiated within m.
- O's direct component objects.
- A global variable, accessible by O, in the scope of m.



Object-Oriented Modeling

An Example



Problem Statement

- Develop a graphic editor that can draw different geometric shapes such as line, circle and triangle. User can select, move or rotate a shape. To do so, editor provides user with a menu listing different commands. Individual shapes can be grouped together and can behave as a single shape.



Identify Classes

- Extract nouns in the problem statement

- Develop a graphic editor that can draw different geometric shapes such as line, circle and triangle. User can select, move or rotate a shape. To do so, editor provides user with a menu listing different commands. Individual shapes can be grouped together and can behave as a single shape.



…Identify Classes

- Eliminate irrelevant classes
- Editor – Very broad scope
- User – Out of system boundary



…Identify Classes

- Add classes by analyzing requirements

- Group – required to behave as a shape
  - "Individual shapes can be grouped together and can behave as a single shape"

- View – editor must have a display area



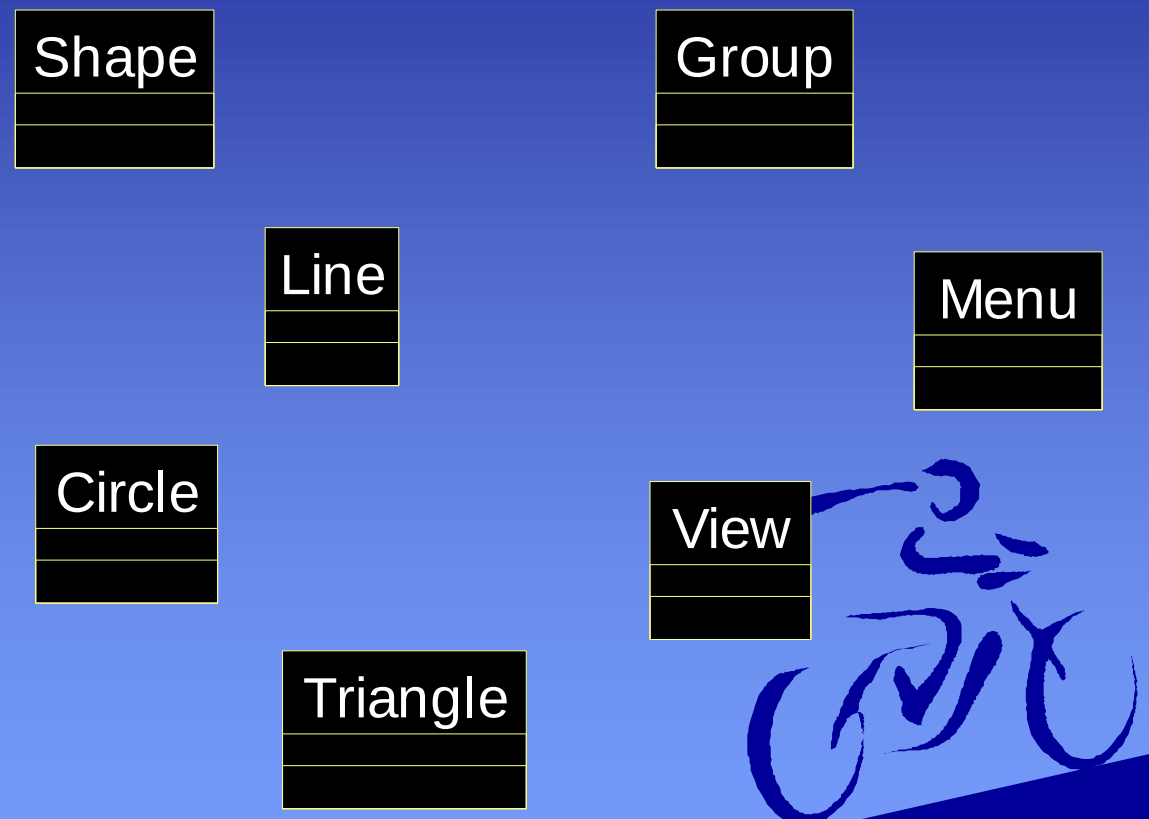
…Identify Classes

➢ Following classes have been identified:

- Shape
- Line
- Circle
- Triangle
- Menu
- Group
- View



Object Model – Graphic Editor



Identify Associations

- Extract verbs connecting objects

- “Individual shapes can be grouped together”
  – Group consists of lines, circles, triangles
  – Group can also consists of other groups (Composition)



… Identify Associations

➢ Verify access paths

- View contains shapes
  - View contains lines
  - View contains circles
  - View contains triangles
  - View contains groups (Aggregation)



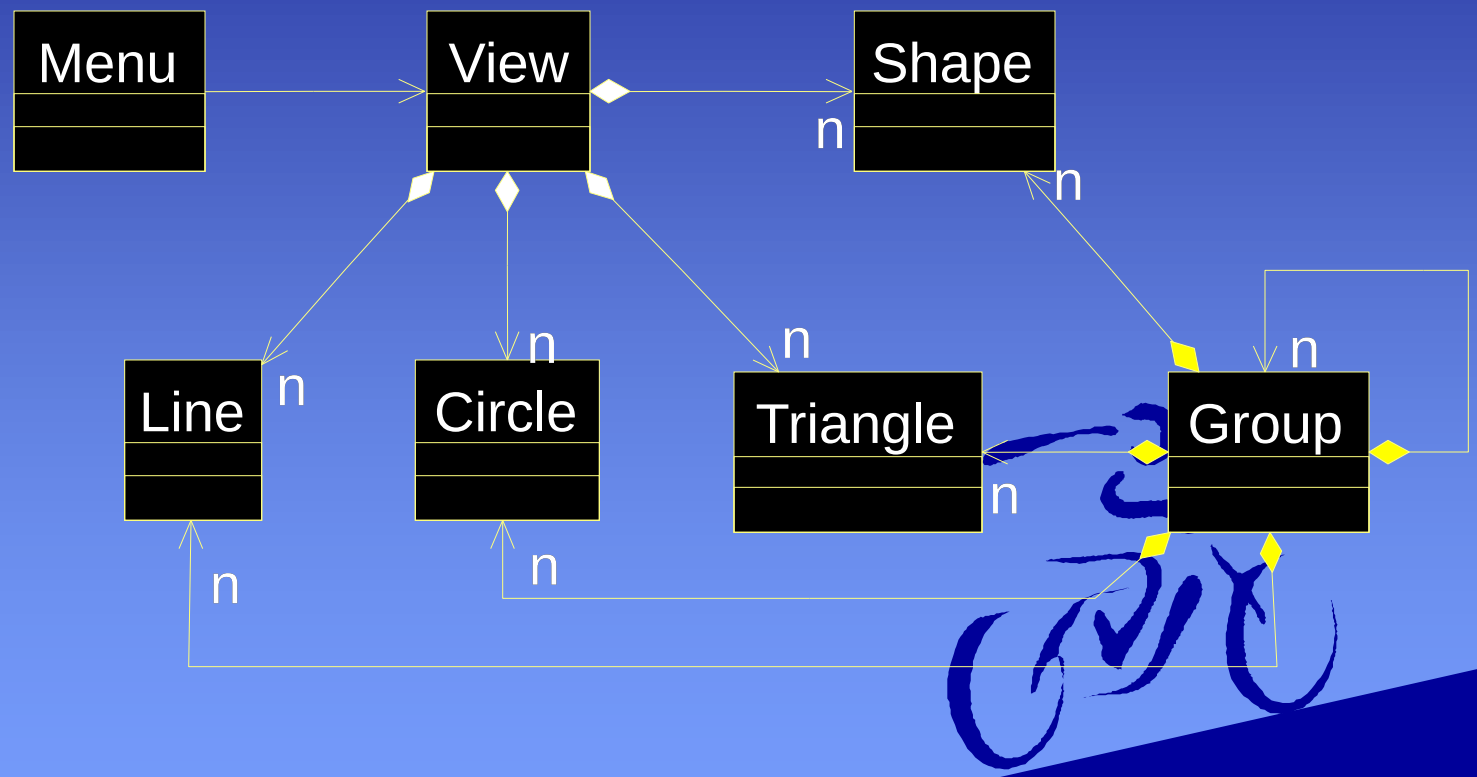
… Identify Associations

- Verify access paths

- Menu sends message to View
  (Simple One-Way Association)



Object Model – Graphic Editor



Identify Attributes

- Extract properties of the object
  - From the problem statement

- Properties are not mentioned



…Identify Attributes

- Extract properties of the object
  - From the domain knowledge

- Line
  - Color
  - Vertices
  - Length
- Circle
  - Color
  - Vertices
  - Radius
- Triangle
  - Color
  - Vertices
  - Angle
- Shape
  - Color
  - Vertices



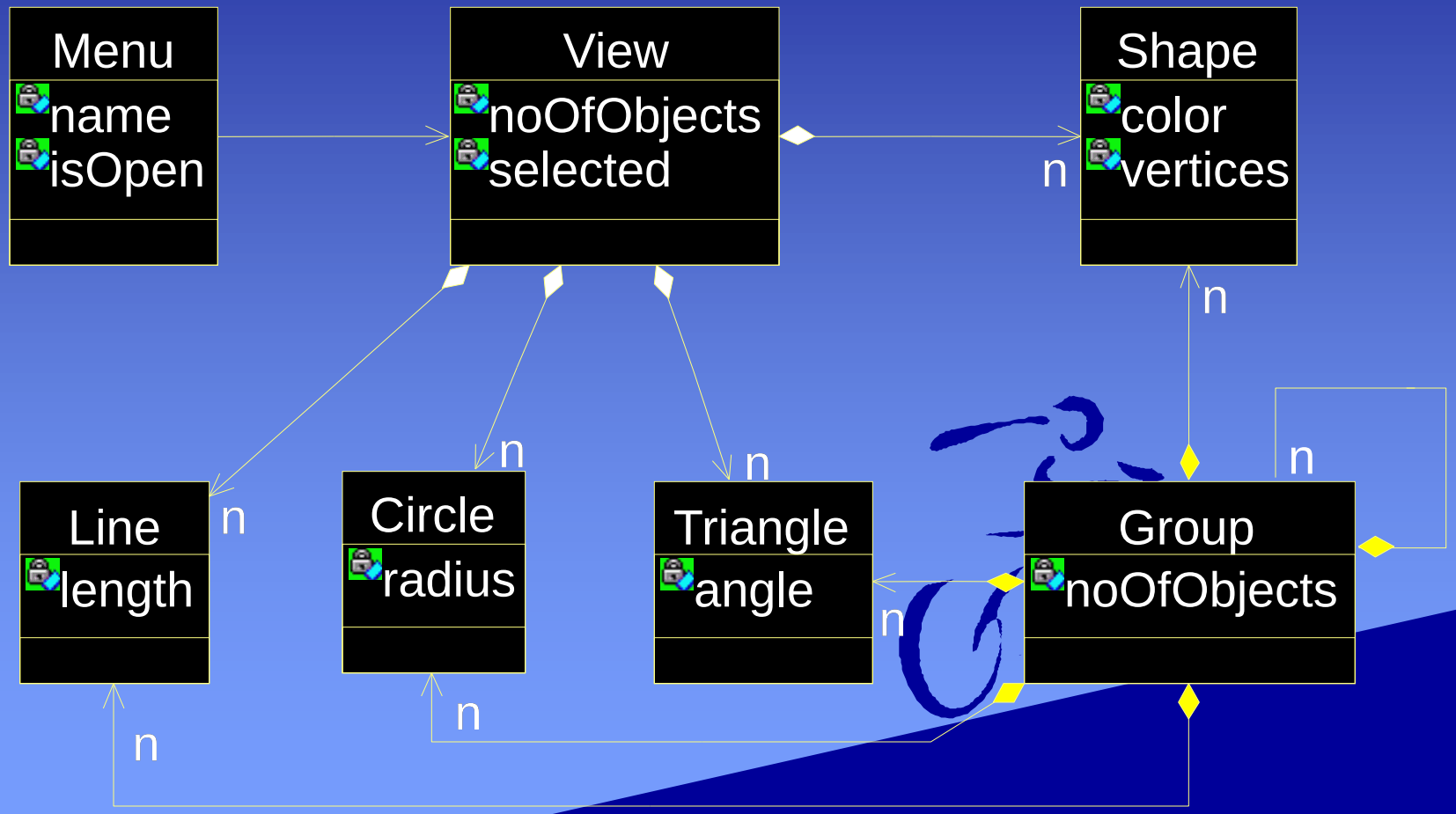
…Identify Attributes

- Extract properties of the object
  - From the domain knowledge

- Group
  - noOfObjects
- View
  - noOfObjects
  - selected
- Menu
  - Name
  - isOpen



Object Model – Graphic Editor



Identify Operations

- Extract verbs connected with an object

- Develop a graphic editor that can draw different geometric shapes such as line, circle and triangle. User can select, move or rotate a shape. To do so, editor provides user with a menu listing different commands. Individual shapes can be grouped together and can behave as a single shape.

… Identify Operations

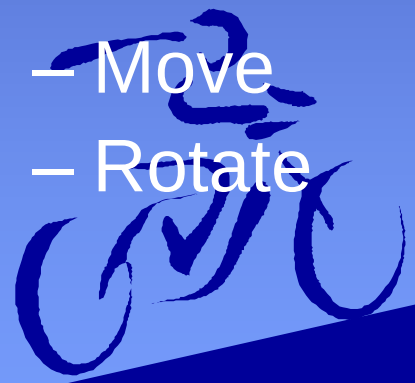
- Eliminate irrelevant operations
- Develop – out of system boundary
- Behave – have broad semantics



…Identify Operations

➢ Following are selected operations:

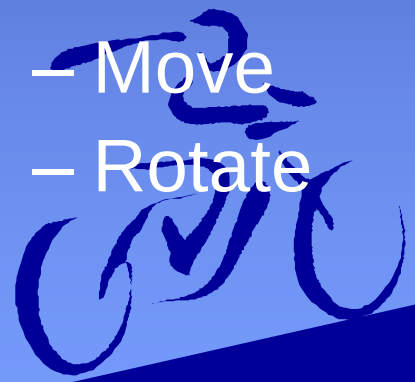
- Line
  - Draw
  - Select
  - Move
  - Rotate
- Circle
  - Draw
  - Select
  - Move
  - Rotate



…Identify Operations

➢ Following are selected operations:

- Triangle
  - Draw
  - Select
  - Move
  - Rotate
- Shape
  - Draw
  - Select
  - Move
  - Rotate



…Identify Operations

➢ Following are selected operations:

- Group
  - Draw
  - Select
  - Move
  - Rotate
- Menu
  - Open
  - Select
  - Move
  - Rotate

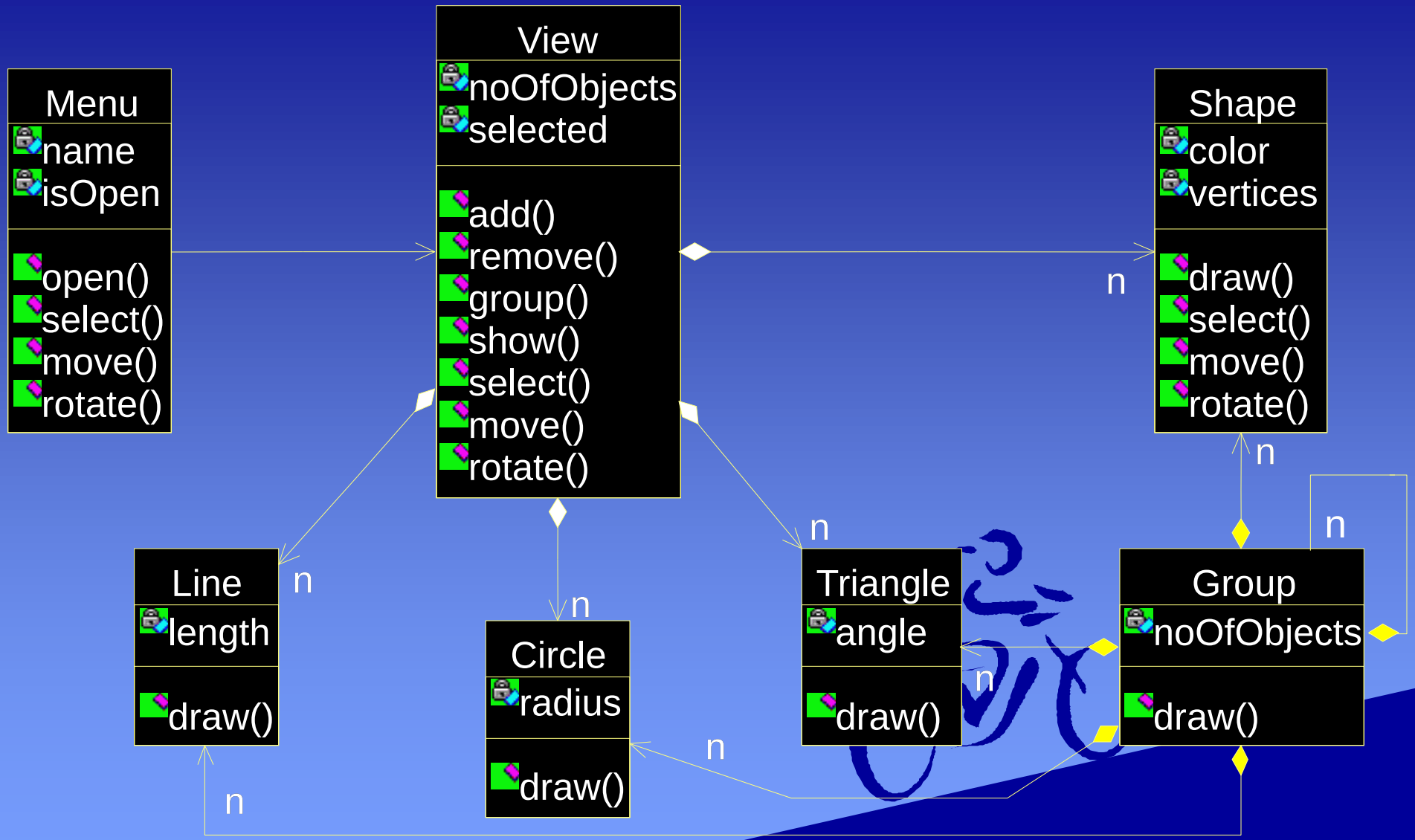


…Identify Operations

- Extract operations using domain knowledge

- View
  - Add
  - Remove
  - Group
  - Show
  - Select
  - Move
  - Rotate




Menu
name
isOpen
open()
select()
move()
rotate()
View
noOfObjects
selected
add()
remove()
group()
show()
select()
move()
rotate()
Shape
color
vertices
draw()
select()
move()
rotate()
n
n
Line
length
draw()
n
n
Circle
radius
draw()
n
n
Triangle
angle
draw()
n
n
Group
noOfObjects
draw()
n
n

Identify Inheritance

- Search “is a kind of” by looking at keywords like “such as”, “for example”, etc

- “…shapes such as line, circle and triangle…”
  - Line, Circle and Triangle inherits from Shape



…Identify Inheritance

- By analyzing requirements

- "Individual shapes can be grouped together and can behave as a single shape"
  - Group inherits from Shape



Refining the Object Model

- Application of inheritance demands an iteration over the whole object model

- In the inheritance hierarchy,
  - All attributes are shared
  - All associations are shared
  - Some operations are shared
  - Others are overridden



…Refining the Object Model

- Share associations
- View contains all kind of shapes
- Group consists of all kind of shapes



…Refining the Object Model

- Share attributes

- Shape – Line, Circle, Triangle and Group
  - Color, vertices



…Refining the Object Model

- Share operations

- Shape – Line, Circle, Triangle and Group
  - Select
  - Move
  - Rotate

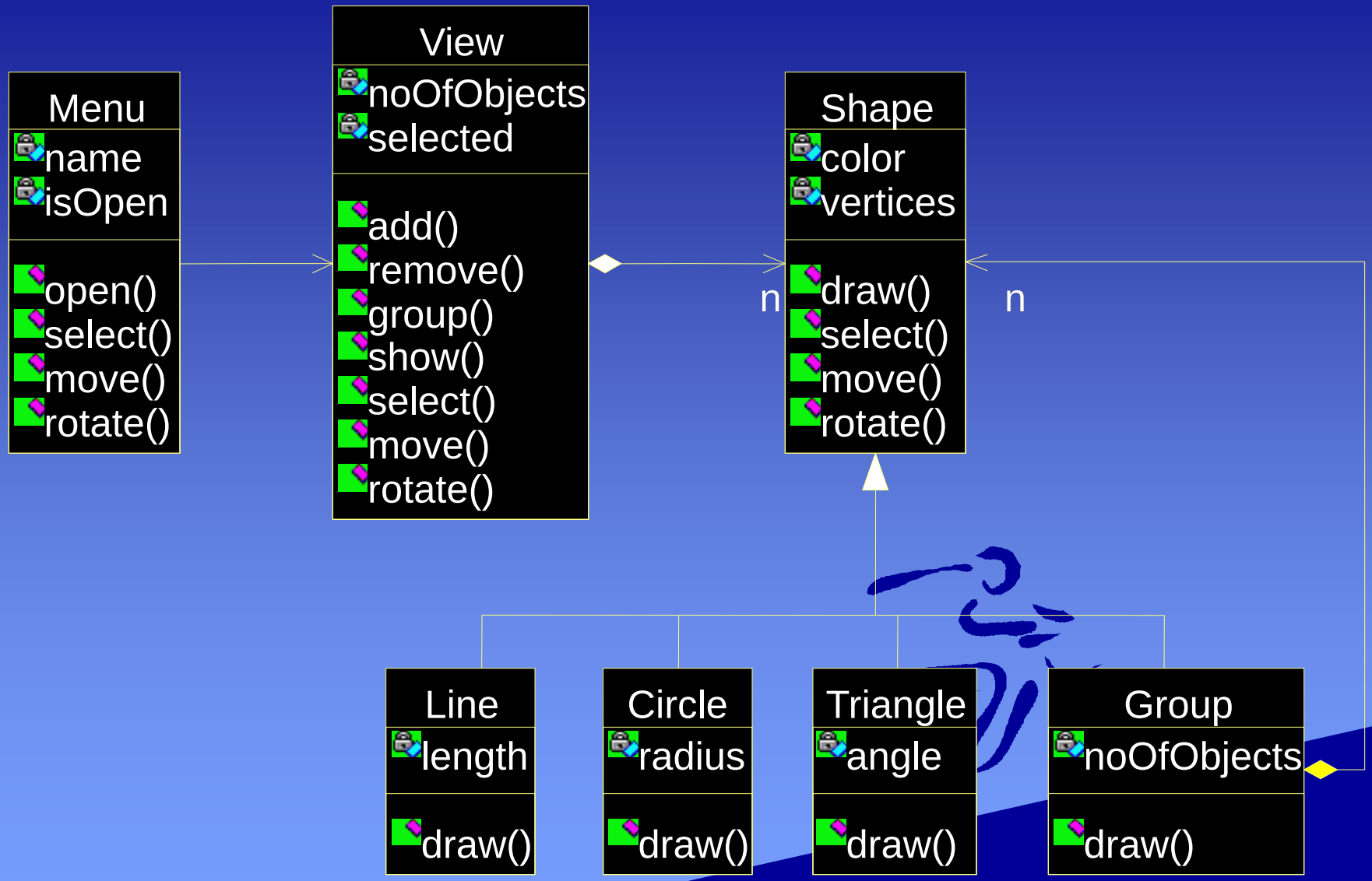


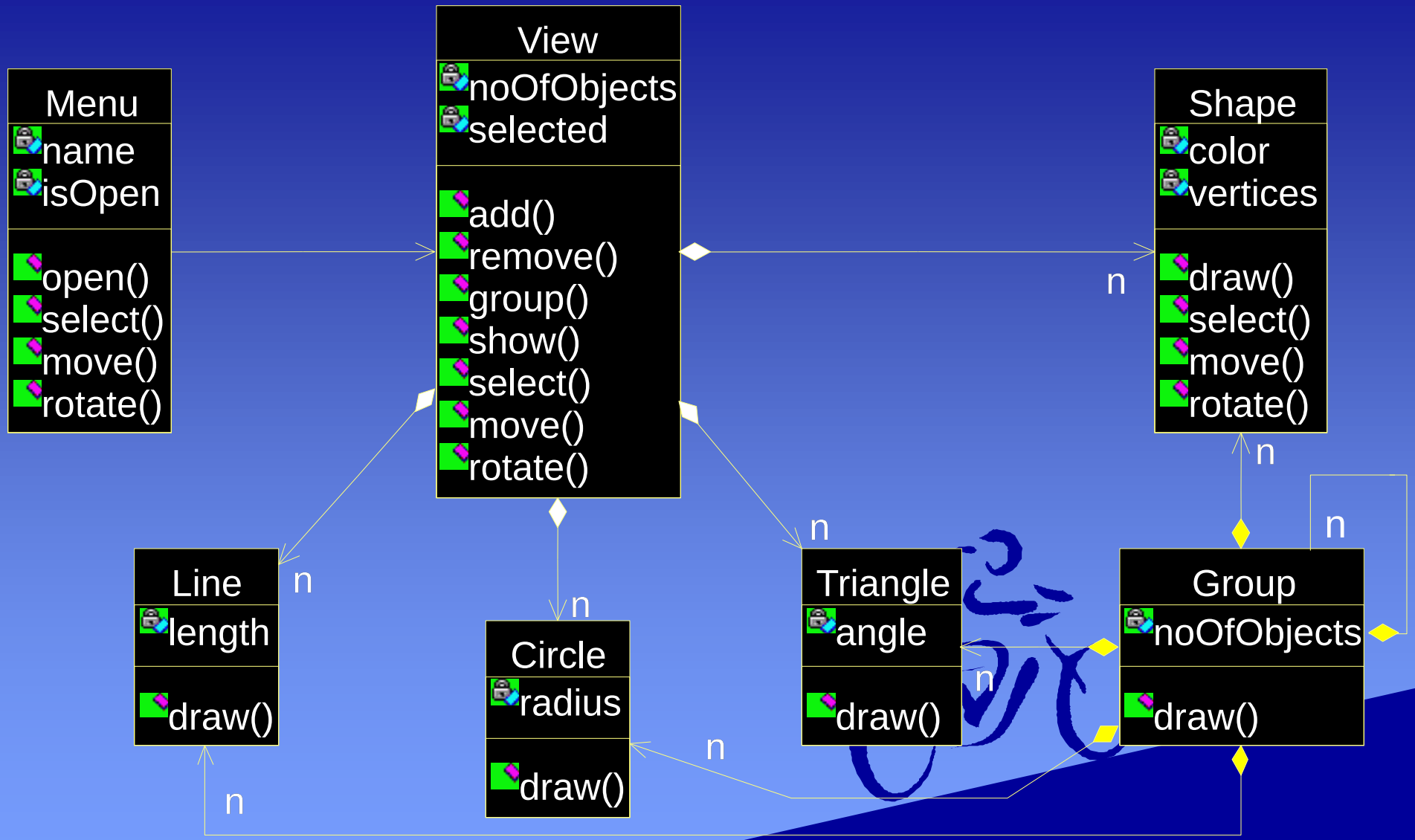
…Refining the Object Model

- Share the interface and override implementation

- Shape – Line, Circle, Triangle and Group
  - Draw




Menu
name
isOpen
open()
select()
move()
rotate()
View
noOfObjects
selected
add()
remove()
group()
show()
select()
move()
rotate()
n
Shape
color
vertices
draw()
select()
move()
rotate()
n
Line
length
draw()
Circle
radius
draw()
Triangle
angle
draw()
Group
noOfObjects
draw()


Menu
name
isOpen
open()
select()
move()
rotate()
View
noOfObjects
selected
add()
remove()
group()
show()
select()
move()
rotate()
Shape
color
vertices
draw()
select()
move()
rotate()
n
n
Line
length
draw()
n
n
Circle
radius
draw()
n
n
Triangle
angle
draw()
n
n
Group
noOfObjects
draw()
n
n

Week 9

Coding Standard/Guidelines

Source : https://www.csee.umbc.edu/courses/undergraduate/341/fall16/projects/coding-standards.shtml

What is a coding standard

- A set of guidelines that recommend programming style, practices, and methods for each aspect of a program

- Usually covers
  - Naming Conventions : Variable, Function , Class Names, File Names
  - White spaces
  - Indentation, Bracket Placement
  - Comments
  - etc. etc..

Why need Coding Standards?

- Help improve the readability of the source code
- Make software maintenance easier
- Less Bugs
- Team Work
- Reduces the cost of maintenance
- Task Automation (helps the scripts to read code better)

File Naming

- For every project, a file named Driver.cpp containing only the main( ) function is required.
- Executable should be named driver.out
- All C++ classes are defined in a separate header file.
  - File name should be Classname.h
  - Implement only getters/setters in the header
- All C++ classes are implemented in a separate source code file
  - File name should be Classname.cpp
  - Implement all the other required functions here

Class Definition Standards

- All class names begin with uppercase
- Only one private, protected and public section in a class definition.
  - public comes first, followed by protected and then private.
- Class methods must have complete function header comments
  - Except getters/setters
  - Class methods must be const whenever possible
- Class data members name **MAY** begin with m_
  - int m_length;

Variable & Function Naming

- Use meaningful descriptive variable names
  - float radius instead of float r;
- Use lowerCamel case for variables
  - int numStudents instead of int num_of_students
- Function MAY begin names with uppercase letters
  - GetName() or getName()

- function prototypes must include parameter names as well as types
- Default values for parameters must be specified in the prototype

Avoid Magic Numbers

- Constants should be used for magic numbers and strings whenever possible
  - const double PI = 3.14159

Wrong

```
int array[30];

for (int i = 0; i < 30; i++)
    array[i] = -3;

if (command == "ADD")
   // do something
else if (command == "MULTIPLY")
  // do something else
```

```
//  a magic number

//  here it is again
//  what does -3 mean?

// magic string may change someday

// another magic string
```

Right

```
const int ARRAYSIZE = 30;
const int INITIALVALUE = -3;
const string ADD = "ADD";
const string MULTIPLY = "MULTIPLY";

    int array[ARRAYSIZE];

    for (int i = 0; i < ARRAYSIZE; i++)
        array[i] = INITIALVALUE;

    if (command == ADD)
       // do something
    else if (command == MULTIPLY)
      // do something else
```

Indentation, Spacing

- Use blank lines to separate pieces of code for readability to separate unrelated sections of code

Wrong

```
while (moreStudents == true) {
    // read student last name
    ReadStudentName();
    // print full name
    PrintStudentName();
    // now verify student info
    goodInfo = VerifyStudentInfo(student);
    if (goodInfo == true)
       cout << "good info" << endl;
}
```

Right

```
while (moreStudents == true)
{
    // read student last name
    ReadStudentName();

    // print full name
    PrintStudentName();

    // now verify student info
    goodInfo = VerifyStudentInfo(student);
    if (goodInfo == true)
       cout << "good info" << endl;
}
```

Indentation, Spacing

- Use a **tab** or **4** spaces for each level of indentation
- Use spaces around all operators.

Wrong	Right
x=y+8;	x = y + 8;

If (x>y)
{statement1;}
else
{statement 2;}
	If (x > y)
	{
	statement1;
	}
	else
	{
	statement 2;
	}

Braces Recommended Styles

K & R Style

```
int power(int x, int y)
{
    int result;

    if (y < 0) {
        result = 0;
    } else {
        result = 1;
        while (y-- > 0) {
            result *= x;
        }
    }

    return result;
}
```

Allman/BSD Style

```
int power(int x, int y)
{
    int result;

    if (y < 0)
    {
        result = 0;
    }
    else
    {
        result = 1;
        while (y-- > 0)
        {
            result *= x;
        }
    }

    return result;
}
```

Brace Usage

- Use a consistent style for braces
- Braces are required for single statement if/else/while/for structures

Wrong

```
if (grade > 90)
        cout << "You got an A" << endl;
  else
        cout << "No A for you" << endl;

  for (i = 0; i < 30; i++)
        array[i] = -3;
```

Right

```
if (grade > 90)
{
      cout << "You got an A" << endl;
}
else
{
      cout << "No A for you" << endl;
}

for (i = 0; i < 30; i++)
{
      array[i] = -3;
}
```

Comments

- Comments are the programmer's main source of documentation.
- **Rule of Thumb:**
  - Approximately every 5 lines of code need AT LEAST 1 comment.
  - Not every line of code needs a comment.
  - Constant declarations MUST have 1 comment.

  - Every cpp and .h file should contain File header comment
  - Every function should contain function header comment.

File Header Comments

EVERY .cpp and .h file should contain an opening comment describing the contents of the file And **MUST** include the following information.

- The file name
- The project number
- Your name
- The date the file was created
- Your section number
- Your tamu e-mail address
- A description of what the code in the file does

```
/**********************************************************
** File     :   Driver.cpp
** Project  :   CSCE 221 Project 1, Spring 2019
** Author   :   Bob Smith
** Date     :   1/22/19
** Section  :   0104
** E-mail   :   bsmith32@tamu.edu
**
**  This file contains the main driver program for Project 1.
**  This program reads the file specified as the first command line
**  argument, counts the number of words, spaces, characters, and
**  displays the results in the format specified in the project description.
**
**********************************************************/
```

Function Header Comments

- Each FUNCTION and CLASS METHOD (Except getters/setters) must have a header comment that includes the following information
    1. The function's name
    2. The function's ***pre-condition(s)*** (if there are no pre-conditions, say so).
    3. The function's ***post-condition(s)***.

Circle.h

```
//--------------------------------------------------------
// Name: CircleArea
// PreCondition:  the radius is greater than zero
// PostCondition: Returns the calculated area of the circle
//--------------------------------------------------------
double CircleArea (double radius ):
```

Circle.cpp

```
// CircleArea
// Given the radius, returns the area
double CircleArea ( double radius )
{
   const double PI = 3.14159;

   // handle unmet precondition
   if (radius < 0.0)
      return 0.0;
   else
      return PI * radius * radius;
}
```

Inline Comments

- MUST appear above the code to which it applies and is indented to the same level as the code

Wrong

```
for (i = 0; i < ARRAYSIZE; i++)  // check every element of the array
{
    if (array[i] % 2 == 1)    // add one to odd values
    {
        array[i]++;
    }
}
```

Right

```
// check every element of the array
for (i = 0; i < ARRAYSIZE; i++)
{
    // add one to odd values
    if (array[i] % 2 == 1)
    {
        array[i]++;
    }
}
```

References

- https://www.csee.umbc.edu/courses/undergraduate/341/fall16/projects/coding-standards.shtml
- https://en.wikipedia.org/wiki/Indentation_style
- http://www.cs.nott.ac.uk/~pszcah/G53QAT/Presentations09/jjb07u/QAT09Presentations-jjb07u.ppt

Software Engineering, Professional Ethics

91

Outline

- Ethics
- Ethical guidelines for computer professionals
- Codes of ethics for computing professionals
- Guidelines for software developers and decision makers
- The SE Code
- The ACM Code
- Computing Curricula ACM/IEEE

Ethical Guidelines for Computer Professionals

- We have responsibilities not only to our customers but also to the general public.
- Responsibilities including minimizing risks to:
  - Privacy
  - Security of data
  - Safety
  - Reliability
  - Ease of use
- Must exercise good practices to reduce likelihood of problems, including maintaining professional competency.

Codes of Ethics for Computing Professionals

- Software Engineering Code of Ethics and Professional Practice
  - Adopted by Association of Computing Machinery and IEEE Computing Society
  - https://www.acm.org/about-acm/code-of-ethics
  - Software Engineering Code of Ethics and Professional Practice (Version 5.2) as recommended by the ACM/IEEE-CS Joint Task Force on Software Engineering Ethics and Professional Practices and jointly approved by the ACM and the IEEE-CS as the standard for teaching and practicing software engineering.

Guidelines for Software Developers and decision makers

- Include users in the design and testing stages – makes systems safe and useful
- Be careful when planning and scheduling a project, writing bids or contracts
- Design for real users (make it crash resistant)
- Don’t assume existing software is safe
- Be open and honest about capabilities and limitations of software
- Require a convincing case for safety
  - A disastrous attitude: Challenger engineers had to prove “beyond a shadow of a doubt that it was not safe to [launch].”

The Software Engineering Code

8 principles that express responsibilities to:

1. The public
2. The client and employer
3. The product
4. Professional judgment
5. Management
6. The profession
7. Colleagues
8. Self

1. The public

1. Accept full responsibility for their own work.
2. Moderate the interests of the software engineer, the employer, the client and the users with the public good.
3. Approve software only if they have a well-founded belief that it is safe, meets specifications, passes appropriate tests, and does not diminish quality of life, diminish privacy or harm the environment. The ultimate effect of the work should be to the public good.
4. Disclose to appropriate persons or authorities any actual or potential danger to the user, the public, or the environment, that they reasonably believe to be associated with software or related documents.

1. The public (cont)

5. Cooperate in efforts to address matters of grave public concern caused by software, its installation, maintenance, support or documentation.
6. Be fair and avoid deception in all statements, particularly public ones, concerning software or related documents, methods and tools.
7. Consider issues of physical disabilities, allocation of resources, economic disadvantage and other factors that can diminish access to the benefits of software.
8. Be encouraged to volunteer professional skills to good causes and contribute to public education concerning the discipline.

2. The client and employer

1. Provide service in their areas of competence, being honest and forthright about any limitations of their experience and education.
2. Not knowingly use software that is obtained or retained either illegally or unethically.
3. Use the property of a client or employer only in ways properly authorized, and with the client's or employer's knowledge and consent.
4. Ensure that any document upon which they rely has been approved, when required, by someone authorized to approve it.
5. Keep private any confidential information gained in their professional work, where such confidentiality is consistent with the public interest and consistent with the law.

2. The client and employer (cont)

6. Identify, document, collect evidence and report to the client or the employer promptly if, in their opinion, a project is likely to fail, to prove too expensive, to violate intellectual property law, or otherwise to be problematic.
7. Identify, document, and report significant issues of social concern, of which they are aware, in software or related documents, to the employer or the client.
8. Accept no outside work detrimental to the work they perform for their primary employer.
9. Promote no interest adverse to their employer or client, unless a higher ethical concern is being compromised; in that case, inform the employer or another appropriate authority of the ethical concern.

3. The product

1. Strive for high quality, acceptable cost and a reasonable schedule, ensuring significant tradeoffs are clear to and accepted by the employer and the client, and are available for consideration by the user and the public.
2. Ensure proper and achievable goals and objectives for any project on which they work or propose.
3. Identify, define and address ethical, economic, cultural, legal and environmental issues related to work projects.
4. Ensure that they are qualified for any project on which they work or propose to work by an appropriate combination of education and training, and experience.
5. Ensure an appropriate method is used for any project on which they work or propose to work.

3. The product (cont)

6. Work to follow professional standards, when available, that are most appropriate for the task at hand, departing from these only when ethically or technically justified.
7. Strive to fully understand the specifications for software on which they work.
8. Ensure that specifications for software on which they work have been well documented, satisfy the users' requirements and have the appropriate approvals.
9. Ensure realistic quantitative estimates of cost, scheduling, personnel, quality and outcomes on any project on which they work or propose to work and provide an uncertainty assessment of these estimates.
10. Ensure adequate testing, debugging, and review of software and related documents on which they work.

3. The product (cont)

11. Ensure adequate documentation, including significant problems discovered and solutions adopted, for any project on which they work.
12. Work to develop software and related documents that respect the privacy of those who will be affected by that software.
13. Be careful to use only accurate data derived by ethical and lawful means, and use it only in ways properly authorized.
14. Maintain the integrity of data, being sensitive to outdated or flawed occurrences.
15. Treat all forms of software maintenance with the same professionalism as new development.

4. Professional judgment

1. Temper all technical judgments by the need to support and maintain human values.
2. Only endorse documents either prepared under their supervision or within their areas of competence and with which they are in agreement.
3. Maintain professional objectivity with respect to any software or related documents they are asked to evaluate.
4. Not engage in deceptive financial practices such as bribery, double billing, or other improper financial practices.
5. Disclose to all concerned parties those conflicts of interest that cannot reasonably be avoided or escaped.
6. Refuse to participate, as members or advisors, in a private, governmental or professional body concerned with software related issues, in which they, their employers or their clients have undisclosed potential conflicts of interest.

5. Management

1. Ensure good management for any project on which they work, including effective procedures for promotion of quality and reduction of risk.
2. Ensure that software engineers are informed of standards before being held to them.
3. Ensure that software engineers know the employer's policies and procedures for protecting passwords, files and information that is confidential to the employer or confidential to others.
4. Assign work only after taking into account appropriate contributions of education and experience tempered with a desire to further that education and experience.
5. Ensure realistic quantitative estimates of cost, scheduling, personnel, quality and outcomes on any project on which they work or propose to work, and provide an uncertainty assessment of these estimates.

5. Management (cont)

6. Attract potential software engineers only by full and accurate description of the conditions of employment.
7. Offer fair and just remuneration.
8. Not unjustly prevent someone from taking a position for which that person is suitably qualified.
9. Ensure that there is a fair agreement concerning ownership of any software, processes, research, writing, or other intellectual property to which a software engineer has contributed.
10. Provide for due process in hearing charges of violation of an employer's policy or of this Code.
11. Not ask a software engineer to do anything inconsistent with this Code.
12. Not punish anyone for expressing ethical concerns about a project.

6. The profession

1. Help develop an organizational environment favorable to acting ethically.
2. Promote public knowledge of software engineering.
3. Extend software engineering knowledge by appropriate participation in professional organizations, meetings and publications.
4. Support, as members of a profession, other software engineers striving to follow this Code.
5. Not promote their own interest at the expense of the profession, client or employer.
6. Obey all laws governing their work, unless, in exceptional circumstances, such compliance is inconsistent with the public interest.

6. The profession (cont)

7. Be accurate in stating the characteristics of software on which they work, avoiding not only false claims but also claims that might reasonably be supposed to be speculative, vacuous, deceptive, misleading, or doubtful.
8. Take responsibility for detecting, correcting, and reporting errors in software and associated documents on which they work.
9. Ensure that clients, employers, and supervisors know of the software engineer's commitment to this Code of ethics, and the subsequent ramifications of such commitment.
10. Avoid associations with businesses and organizations which are in conflict with this code.

6. The profession (cont)

11. Recognize that violations of this Code are inconsistent with being a professional software engineer.
12. Express concerns to the people involved when significant violations of this Code are detected unless this is impossible, counter-productive, or dangerous.
13. Report significant violations of this Code to appropriate authorities when it is clear that consultation with people involved in these significant violations is impossible, counter-productive or dangerous.

7. Colleagues

1. Encourage colleagues to adhere to this Code.
2. Assist colleagues in professional development.
3. Credit fully the work of others and refrain from taking undue credit.
4. Review the work of others in an objective, candid, and properly-documented way.
5. Give a fair hearing to the opinions, concerns, or complaints of a colleague.

7. Colleagues (cont)

6. Assist colleagues in being fully aware of current standard work practices including policies and procedures for protecting passwords, files and other confidential information, and security measures in general.
7. Not unfairly intervene in the career of any colleague; however, concern for the employer, the client or public interest may compel software engineers, in good faith, to question the competence of a colleague.
8. In situations outside of their own areas of competence, call upon the opinions of other professionals who have competence in that area.

8. Self

1. Further their knowledge of developments in the analysis, specification, design, development, maintenance and testing of software and related documents, together with the management of the development process.
2. Improve their ability to create safe, reliable, and useful quality software at reasonable cost and within a reasonable time.
3. Improve their ability to produce accurate, informative, and well-written documentation.
4. Improve their understanding of the software and related documents on which they work and of the environment in which they will be used.
5. Improve their knowledge of relevant standards and the law governing the software and related documents on which they work.

8. Self (cont)

6. Improve their knowledge of this Code, its interpretation, and its application to their work.
7. Not give unfair treatment to anyone because of any irrelevant prejudices.
8. Not influence others to undertake any action that involves a breach of this Code.
9. Recognize that personal violations of this Code are inconsistent with being a professional software engineer.

ACM Code

- 24 statements of personal responsibility, including:
  - General moral imperatives
  - Professional responsibilities
  - Organizational leadership imperatives
  - Compliance with the Code

Computing Curricula ACM/IEEE

- Social context of computing
- Methods and tools of analysis of ethical argument
- Professional and ethical responsibilities
- Risks and liabilities of safety-critical systems
- Intellectual property
- Privacy and civil liberties
- Social implications of the Internet
- Computer crime
- Philosophical foundations of ethics

115

The Increasing General Public Awareness on Ethical Aspects of Technology

- The high level of media attention given to computer-related disasters in technical systems has increased interest in Computer Ethics:
  - The explosion of Arianne V in 1996
  - The Therac-25 computerized radiation machine overdoses

116

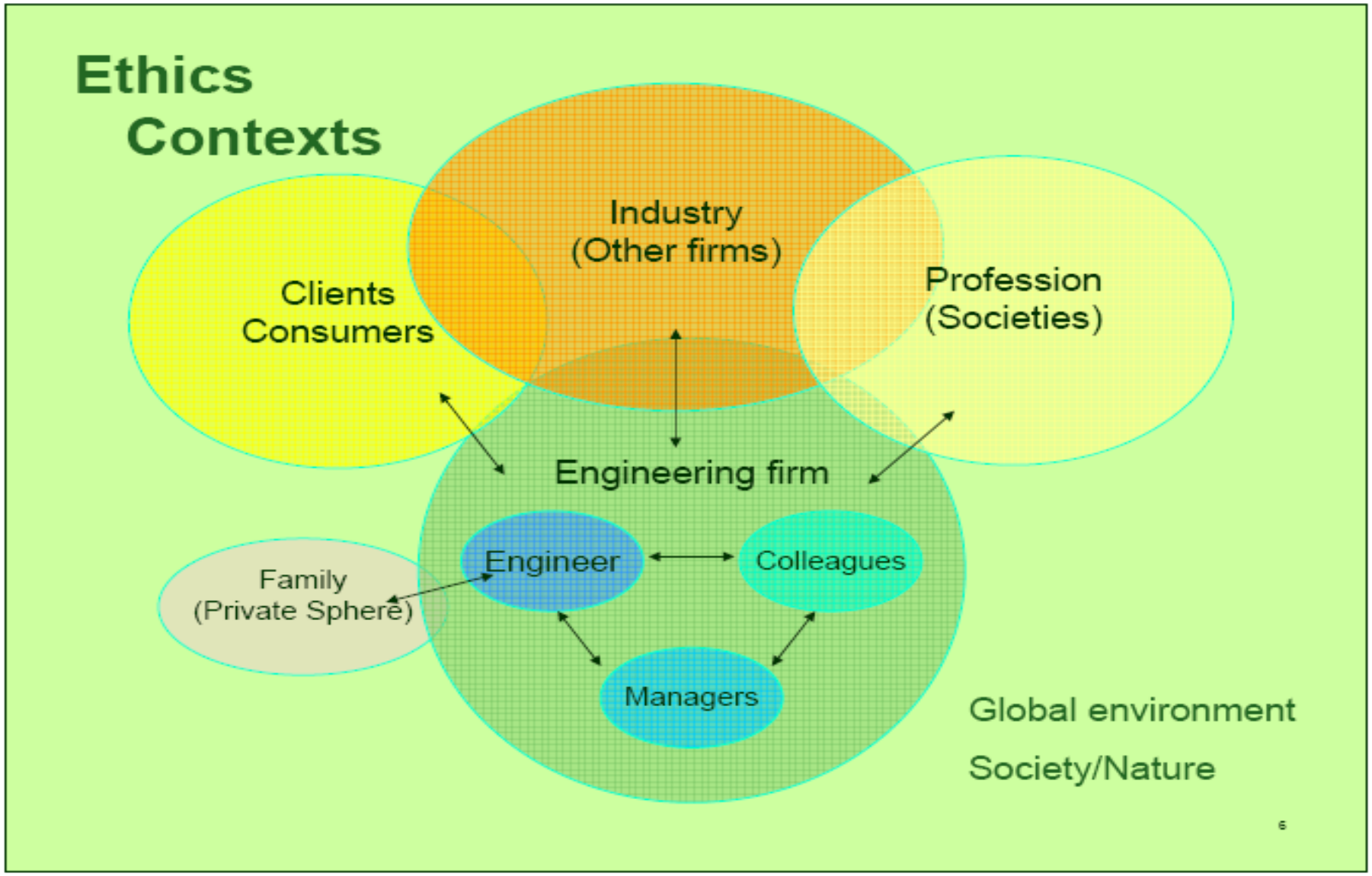
Why Ethics?

- "*There are few things wholly evil or wholly good. Almost everything...is an inseparable compound of the two, so that our best judgment of the preponderance between them is continually demanded.*"

  Abraham Lincoln

117

Ethics Contexts



118

Engineering as Social Experimentation

- "*All products of technology present some potential dangers, and thus engineering is an inherently risky activity. In order to underscore this fact and help in exploring its ethical implications, we suggest that engineering should be viewed as an experimental process. It is not, of course, an experiment conducted solely in a laboratory under controlled conditions. Rather, it is an experiment on a social scale involving human subjects.*"

Ethics in Engineering, Martin MW and Schinzinger R, McGraw-Hill, 1996

119

Social Importance of Engineering

- Engineering has a direct and vital effect on the quality of life of people.
- Accordingly, the services provided by engineers must be dedicated to the protection of the public safety, health and welfare.

120

Why is the Professional Ethics Important for Computer Scientists and Engineers?

- Because the Professional Ethics shall be a part of education *for every socially important profession, as* one of essential constituents of the meaning of the term *professionalism!*

121

Summary and Reading Suggestions

- Professional Ethics and Responsibilities
  - Sara Baase: “From A Gift of Fire”, Second Edition, 2003, Prentice Hall.
- American *ACM/IEEE Computing Curriculum*
  - http://www.computer.org/education/cc2001/index.htm

122

Software Construction and Development

Week-10

DevOps

- The DevOps is the combination of two words, one is **Development** and other is **Operations**. It is a culture to promote the development and operation process collectively.
- The DevOps will help you to learn DevOps basics and provide depth knowledge of various DevOps tools such as Git, Ansible, Docker, Puppet, Jenkins, Chef, Nagios, and Kubernetes.

Why DevOps?

- The operation and development team worked in complete isolation.
- After the design-build, the testing and deployment are performed respectively. That's why they consumed more time than actual build cycles.
- Without the use of DevOps, the team members are spending a large amount of time on designing, testing, and deploying instead of building the project.
- Manual code deployment leads to human errors in production.
- Coding and operation teams have their separate timelines and are not in synch, causing further delays.

Water Fall Vs Agile

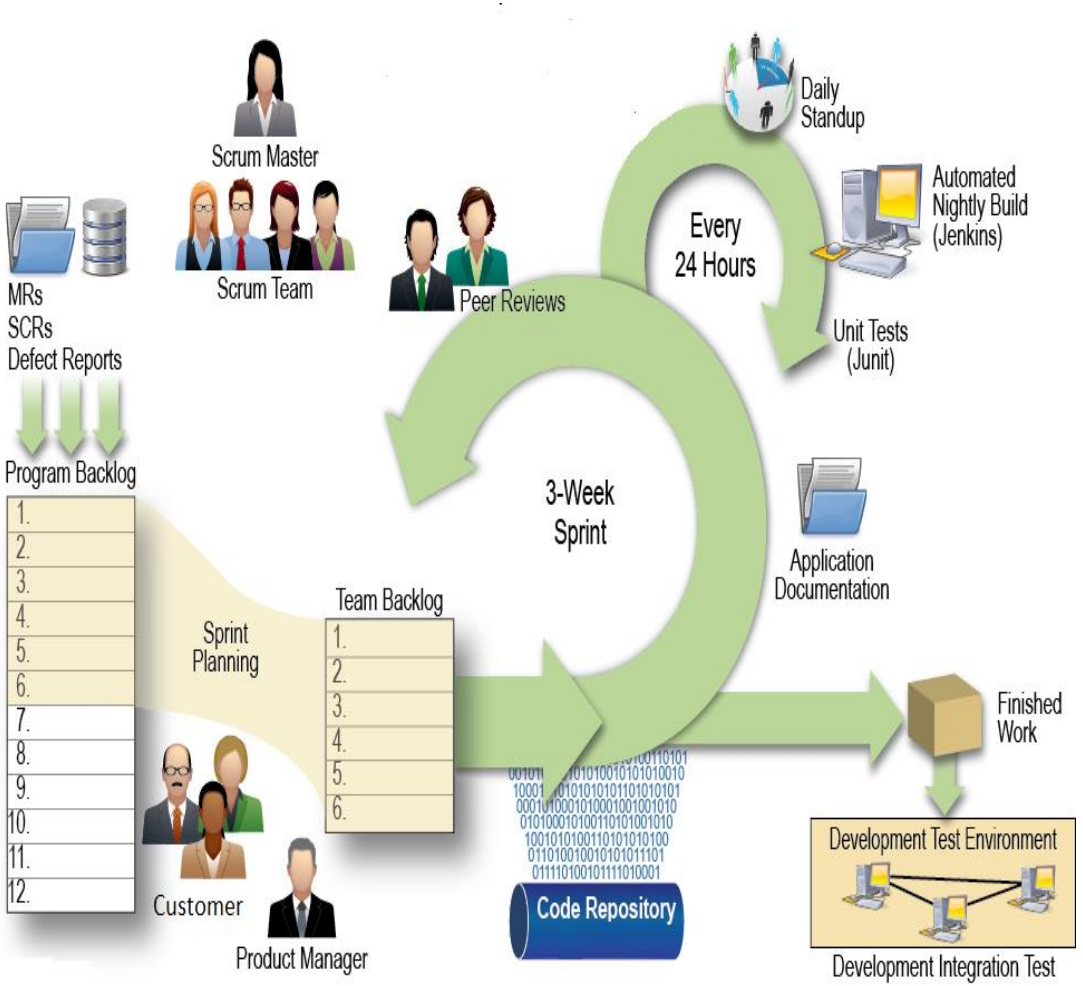
Waterfall Development Process

- Linear/Sequential
- Stages
- Advantages
- Disadvantages

Agile

- Iterative Process
- Workflow
- Advantages
- Disadvantages

- Agile Vs Waterfall
  - Linear fashion of software development



Why DevOps?

- **Efficiency** - Faster time to market
- **Predictability** - Lower failure rate of new releases
- **Reproducibility** – Version everything
- **Maintainability** - Faster time to recovery in the event of a new release crashing or otherwise disabling the current system

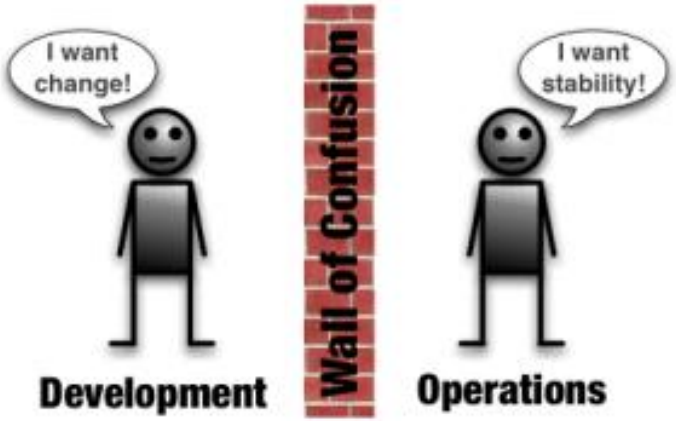


Image from - dev2ops.org

Break down the wall between development and operations

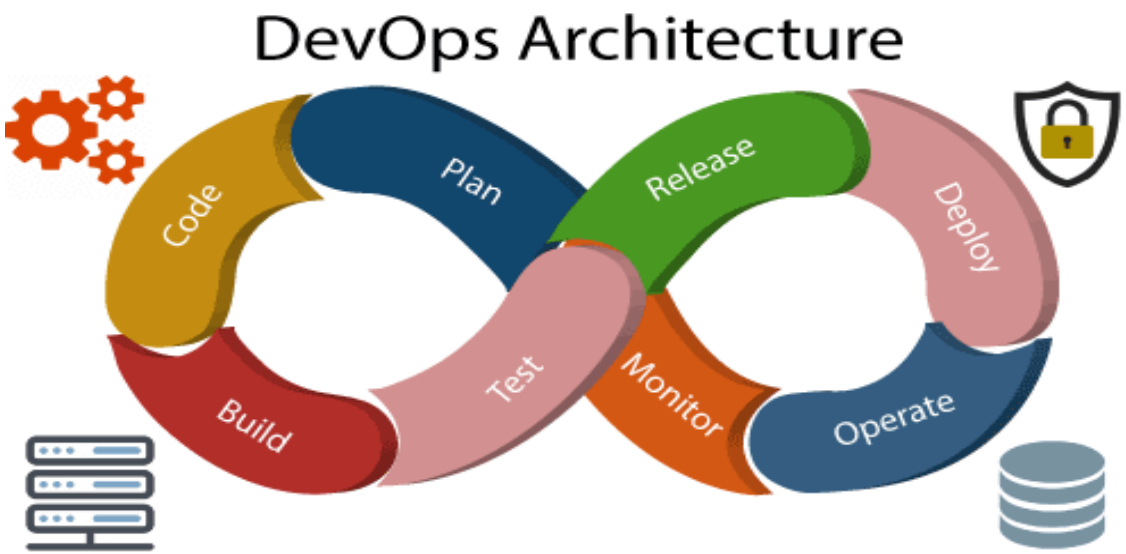
DevOps Advantages

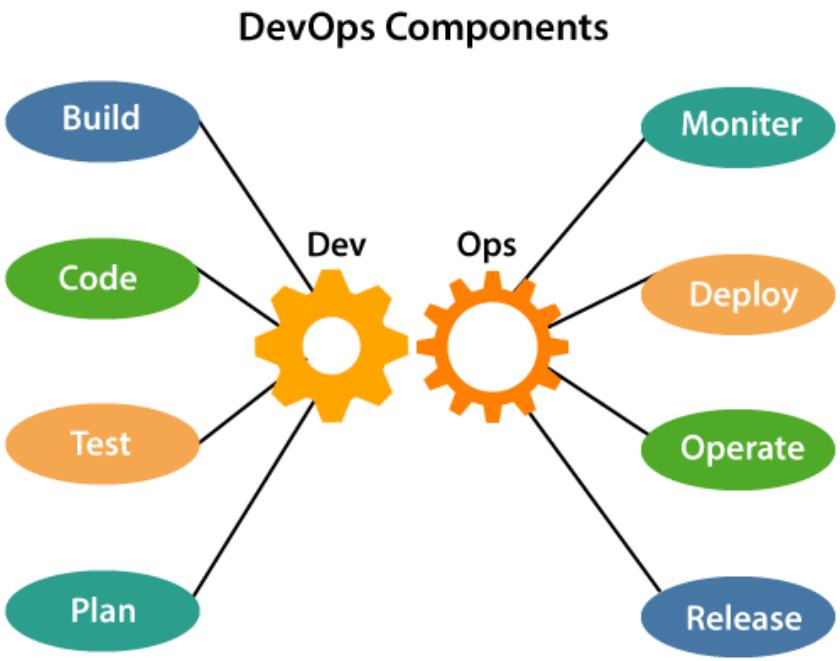
- DevOps is an excellent approach for quick development and deployment of applications.
- It responds faster to the market changes to improve business growth.
- DevOps escalate business profit by decreasing software delivery time and transportation costs.
- DevOps clears the descriptive process, which gives clarity on product development and delivery.
- It improves customer experience and satisfaction.
- DevOps simplifies collaboration and places all tools in the cloud for customers to access.
- DevOps means collective responsibility, which leads to better team engagement and productivity.

DevOps Disadvantages

- DevOps professional or expert's developers are less available.
- Developing with DevOps is so expensive.
- Adopting new DevOps technology into the industries is hard to manage in short time.
- Lack of DevOps knowledge can be a problem in the continuous integration of automation projects.

DevOps Architecture





DevOps Architecture

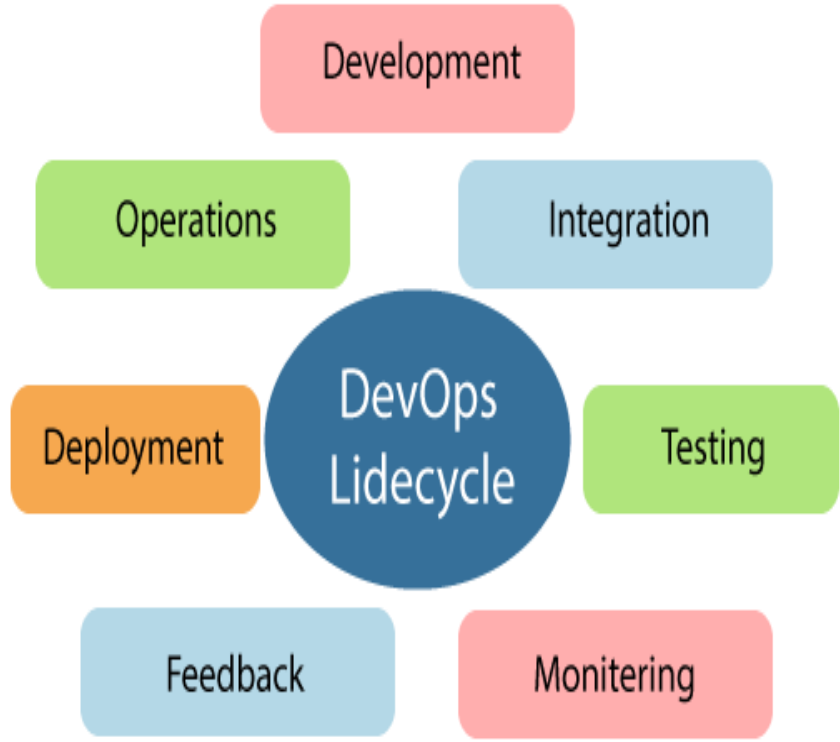
- **1) Build**
- Without DevOps, the cost of the consumption of the resources was evaluated based on the pre-defined individual usage with fixed hardware allocation. And with DevOps, the usage of cloud, sharing of resources comes into the picture, and the build is dependent upon the user's need, which is a mechanism to control the usage of resources or capacity.
- **2) Code**
- Many good practices such as Git enables the code to be used, which ensures writing the code for business, helps to track changes, and many others. The code can be appropriately arranged in files, folders, etc. And they can be reused.
- **3) Test**
- In the case of manual testing, it consumes more time in testing and moving the code to the output. The testing can be automated, which decreases the time for testing so that the time to deploy the code to production can be reduced as automating the running of the scripts will remove many manual steps.
- **4) Plan**
- DevOps use Agile methodology to plan the development. With the operations and development team in sync, it helps in organizing the work to plan accordingly to increase productivity.

DevOps Architecture

- **5) Monitor**
- Continuous monitoring is used to identify any risk of failure. Also, it helps in tracking the system accurately so that the health of the application can be checked. The monitoring becomes more comfortable with many third-party tools such as **Splunk**.
- **6) Deploy**
- Many systems can support the scheduler for automated deployment. The cloud management platform enables users to capture accurate insights and view the optimization scenario, analytics on trends by the deployment of dashboards.
- **7) Operate**
- DevOps changes the way traditional approach of developing and testing separately. The teams operate in a collaborative way where both the teams actively participate throughout the service lifecycle. The operation team interacts with developers, and they come up with a monitoring plan which serves the IT and business requirements.
- **8) Release**
- Deployment to an environment can be done by automation. But when the deployment is made to the production environment, it is done by manual triggering.

DevOps Lifecycle

- **1) Continuous Development**
- This phase involves the planning and coding of the software. The vision of the project is decided during the planning phase. And the developers begin developing the code for the application. There are no DevOps tools that are required for planning.
- **2) Continuous Integration**
- This stage is the heart of the entire DevOps lifecycle. It is a software development practice in which the developers require to commit changes to the source code more frequently on a daily or weekly basis. Building code is not only involved compilation, but it also includes unit testing, integration testing, code review, and packaging. Jenkins is a popular tool used in this phase.
- **3) Continuous Testing**
- This phase, where the developed software is continuously testing for bugs. For constant testing, automation testing tools such as TestNG, JUnit, Selenium, etc are used. These tools allow QAs to test multiple code-bases thoroughly in parallel to ensure that there is no flaw in the functionality. This entire testing phase can automate with the help of a Continuous Integration tool called Jenkins.



DevOps Lifecycle

- **4) Continuous Monitoring**

- Monitoring is a phase that involves all the operational factors of the entire DevOps process, where important information about the use of the software is recorded and carefully processed to find out trends and identify problem areas. It may occur in the form of documentation files or maybe produce large-scale data about the application parameters when it is in a continuous use position. The system errors such as server not reachable, low memory, etc are resolved in this phase. It maintains the security and availability of the service.
  - **5) Continuous Feedback**
- The application development is consistently improved by analyzing the results from the operations of the software. This is carried out by placing the critical phase of constant feedback between the operations and the development of the next version of the current software application.
  - **6) Continuous Deployment**
- In this phase, the code is deployed to the production servers. Also, it is essential to ensure that the code is correctly used on all the servers. The new code is deployed continuously, and configuration management tools play an essential role in executing tasks frequently and quickly. Here are some popular tools which are used in this phase, such as Chef, Puppet, Ansible, and SaltStack.
  - **Continuous Operations**
- It is clear from the discussion that continuity is the critical factor in the DevOps in removing steps that often distract the development, take it longer to detect issues and produce a better version of the product after several months. With DevOps, we can make any software product more efficient and increase the overall count of interested customers in your product.

DevOps Principles

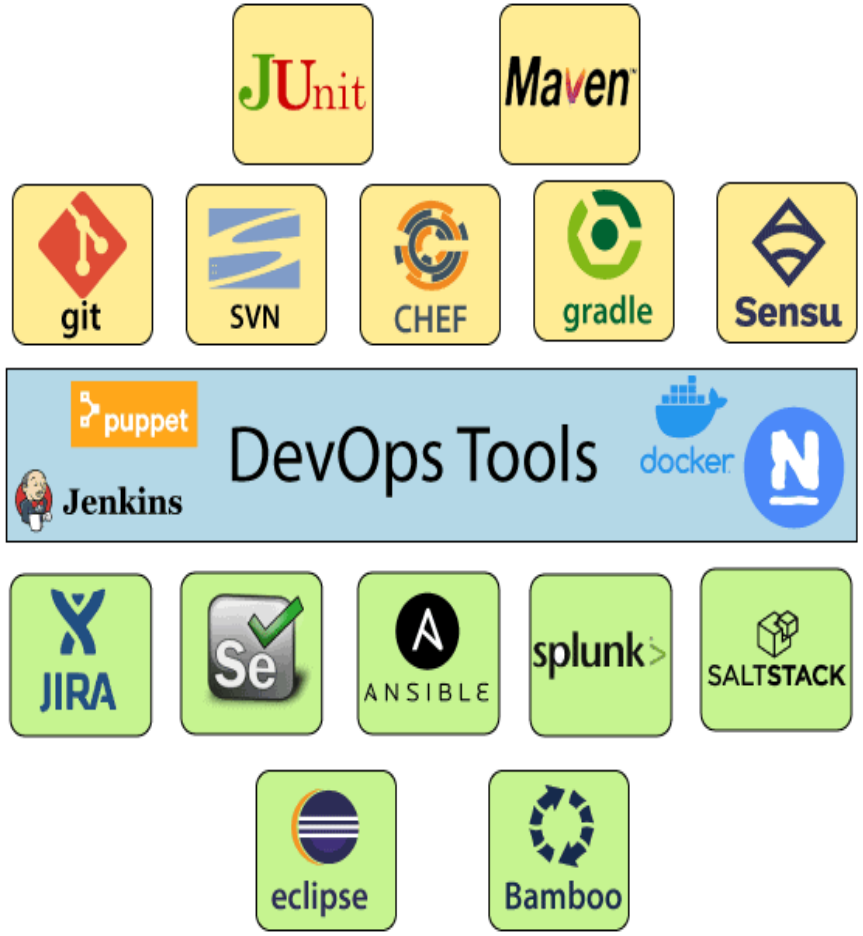
- **End to End Responsibility:**

  DevOps team need to provide performance support until they become the end of life. It enhances the responsibility and the quality of the products engineered.

- **Continuous Improvement:**
  - DevOps culture focuses on continuous improvement to minimize waste. It continuously speeds up the growth of products or services offered.
- **Automate Everything:**
  - Automation is an essential principle of the DevOps process. This is for software development and also for the entire infrastructure landscape.
- **Custom Centric Action:**
  - DevOps team must take customer-centric for that they should continuously invest in products and services.
- **Monitor and test everything:**
  - The DevOps team needs to have robust monitoring and testing procedures.
- **Work as one team:**
  - In the DevOps culture role of the designers, developers, and testers are already defined. All they needed to do is work as one team with complete collaboration.

DevOps Tools

- **Puppet**
- Puppet is the most widely used DevOps tool. It allows the delivery and release of the technology changes quickly and frequently. It has features of versioning, automated testing, and continuous delivery.
- **Ansible**
- Ansible is a leading DevOps tool. Ansible is an open-source IT engine that automates application deployment, cloud provisioning, intra service orchestration, and other IT tools.
- **Docker**
- Docker is a high-end DevOps tool that allows building, ship, and run distributed applications on multiple systems.
- **Nagios**
- Nagios is one of the more useful tools for DevOps. It can determine the errors and rectify them with the help of network, infrastructure, server, and log monitoring systems.
- **Jenkins**
- Jenkins is a DevOps tool for monitoring the execution of repeated tasks. Jenkins is a software that allows continuous integration. Jenkins will be installed on a server where the central build will take place. It helps to integrate project changes more efficiently by finding the issues quickly.



Software Testing

- To understand the concept of software testing correctly, we need to understand a few related concepts.
- Verification and Validation

Verification and Validation

Verification

Does the product meet system specifications.

Have you built the product right?

Validation

Does the product meet user expectations.

Have you built the right product?

(Boehm)

Defect

- software defect is that phenomenon in which software deviates from its expected behavior. This is non-compliance from the expected behavior with respect to written specifications or the stakeholder needs.

Software Testing Objective

The correct approach to testing a scientific theory is not to try to verify it, but to seek to refute the theory – that is to prove that it has errors. (Popper 1965)

- The goal of testing is to expose latent defects in a software system before it is put to use.

Software Testing Objective

- A software tester tries to break the system.

- The objective is to show the presence of a defect.

- Testing cannot show the absence of a defect. It only increases your confidence level in the software.

Limitation of Testing

- A funstion to compare two strings of characters stored in an array for equality.

```
bool isStringsEqual(char a[], char b[]);
```

Limitation of Testing

a	b	Expected Result
"cat"	"dog"	false
""	""	true
"hen"	"hen"	true
"hen"	"heN"	false
""	" "	false
""	"ball"	false
"cat"	""	false
"HEN"	"hen"	false
"rat"	"door"	false
" "	" "	true

Code

```
bool isStringsEqual(char a[], char b[])
{
    bool result;
    if (strlen(a) != strlen(b))
        result = false;
    else {
        for (int i =0; i < strlen(a); i++)
            if (a[i] == b[i]) result = true;
            else result = false;
    }
    return result;
}
```

Limitation of Testing

- In order to prove that a formula or hypothesis is incorrect you have to show only one example.

- To prove that it is correct any number of examples are insufficient, You have to give a formal proof of its correctness.

Limitation of Testing

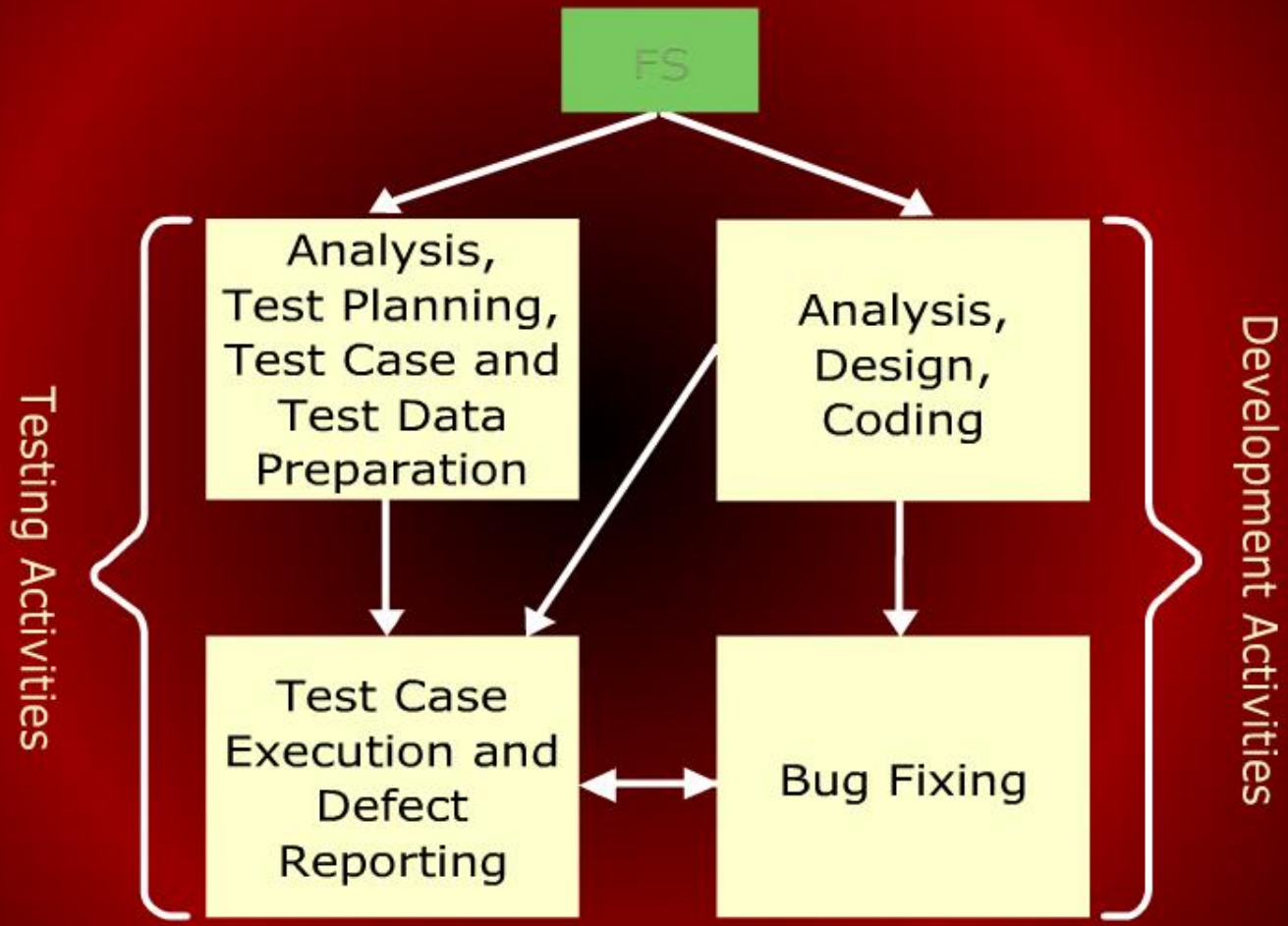
- These examples only help you in coming-up with a hypothesis but they are not proofs by themselves.

- They only enhance your comfort level in that particular hypothesis (i.e. in your piece of software).

Test Cases and Test Data

- Input and output specification plus a statement of the function under test.
- Steps to perform the function Expected results that the software application produces

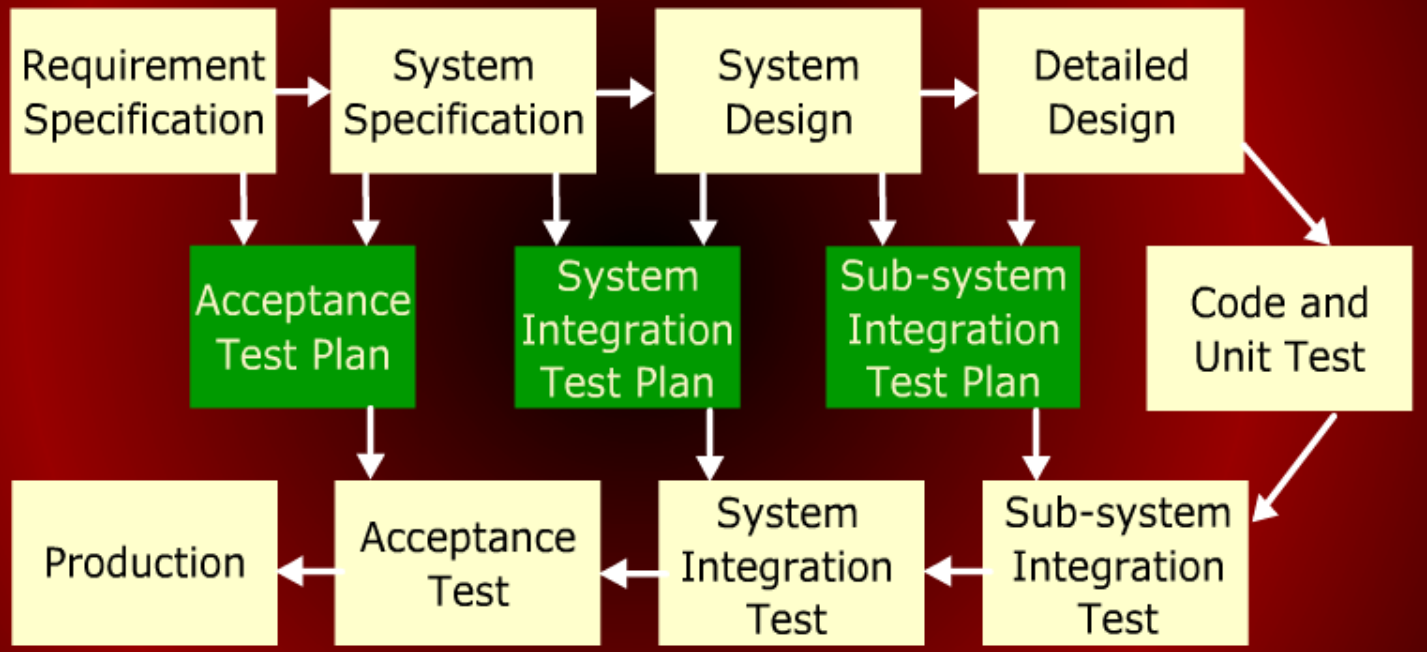
Testing vs Development



Developer as the Tester

Development	Testing
- Development is a creative activity.	- Testing is a destructive activity.
- The objective of development is to show that the program works.	- The objective of testing is to show that the program does not work.

Testing and Software Phases



Testing Types

Structural or White Box Testing

- Analyses the internal structure of the program.
- Test cases are written to test these structures.

Guidelines for Finding Equivalence Classes

- Organize your equivalence classes.
- Determine groups of normal courses.
- Determine boundary conditions.
- Do not forget invalid inputs.

Example – isStringsEqual

- One string is empty.
  - First is NULL.
  - Second is NULL.

Example – isStringsEqual

2. Unequal Strings

- Two unequal strings of arbitrary length.
  - Two unequal strings of same length.
  - Two unequal strings of different length.
- Check for case sensitivity.
  - Same strings with different characters capitalized.

Example – isStringsEqual

- One string is empty.
  - First is NULL.
  - Second is NULL.

Test Cases - Equal Strings

- all lower case	"cat" "cat"	yes
- all upper case	"CAT" "CAT"	yes
- mixed case	"Cat" "Cat"	yes
- numeric values	"123" "123"	yes
- two strings with blanks only	" " " "	yes
- character and numeric mixed	"Cat1" "Cat1"	yes
- strings with special characters	"Cat#1" "Cat#1"	yes
- two NULL strings	"" ""	yes

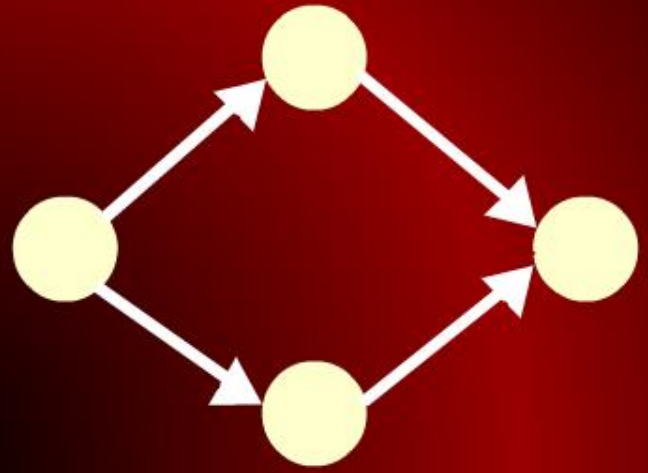
Test Cases - Unequal Strings

- two strings of different length	"cat" "mouse"	false
- two strings of same length	"cat" "dog"	false
- same strings with different characters capitalized	"Cat" "caT"	false
- first is NULL	"" "cat"	false
- second is NULL	"cat" ""	false

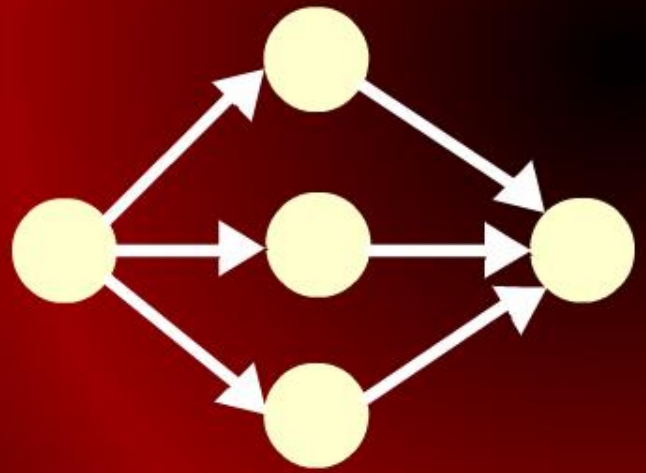
Flow Graph Notation



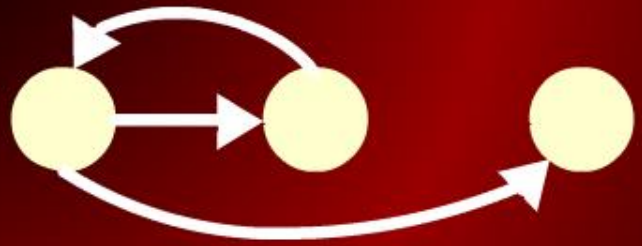
Sequence



if



case



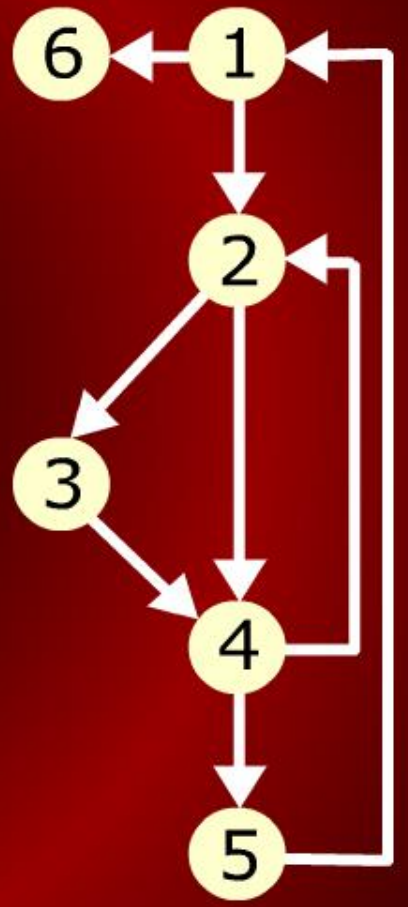
while

Flow Graph for a Sort Procedure

```
sorted = false;
while (!sorted)                    //1
{
    sorted = true;
    for (i=0; i < N-1; i++)        //2
    {
        if a[i] > a[i+1] {
            swap(a[i], a[i+1]);    //3
            sorted = false;
        }                          //4
    }                              //5
}                                  //6
```

Flow Graph for a Sort Procedure

```
sorted = false;
while (!sorted)                    //1
{
    sorted = true;
    for (i=0; i < N-1; i++)        //2
    {
        if a[i] > a[i+1] {
            swap(a[i], a[i+1]);    //3
            sorted = false;
        }                          //4
    }                              //5
}                                  //6
```



White Box Testing

White Box Testing

```
if (a == b)     //1
   c = d;       //2
a++;            //3
```



Statement Coverage
(One test case)

a=1, b=1

White Box Testing

```
if (a == b)     //1
   c = d;       //2
a++;            //3
```



Branch Coverage
(Two test cases)

a=1, b=1
a=1, b=2

White Box Testing

```
if (a == b)     //1
   c = d;       //2
a++;            //3
```



Path Coverage
(Two test cases)

a=1, b=1
a=1, b=2

Path Coverage

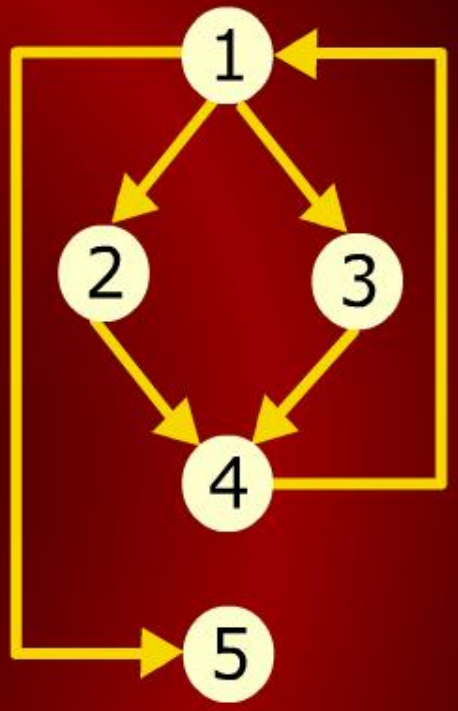
```
for (i = 0; i < N; i++)          //1
{
    if (condition1)
        // do something          //2
    else
        // do something          //3
    // do something              //4
}
                                 //5
```

Path Coverage

```
for (i = 0; i < N; i++)        //1
{
    if (condition1)
        // do something        //2
    else
        // do something        //3
    // do something            //4
}
                               //5
```

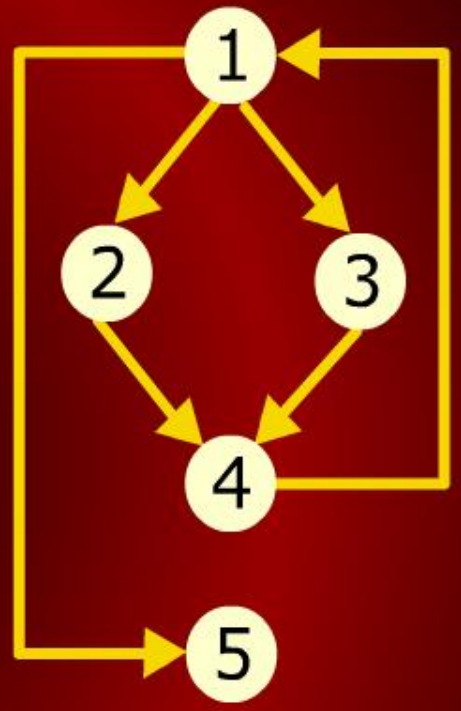
When N = 0

1 - 5



Path Coverage

```
for (i = 0; i < N; i++)        //1
{
    if (condition1)
        // do something        //2
    else
        // do something        //3
    // do something            //4
}
                               //5
```



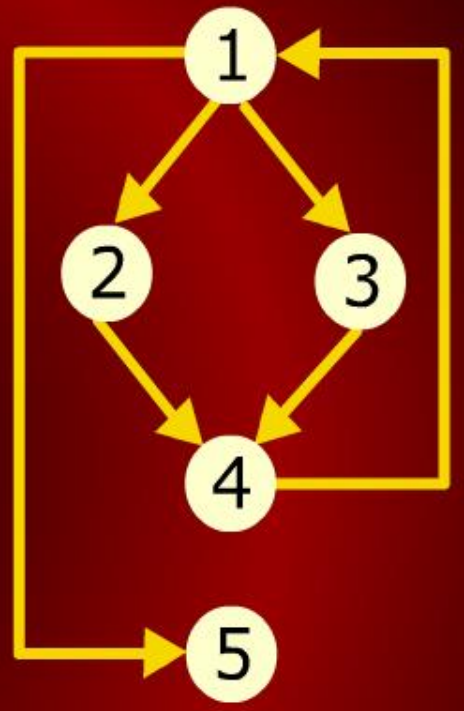
When N = 1

1 - 2 - 4 - 1 - 5

1 - 3 - 4 - 1 - 5

Path Coverage

```
for (i = 0; i < N; i++)        //1
{
    if (condition1)
        // do something        //2
    else
        // do something        //3
    // do something            //4
}
                               //5
```



When N = 2

1 - 2 - 4 - 1 - 2 - 4 - 1 - 5

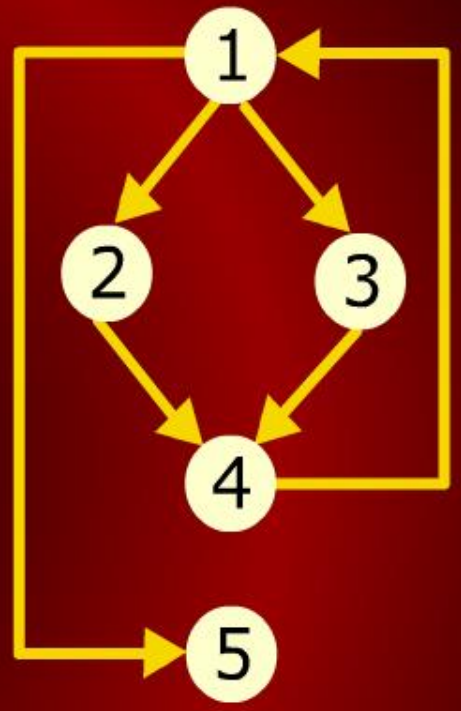
1 - 2 - 4 - 1 - 3 - 4 - 1 - 5

1 - 3 - 4 - 1 - 2 - 4 - 1 - 5

1 - 3 - 4 - 1 - 3 - 4 - 1 - 5

Path Coverage

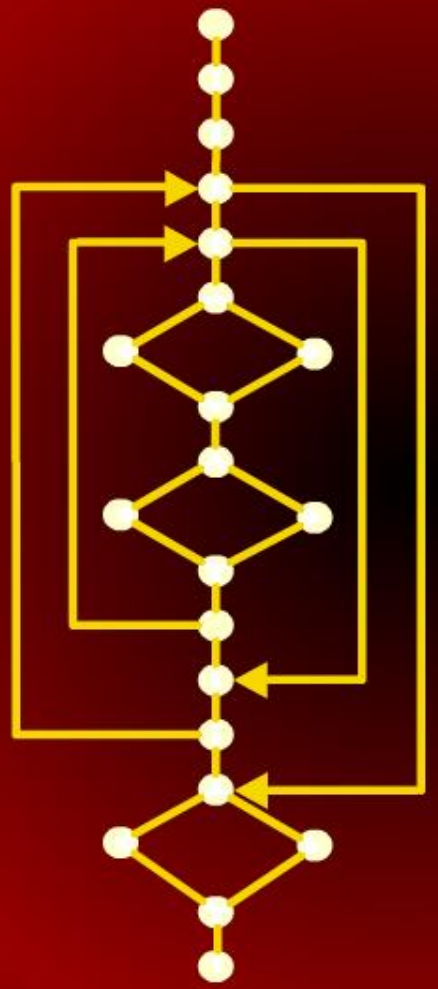
```
for (i = 0; i < N; i++)        //1
{
    if (condition1)
        // do something        //2
    else
        // do something        //3
    // do something            //4
}
                               //5
```



When N = 20

Total number of paths are more than 1 million.

Path Coverage



- 1852 different paths

Cyclomatic Complexity

Independent Path

Any path through the program (from start to end) that introduces at least one new set of processing statements or a new condition.

- It covers statements and all the branches.

Cyclomatic Complexity

- It is a quantitative measure of the logical complexity of a program.

- It defines the number of independent paths in the basis set of a program.

Cyclomatic Complexity

Cyclomatic Complexity, C(G), for a flow graph G is defined as:

$$C(G) = E - N + 2$$

where E is the number of edges and N is the number of nodes in the flow graph G.

Cyclomatic Complexity

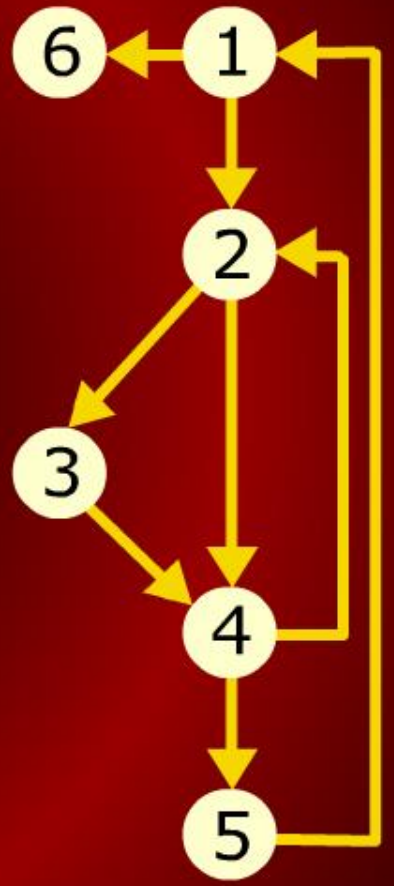
- It provides an upper bound for the number of tests that must be conducted to ensure that all statements and branches have been executed at least once.

Cyclomatic Complexity (Example)

```
sorted = false;
while (!sorted) {                    //1
    sorted = true;
    for (i=0; i < N-1; i++) {        //2
        if (a[i] > a[i+1]) {
            swap(a[i], a[i+1]);      //3
            sorted = false;
        }                            //4
    }                                //5
}
                                     //6
```

Cyclomatic Complexity (Example)

```
sorted = false;
while (!sorted) {                    //1
   sorted = true;
   for (i=0; i < N-1; i++) {         //2
      if (a[i] > a[i+1]) {
         swap(a[i], a[i+1]);         //3
         sorted = false;
      }                              //4
   }                                 //5
}
                                     //6
```



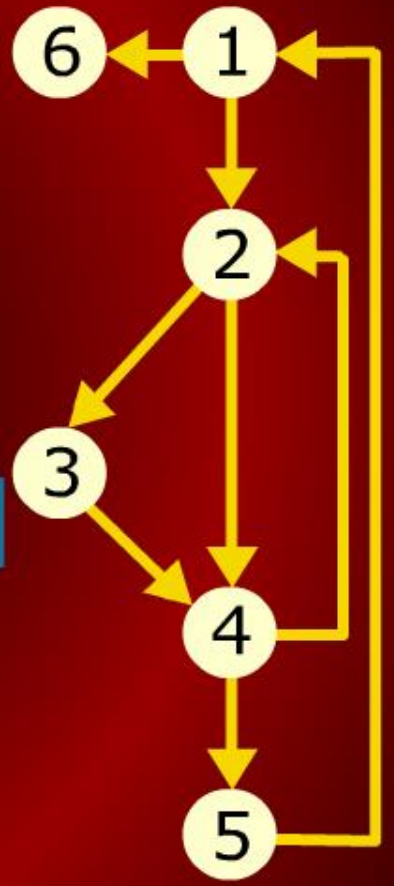
Number of edges = 8

Number of nodes = 6

C(G) = 8 - 6 + 2 = 4

Cyclomatic Complexity (Example)

```
sorted = false;
while (!sorted) {                    //1
   sorted = true;
   for (i=0; i < N-1; i++) {         //2
      if (a[i] > a[i+1]) {
         swap(a[i], a[i+1]);         //3
         sorted = false;
      }                              //4
   }                                 //5
}
                                     //6
```



Independent Paths

1 - 6

1 - 2 - 3 - 4 - 5 - 1 -6

1 - 2 - 4 - 5 - 1 - 6

1 - 2 - 4 - 2 - 3 - 4 - 5 - 1 - 6

Infeasible Paths

```
if (a == b)          //1
   c = c-1;          //2
if (a != b)          //3
   c = c+1;          //4
                     //5
```



Infeasible Paths

1-2-3-4-5

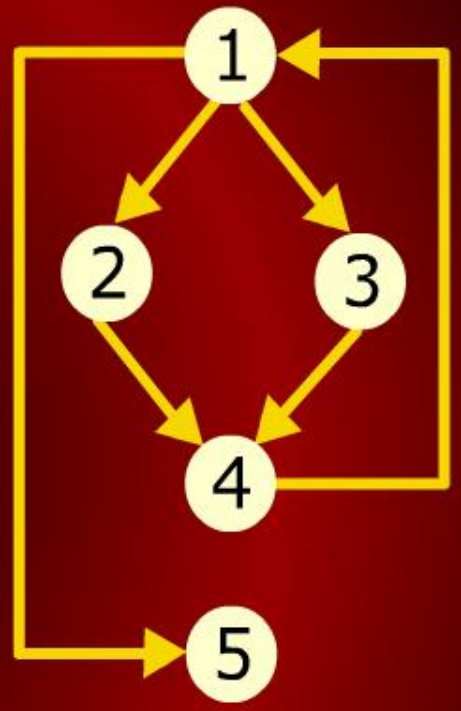
1-3-5

Infeasible Paths

A path through a program which is never traversed for any input data.

Path Coverage

```
for (i = 0; i < N; i++)        //1
{
    if (condition1)
        // do something        //2
    else
        // do something        //3
    // do something            //4
}
                               //5
```



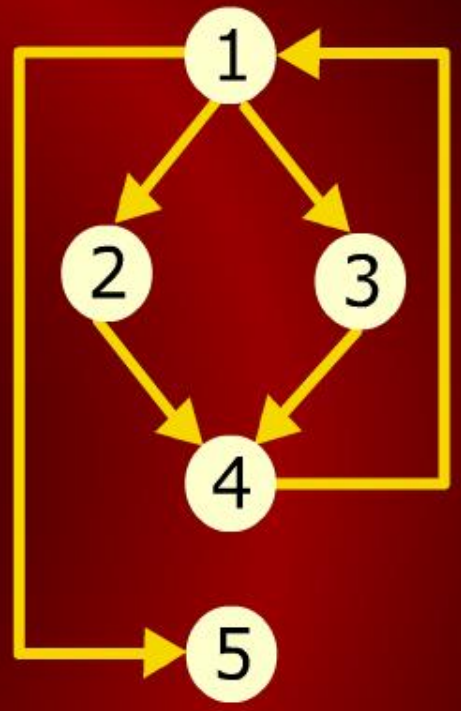
When N = 1

1 - 2 - 4 - 1 - 5

1 - 3 - 4 - 1 - 5

Path Coverage

```
for (i = 0; i < N; i++)        //1
{
    if (condition1)
        // do something        //2
    else
        // do something        //3
    // do something            //4
}
                               //5
```



Total number of possible paths are 2^N .

Developer as the Tester

Development	Testing
- Development is a creative activity.	- Testing is a destructive activity.
- The objective of development is to show that the program works.	- The objective of testing is to show that the program does not work.

Version control with Git

week 12

Problems Working Alone

- Ever done one of the following?
  - Had code that worked, made a bunch of changes and saved it, which broke the code, and now you just want the working version back…
  - Accidentally deleted a critical file, hundreds of lines of code gone…
  - Somehow messed up the structure/contents of your code base, and want to just "undo" the crazy action you just did
  - Hard drive crash!!!! Everything's gone, the day before deadline.

- Possible options:
  - Save as (MyClass-v1.java)
    - Ugh. Just ugh. And now a single line change results in duplicating the entire file…

Problems Working in teams

- Whose computer stores the "official" copy of the project?
  - Can we store the project files in a neutral "official" location?
- Will we be able to read/write each other's changes?
  - Do we have the right file permissions?
  - Lets just email changed files back and forth! Yay!
- What happens if we both try to edit the same file?
  - Bill just overwrote a file I worked on for 6 hours!
- What happens if we make a mistake and corrupt an important file?
  - Is there a way to keep backups of our project files?
- How do I know what code each teammate is working on?

Solution: Version Control

- **version control system**: Software that tracks and manages changes to a set of files and resources.

- You use version control all the time
  - Built into word processors/spreadsheets/presentation software
    - The magical “undo” button takes you back to “the version before my last action”
  - Wiki’s
    - Wiki’s are all about version control, managing updates, and allowing rollbacks to previous versions

Software Version control

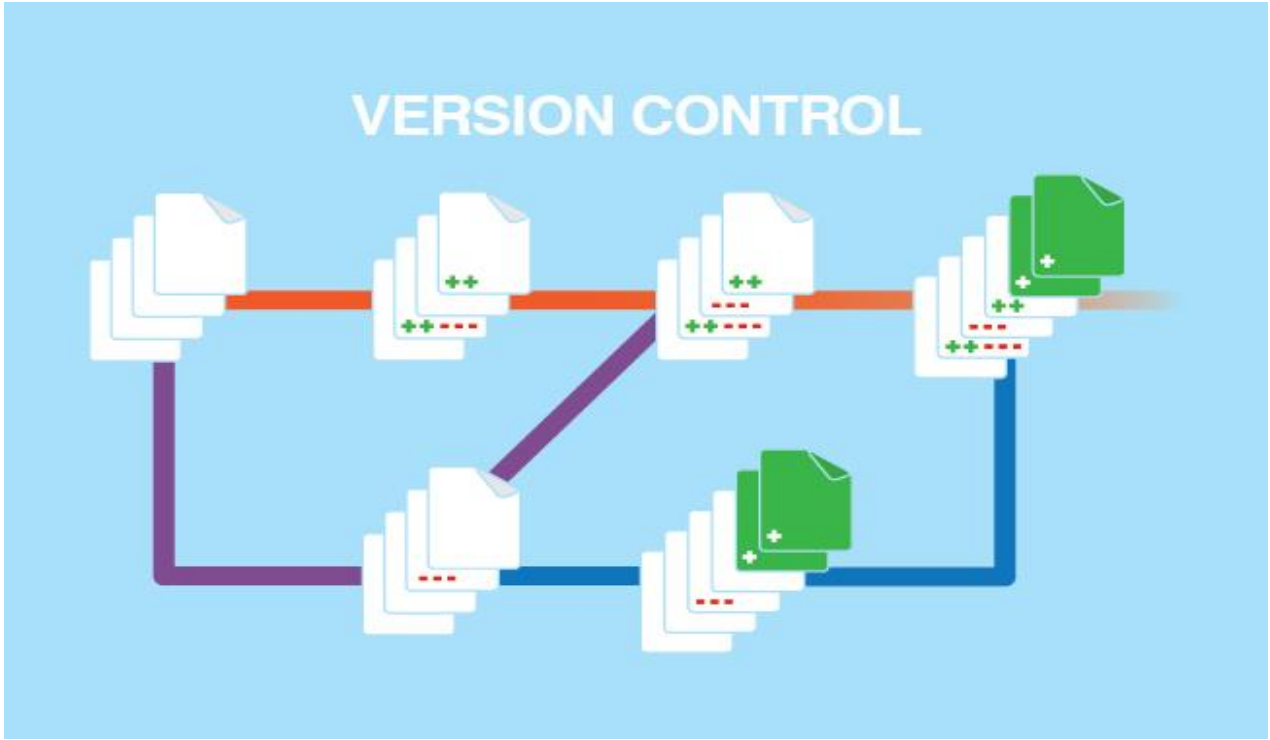
- Many version control systems are designed and used especially for software engineering projects
  - examples: CVS, Subversion (SVN), **Git**, Monotone, BitKeeper, Perforce

- helps teams to work together on code projects
  - a shared copy of all code files that all users can access
  - keeps current versions of all files, and backups of past versions
  - can see what files others have modified and view the changes
  - manages conflicts when multiple users modify the same file

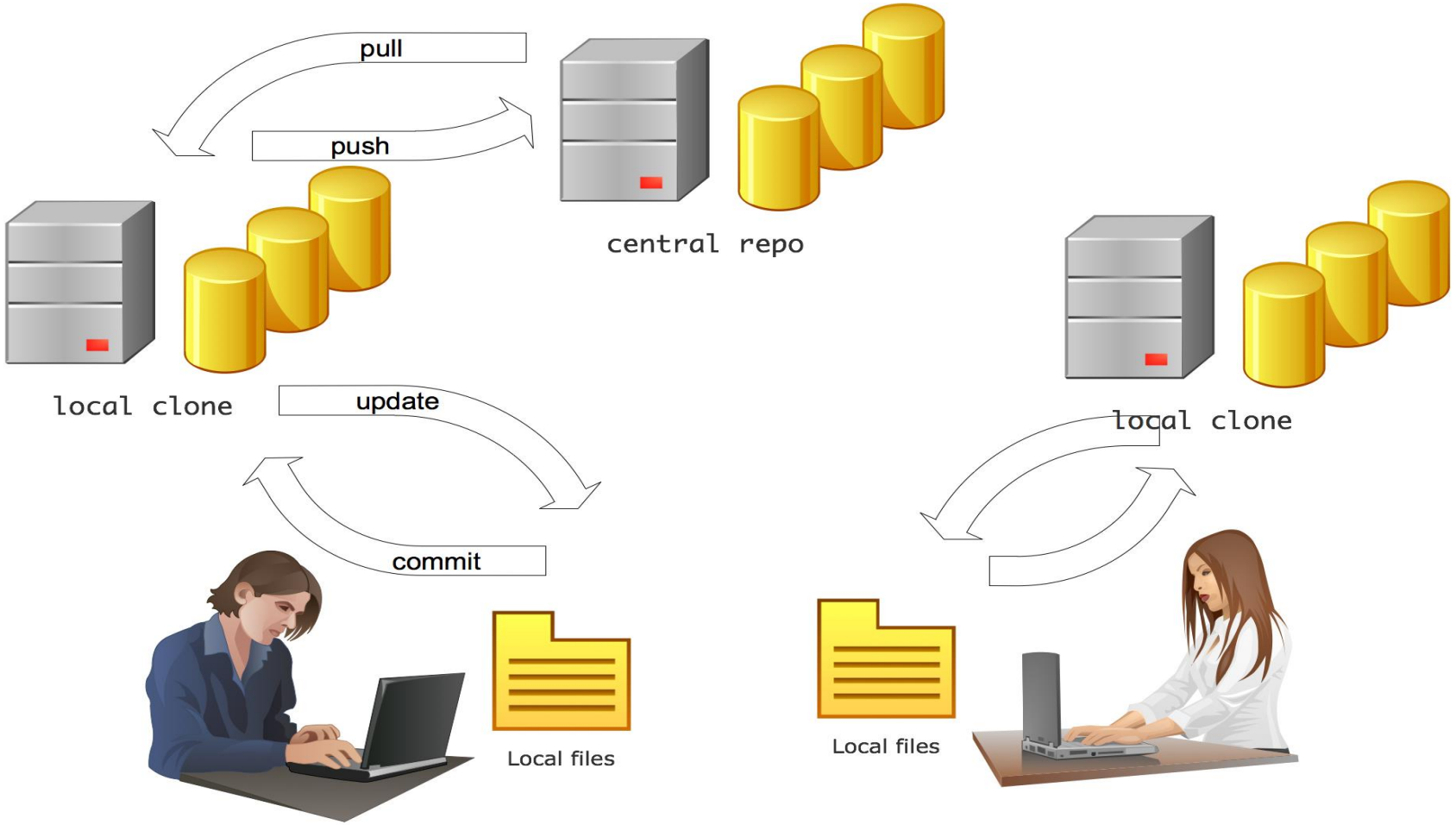
  - not particular to source code; can be used for papers, photos, etc.
    - but often works best with plain text/code files

version Control

- Version Control



Source Code Management



Version control tools



What is Git?

- Git is a popular version control system. It was created by Linus Torvalds in 2005, and has been maintained by Junio Hamano since then.
- **It is used for:**
  - Tracking code changes
  - Tracking who made changes
  - Coding collaboration
- **What does Git do?**
  - Manage projects with **Repositories**
  - **Clone** a project to work on a local copy
  - Control and track changes with **Staging** and **Committing**
  - **Branch** and **Merge** to allow for work on different parts and versions of a project
  - **Pull** the latest version of the project to a local copy
  - **Push** local updates to the main project

Working with Git

- Initialize Git on a folder, making it a **Repository**
- Git now creates a hidden folder to keep track of changes in that folder
- When a file is changed, added or deleted, it is considered **modified**
- You select the modified files you want to **Stage**
- The **Staged** files are **Committed**, which prompts Git to store a **permanent** snapshot of the files
- Git allows you to see the full history of every commit.
- You can revert back to any previous commit.
- Git does not store a separate copy of every file in every commit, but keeps track of changes made in each commit!

Why Git?

Over 70% of developers use Git!

Developers can work together from anywhere in the world.

Developers can see the full history of the project.

Developers can revert to earlier versions of a project.

Image from - dev2ops.org

What is GitHub?

Git is not the same as GitHub.
GitHub makes tools that use Git.
GitHub is the largest host of source code in the world, and has been owned by Microsoft since 2018.

Git Install:

You can download Git for free from the following website:
https://www.git-scm.com/

Repositories

- **Repository (aka "repo")**: a location storing a copy of all files.
  - you don't edit files directly in the repo;
  - you edit a local **working copy** or "working tree"
  - then you **commit** your edited files into the repo

- There may be only one repository that all users share (CVS, Subversion)
- Or each user could also have their own copy of the repository (Git, Mercurial)

- Files in your working directory must be **add**ed to the repo in order to be tracked.

What to put in a Repo?

- Everything needed to create your project:
  - Source code (Examples: .java, .c, .h, .cpp)
  - Build files (Makefile, build.xml)
  - Other resources needed to build your project: icons, text etc.

- Things generally NOT put in a repo (these can be easily re-created and just take up space):
  - Object files (.o)
  - Executables (.exe)

Repository Location

- Can create the repository anywhere
  - Can be on the same computer that you're going to work on, which might be ok for a personal project where you just want rollback protection

- But, usually you want the repository to be robust:
  - On a computer that's up and running 24/7
    - Everyone always has access to the project
  - On a computer that has a redundant file system (ie RAID)
    - No more worries about that hard disk crash wiping away your project!

- Options:
  - attu, CSE GitLab, GitHub (do NOT use GitHub for homework!!!)

Aside: So what is GitHub?

- GitHub.com is a site for online storage of Git repositories.
- Many open source projects use it, such as the Linux kernel.
- You can get free space for open source projects or you can pay for private projects.
- Do NOT use GitHub to store your homework!!

Question: Do I have to use GitHub to use Git?

Answer: No!

- you can use Git completely locally for your own purposes, or

Git Resources

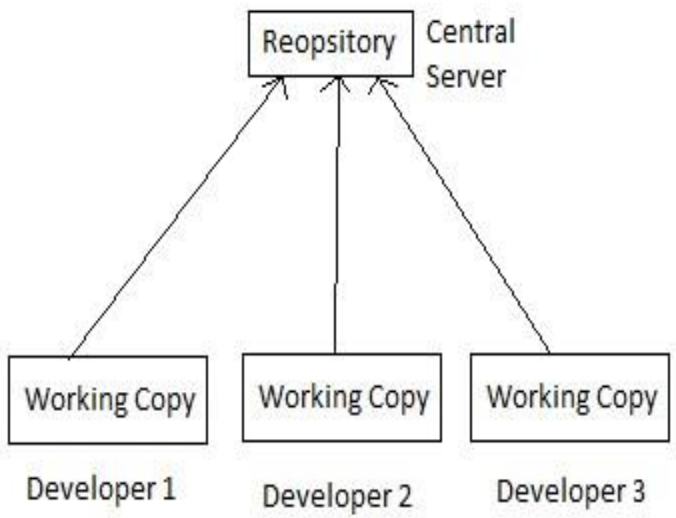
- At the command line: (where <verb> = config, add, commit, etc.)

```
$ git help <verb>
$ git <verb> --help
$ man git-<verb>
```

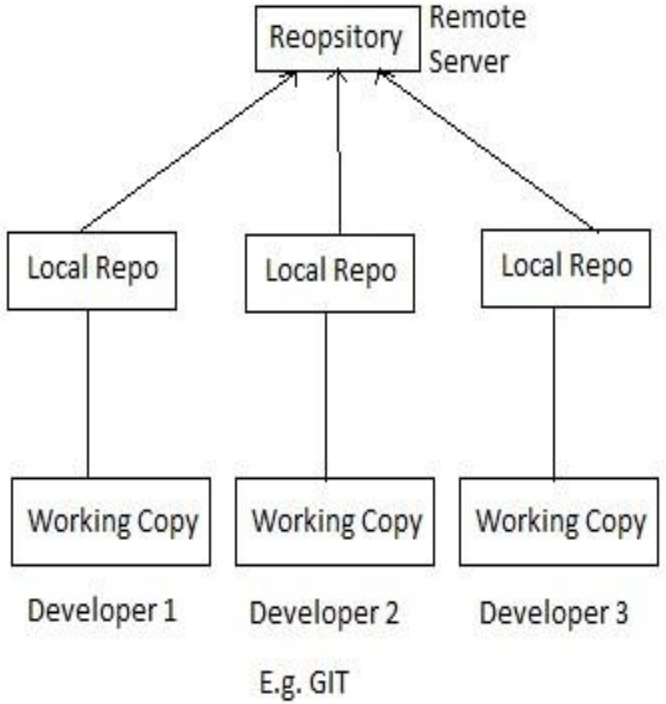
- Free on-line book: https://git-scm.com/book/en/v2
- Git tutorial: http://schacon.github.com/git/gittutorial.html
- Reference page for Git: http://gitref.org/index.html
- Git website: http://git-scm.com/

- Git for Computer Scientists: http://eagain.net/articles/git-for-computer-scientists/

Centralized vs Distributed



Centralized Version Control System

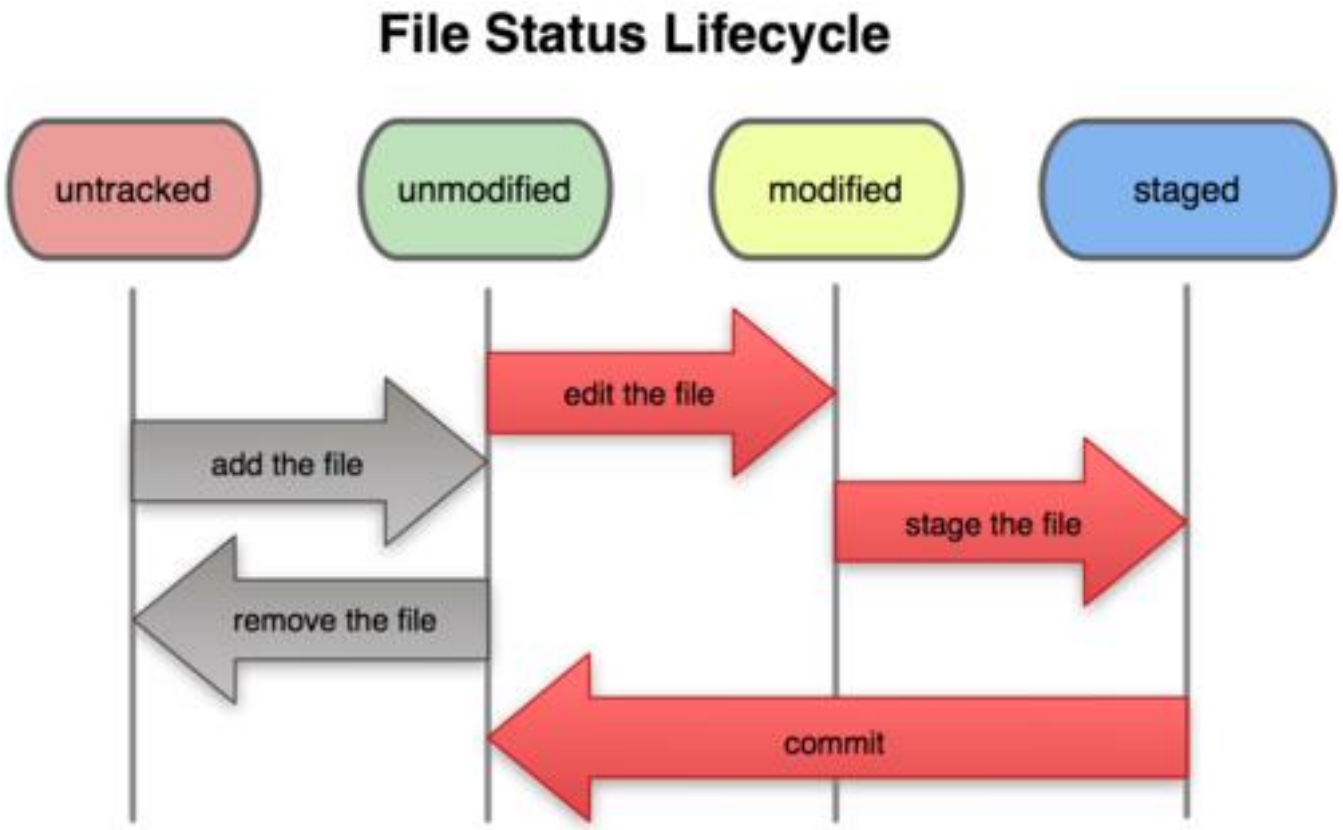


Distributed Version Control System

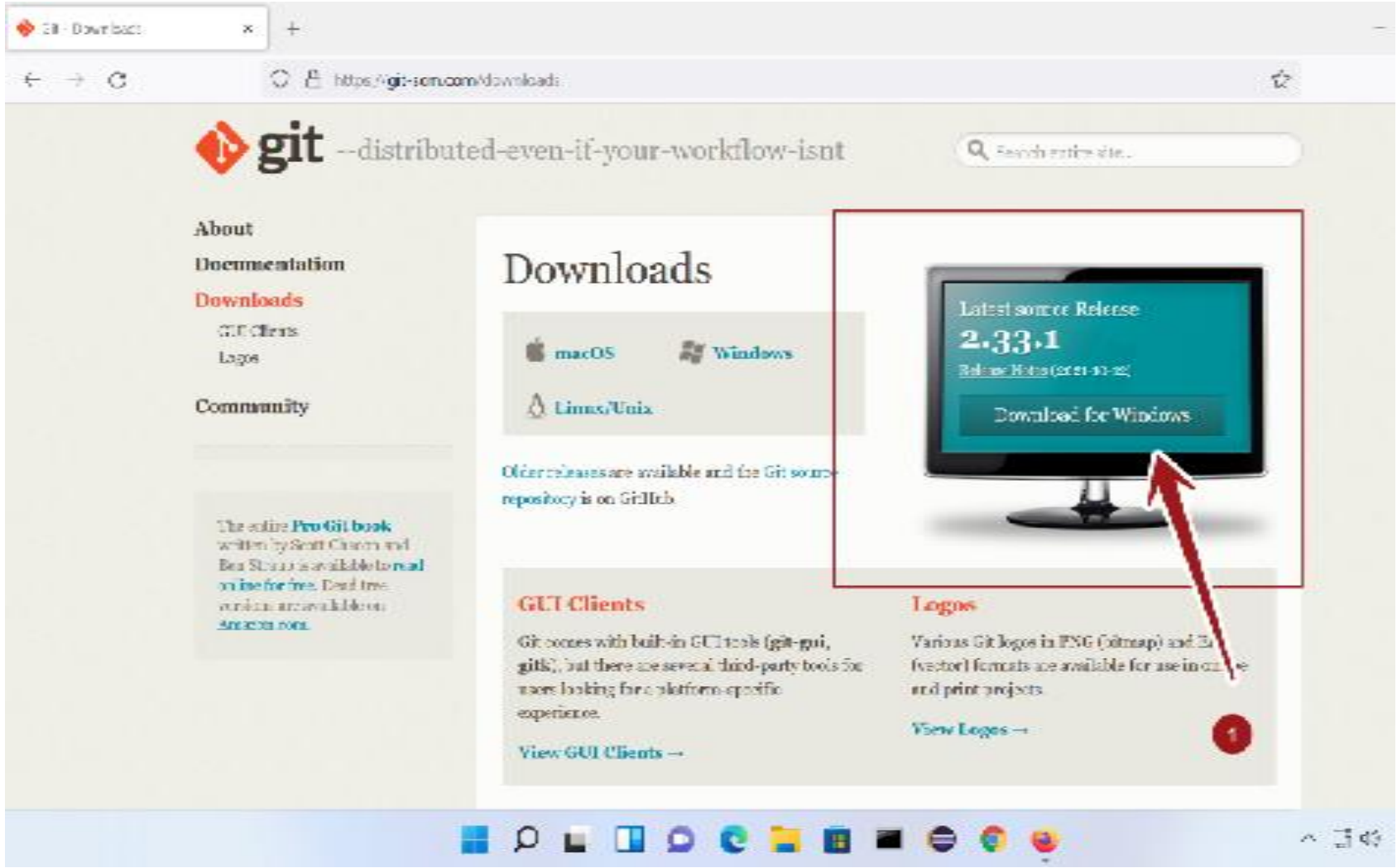
GiT Features



Git file lifecycle



GiT Installation



Basic Workflow

Basic Git workflow:

1. **Modify** files in your working directory.
2. **Stage** files, adding snapshots of them to your staging area.
3. Do a **commit**, which takes the files as they are in the staging area and stores that snapshot permanently to your Git directory (your local copy of the repo).

- Notes:
  - If a particular version of a file is in the **git directory**, it's considered **committed**.
  - If it's modified but has been added to the **staging area**, it is **staged**.
  - If it was **changed** since it was checked out but has not been staged, it is **modified**.

Get ready to use Git!

1. Set the name and email for Git to use when you commit:

```
$ git config --global user.name “Bugs Bunny”
$ git config --global user.email bugs@gmail.com
```

- You can call `git config --list` to verify these are set.
- These will be set globally for all Git projects you work with.
- You can also set variables on a project-only basis by not using the `--global` flag.
- You can also set the editor that is used for writing commit messages:
  $ `git config --global core.editor emacs` (it is vim by default)
- The latest version of git will also prompt you that `push.default` is not set, you can make this warning go away with:

```
$ git config --global push.default simple
```

Create a local copy of a repo

2. Two common scenarios: (only do one of these)

   a) To **clone an already existing repo** to your current directory:

   ```
   $ git clone <url> [local dir name]
   ```

   This will create a directory named *local dir name*, containing a working copy of the files from the repo, and a `.git` directory which you can ignore (used to hold the staging area and your actual repo)

Example: `git clone git@gitlab.cs.washington.edu:rea/superTest.git`

b) To **create a Git repo** in your current directory:

```
$ git init
```

This will create a `.git` directory in your current directory which you can ignore (used to hold the staging area and your actual repo).

Then you can commit files in your current directory into the repo:

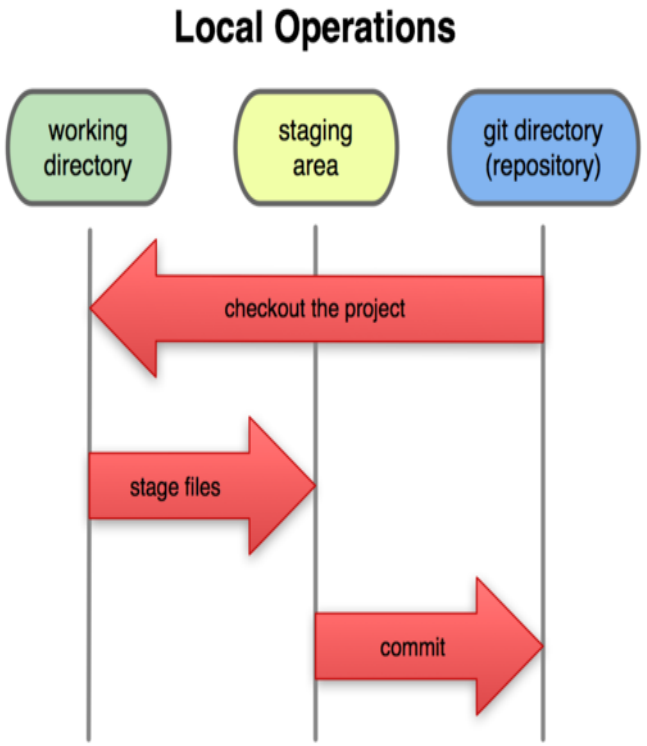
```
$ git add file1.java
$ git commit –m “initial project version”
```

Git operations & Commands

- Download Git and Install
- Run Terminal
- GiT Bash Open (check Git Version using command git--version and press enter)
- Create Sample Project Folder using mkdir command
  - by default in C where windows resides
  - Path (C\Users\User name\folder name)
- Create a file in which we have to work using “cd” command
- Add webpage using Notepad++ editor
- we can create multiple files on the same location
- Now, we use GiT for these two files

Git operations & Commands

- Now, we use GiT for these two files “home.html and page.html” using GiT Initiate
- GiT init command (GiT named folder is created in working directory)
- Staging Area command
  - git add home.html
  - after changes in html file
  - run git diff command
- Now, we can track changes



Git operations & Commands

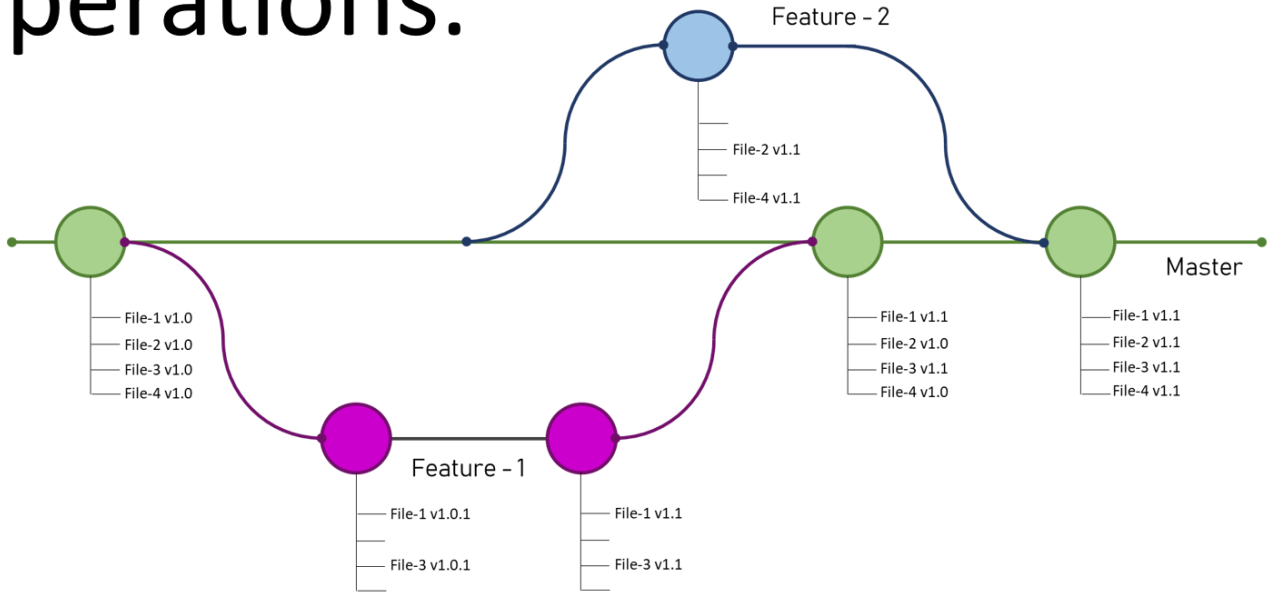
- "Git status" command will tell you the number of added files in staging area
- How to push and pull files on Local and Remote Directories from staging area?
  - git commit-a-m (push all files to local repository and an attached message what,where and when to commit changes in each file-track changes)
  - Then, push files to remote repository, for this sign up to GiT Hub
  - Create Repository with "New" option. https://github.com/join
  - New/name of repository/public or private/ Create (only one)
  - Use URL of Repository to push files from LR to RR.

Git operations & Commands

- Add Repository in Git using command (can add multiple..)
  - “git remote add <repository name> <repository url> ”
- example git remote add origin https://github.com....
- Now, push files from LR to RR, use the command
  - “git push origin master”
  - credentials for the first time
- Add a new branch using command
  - git branch fix
  - git branch command will show the total created branches
    - fix and master

Branching

GIT Branch and its Operations.



Branching

To create a branch called experimental:

- `$ git branch experimental`

To list all branches: (* shows which one you are currently on)

- `$ git branch`

To switch to the experimental branch:

- `$ git checkout experimental`

Later on, changes between the two branches differ, to merge changes from experimental into the master:

- `$ git checkout master`
- `$ git merge experimental`

Note: `git log --graph` can be useful for showing branches.

Note: These branches are in *your local repo*!

Git commands

command	description
`git clone` ***`url [dir]`***	copy a git repository so you can add to it
`git add` ***`files`***	adds file contents to the staging area
`git commit`	records a snapshot of the staging area
`git status`	view the status of your files in the working directory and staging area
`git diff`	shows diff of what is staged and what is modified but unstaged
`git help` ***`[command]`***	get help info about a particular command
`git pull`	fetch from a remote repo and try to merge into the current branch
`git push`	push your new branches and data to a remote repository
others: `init`, `reset`, `branch`, `checkout`, `merge`, `log`, `tag`	

After editing a file…

```
$emacs rea.txt
$ git status
# On branch master
# Changes not staged for commit:
#   (use "git add <file>..." to update what will be committed)
#   (use "git checkout -- <file>..." to discard changes in working directory)
#
#       modified:   rea.txt
#
no changes added to commit (use "git add" and/or "git commit -a")
$ git status -s
 M rea.txt                                   ← Note: M is in second column = “working  tree”
$ git diff                          ← Shows modifications that have not been staged.
diff --git a/rea.txt b/rea.txt
index 66b293d..90b65fd 100644
--- a/rea.txt
+++ b/rea.txt
@@ -1,2 +1,4 @@
 Here is rea's file.
+
+One new line added.
$ git diff --cached                 ← Shows nothing, no modifications have been staged yet.
$
```

After adding file to staging area…

```
$ git add rea.txt
$ git status
# On branch master
# Changes to be committed:
#   (use "git reset HEAD <file>..." to unstage)
#
#       modified:   rea.txt
#
$ git status -s
M  rea.txt                                  ← Note: M is in first column = “staging area”
$ git diff                    ← Note: Shows nothing, no modifications that have not been staged.
$ git diff --cached           ← Note: Shows staged modifications.
diff --git a/rea.txt b/rea.txt
index 66b293d..90b65fd 100644
--- a/rea.txt
+++ b/rea.txt
@@ -1,2 +1,4 @@
 Here is rea's file.
+
+One new line added.
```

Pulling and Pushing

Good practice:

1. **Add** and **Commit** your changes to your local repo
2. **Pull** from remote repo to get most recent changes (fix conflicts if necessary, add and commit them to your local repo)
3. **Push** your changes to the remote repo

To fetch the most recent updates from the remote repo into your local repo, and put them into your working directory:

```
$ git pull origin master
```

To push your changes from your local repo to the remote repo:

```
$ git push origin master
```

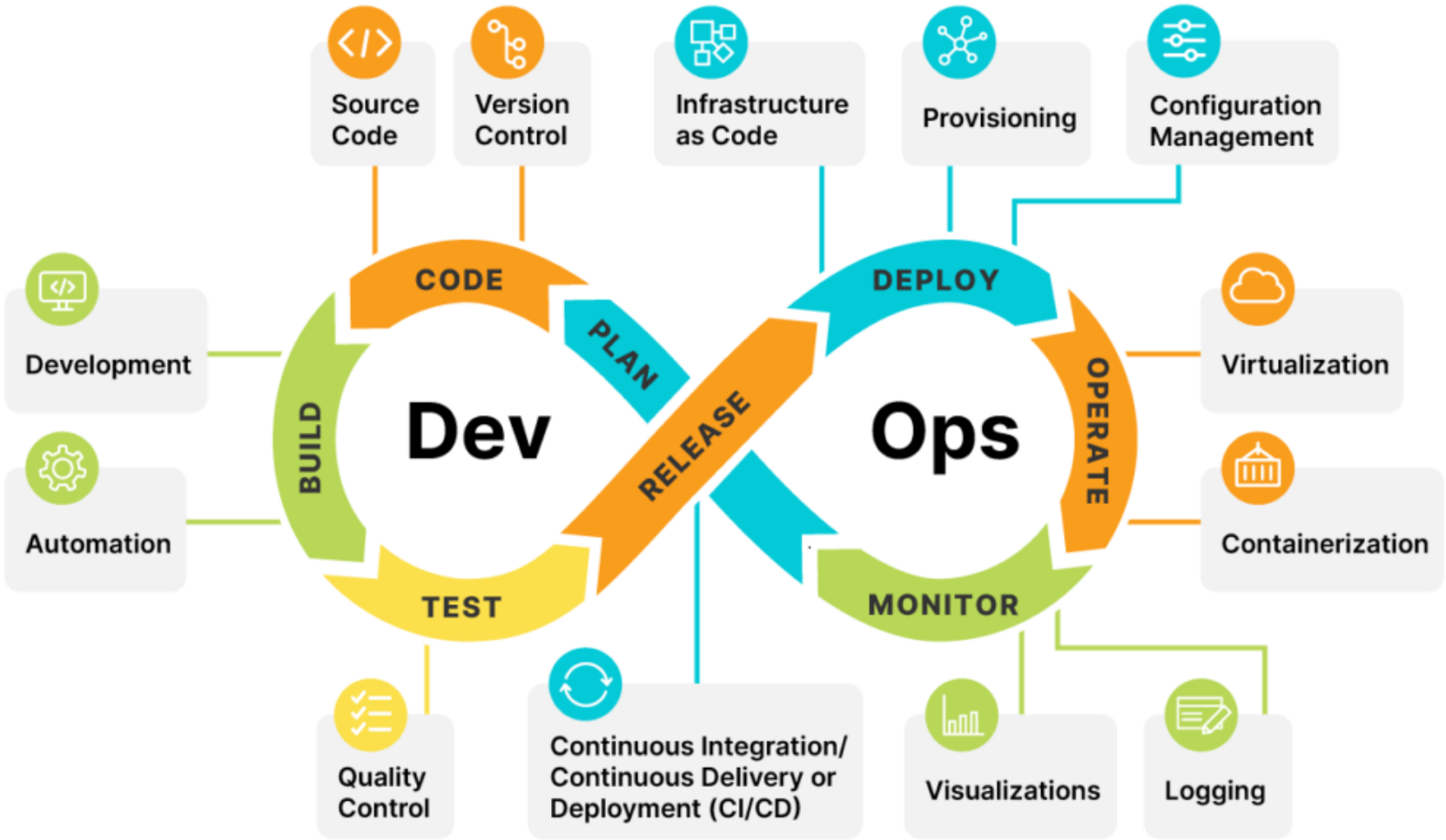
Notes: `origin` = an alias for the URL you cloned from

`master` = the remote branch you are pulling from/pushing to,
(the local branch you are pulling to/pushing from is your current branch)

Software construction and development Practical - I

Week 13-DevOps-Practical

DEVOPS


Source Code
Version Control
Infrastructure as Code
Provisioning
Configuration Management
Development
Automation
CODE
PLAN
BUILD
TEST
RELEASE
DEPLOY
OPERATE
MONITOR
Dev
Ops
Virtualization
Containerization
Quality Control
Continuous Integration/ Continuous Delivery or Deployment (CI/CD)
Visualizations
Logging

Software Development Practices

- Programming Languages
- Version Control Systems
- Methodologies
- Software Development Life Cycle (SDLC)

Operating System and Networking

Learn system administration concepts such as operating systems, networking, server management, security, and virtualization technologies.

CI/CD

Explore and master key DevOps practices like continuous integration and continuous delivery (CI/CD), including tools like Jenkins, Travis CI, and GitLab CI/CD.

Essential DevOps tools

Acquire proficiency in essential DevOps tools and technologies, such as configuration management (Ansible, Chef, Puppet), containerization and orchestration (Docker, Kubernetes), cloud computing platforms (AWS, Azure, GCP), infrastructure automation (Terraform, CloudFormation), and monitoring/logging (Prometheus, Grafana, ELK stack).

Software construction and development Practical - I

Week 13-Git Practical

GitLab

Create new project



Create blank project

Create a blank project to store your files, plan your work, and collaborate on code, among other things.



Create from template

Create a project pre-populated with the necessary files to get you started quickly.



Import project

Migrate your data from an external source like GitHub, Bitbucket, or another instance of GitLab.



Run CI/CD for external repository

Connect your external repository to GitLab CI/CD.

New Project

Your work / Projects / New project / **Create blank project**

Create blank project

Create a blank project to store your files, plan your work, and collaborate on code, among other things.

Project name

testproject

Must start with a lowercase or uppercase letter, digit, emoji, or underscore. Can also contain dots, pluses, dashes, or spaces.

Project URL

https://gitlab.com/u.bilal/

Project slug

/ testproject

Want to organize several dependent projects under the same namespace? Create a group.

Project deployment target (optional)

Select the deployment target

Visibility Level

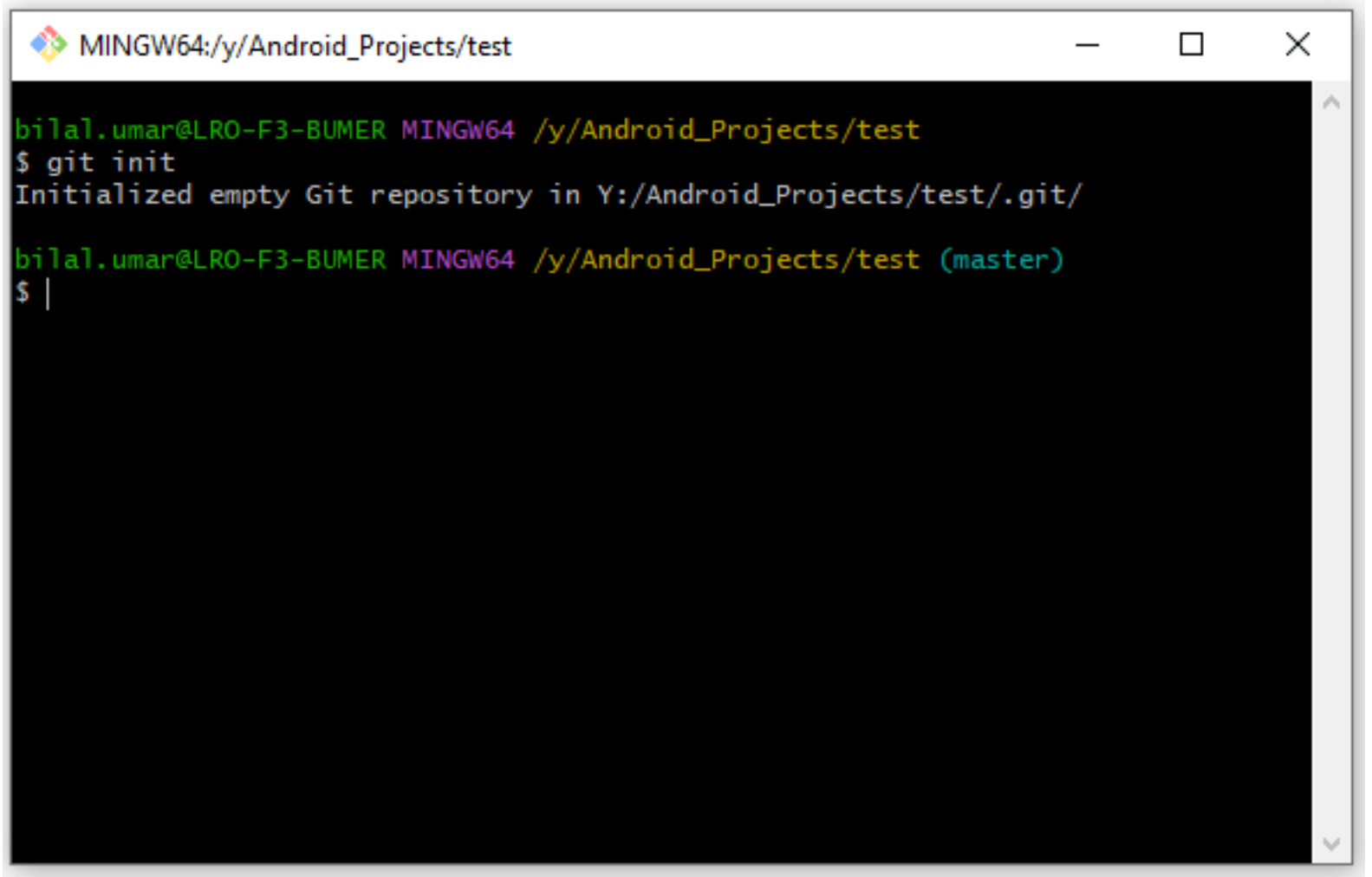
- Private
  Project access must be granted explicitly to each user. If this project is part of a group, access is granted to members of the group.
- Public
  The project can be accessed without any authentication.

Project Configuration

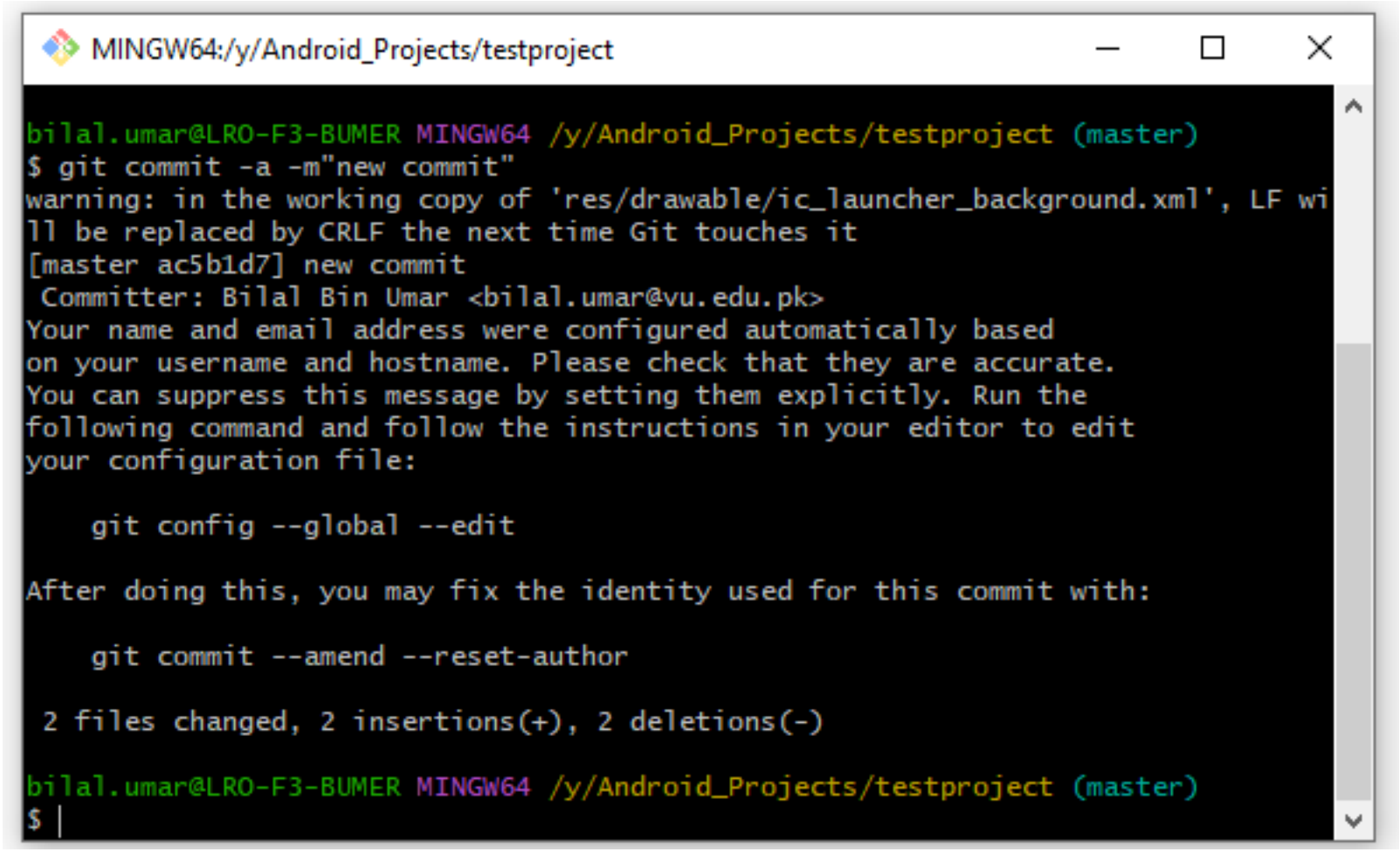
- [x] Initialize repository with a README
  Allows you to immediately clone this project's repository. Skip this if you plan to push up an existing repository.
- [ ] Enable Static Application Security Testing (SAST)
  Analyze your source code for known security vulnerabilities. Learn more.

Create project | Cancel

Git init


MINGW64:/y/Android_Projects/test
bilal.umar@LRO-F3-BUMER MINGW64 /y/Android_Projects/test
$ git init
Initialized empty Git repository in Y:/Android_Projects/test/.git/
bilal.umar@LRO-F3-BUMER MINGW64 /y/Android_Projects/test (master)
$

Git commit


MINGW64:/y/Android_Projects/testproject
bilal.umar@LRO-F3-BUMER MINGW64 /y/Android_Projects/testproject (master)
$ git commit -a -m"new commit"
warning: in the working copy of 'res/drawable/ic_launcher_background.xml', LF wi
ll be replaced by CRLF the next time Git touches it
[master ac5b1d7] new commit
Committer: Bilal Bin Umar <bilal.umar@vu.edu.pk>
Your name and email address were configured automatically based
on your username and hostname. Please check that they are accurate.
You can suppress this message by setting them explicitly. Run the
following command and follow the instructions in your editor to edit
your configuration file:
git config --global --edit
After doing this, you may fix the identity used for this commit with:
git commit --amend --reset-author
2 files changed, 2 insertions(+), 2 deletions(-)
bilal.umar@LRO-F3-BUMER MINGW64 /y/Android_Projects/testproject (master)
$

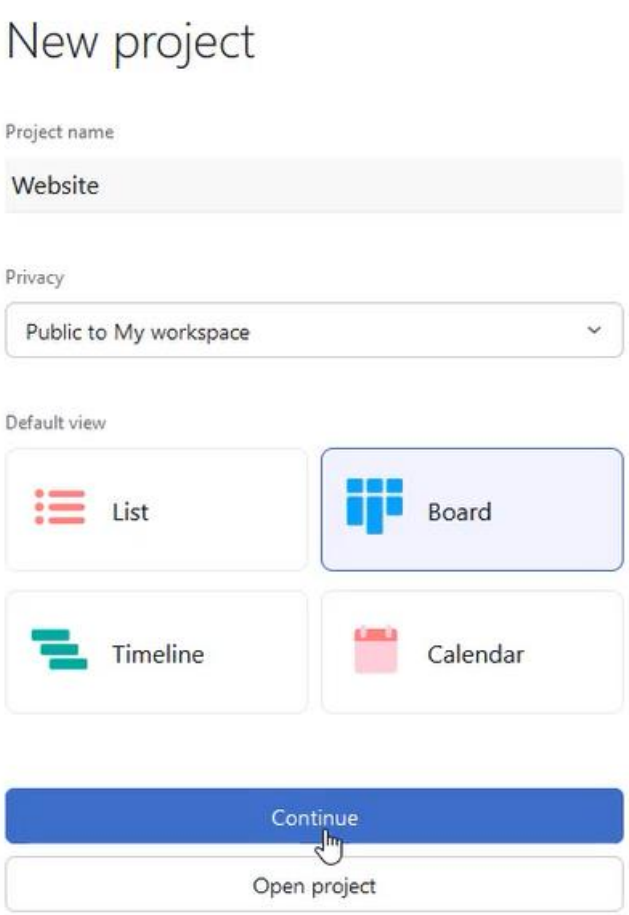
Git Push

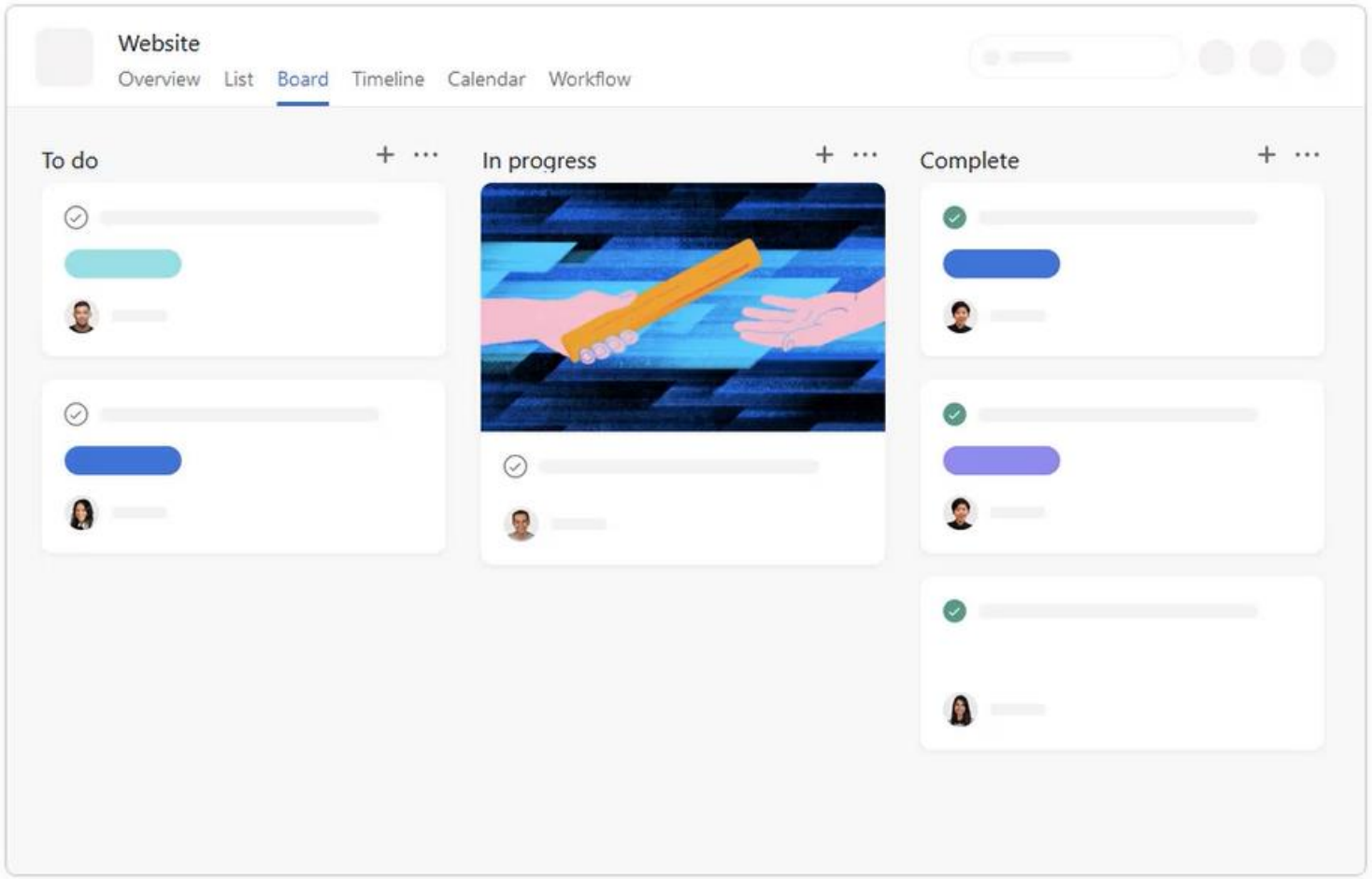
```
bilal.umar@LRO-F3-BUMER MINGW64 /y/Android_Projects/testproject (master)
$ git push origin master
Enumerating objects: 16, done.
Counting objects: 100% (16/16), done.
Delta compression using up to 4 threads
Compressing objects: 100% (9/9), done.
Writing objects: 100% (11/11), 820 bytes | 205.00 KiB/s, done.
Total 11 (delta 7), reused 0 (delta 0), pack-reused 0
To https://gitlab.com/u.bilal/testproject.git
   d146eec..ac5b1d7  master -> master
```

Software construction and development Practical - I

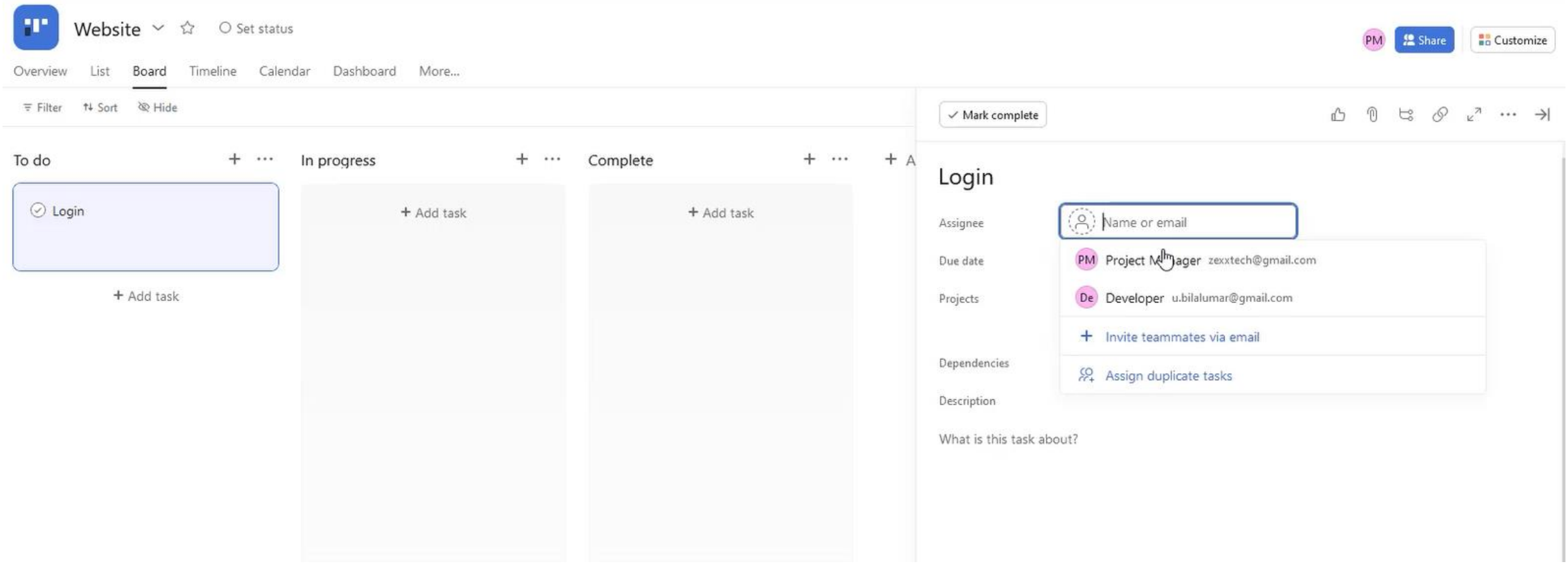
Week 13-Canban Practical

Create a new project in Asana

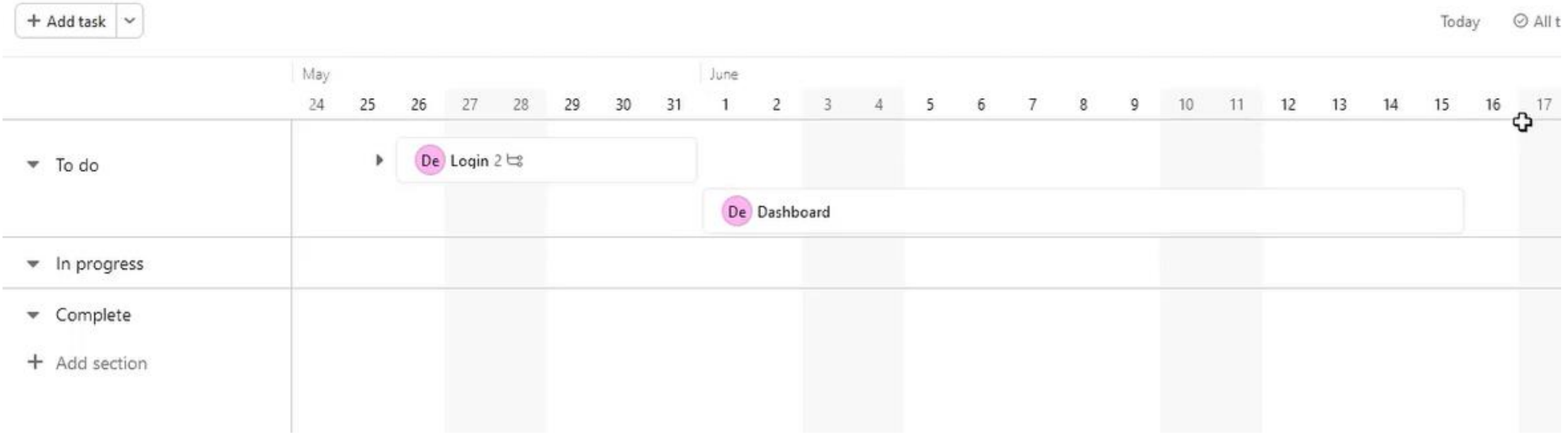




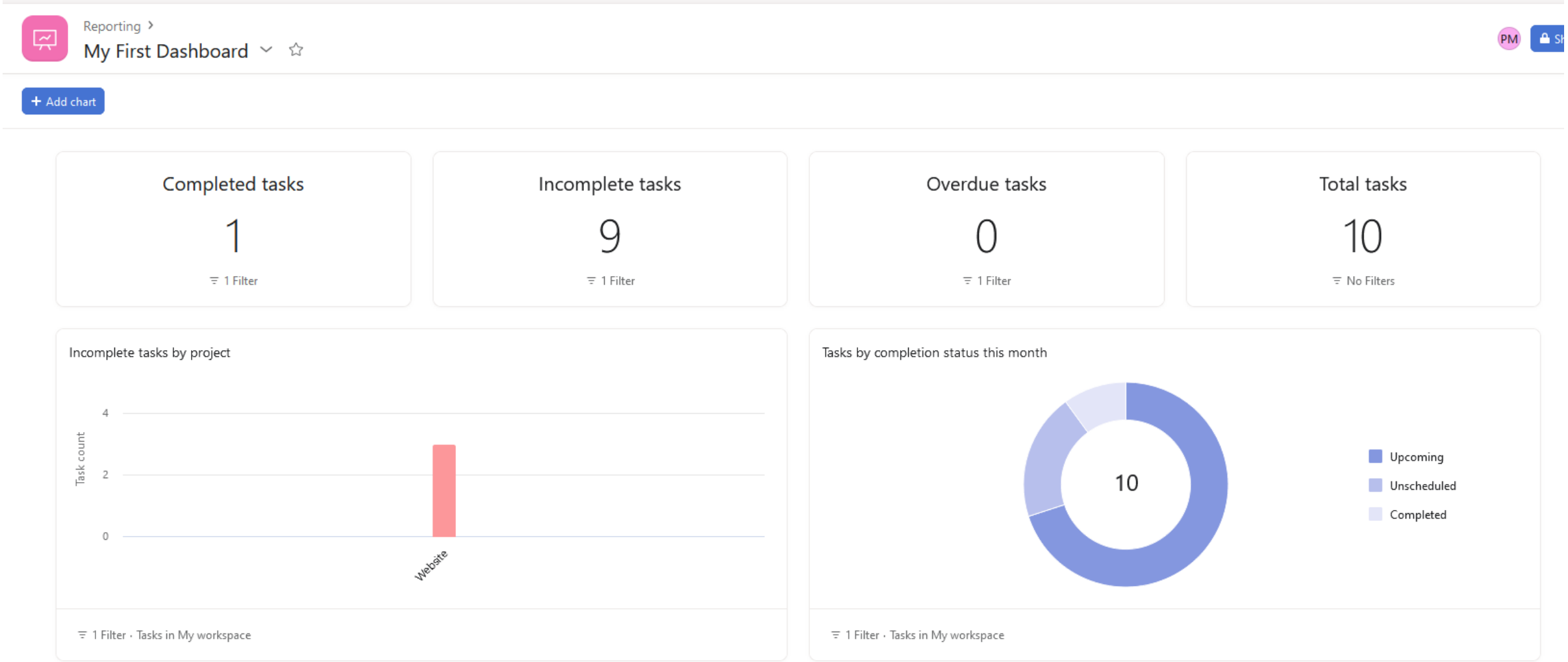
Add and Assign Tasks



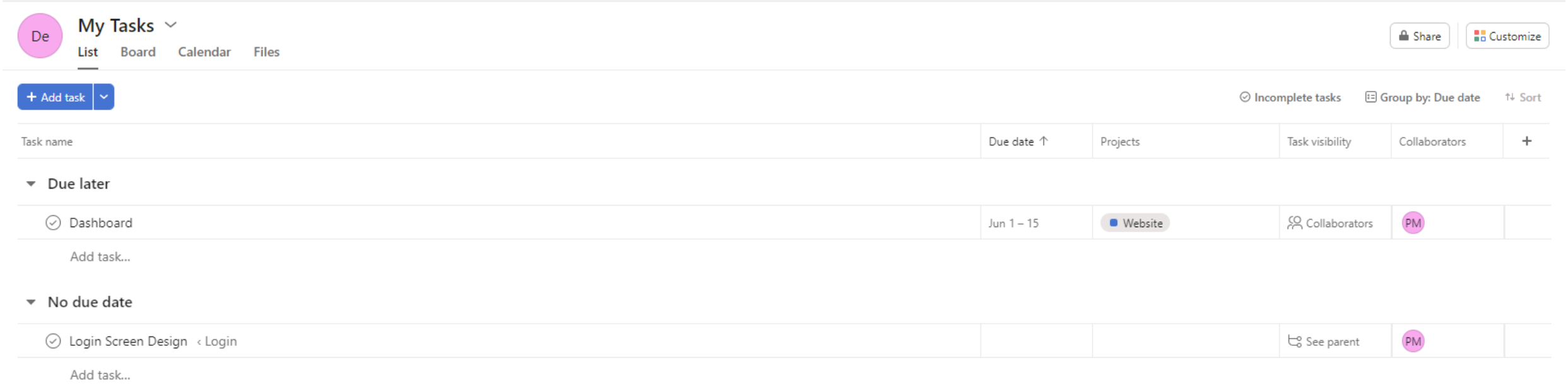
Set Timelines for a Task



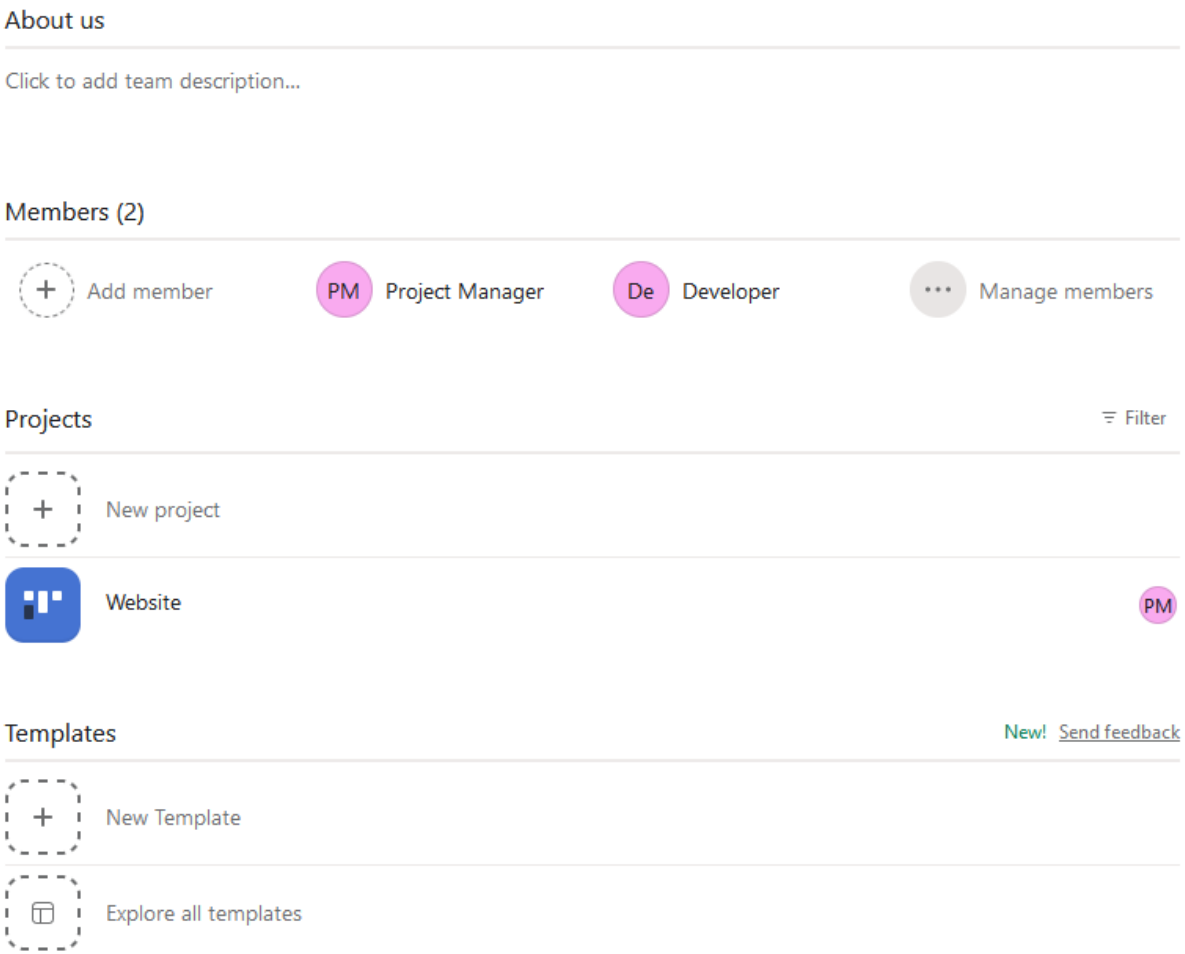
Project Manager Dashboard View



Developer View of Pending and Completed Tasks



Work Space and Team



Why?

- Why testing?
  - Improve software design
  - Make software easier to understand
  - Reduce debugging time
  - Catch integration errors
- In short, to Produce Better Code
- Preconditions
  - Working code
  - Good set of unit tests

What should be tested ?

- Test for boundary conditions
- Test for both success and failure
- Test for general functionality
- Etc..

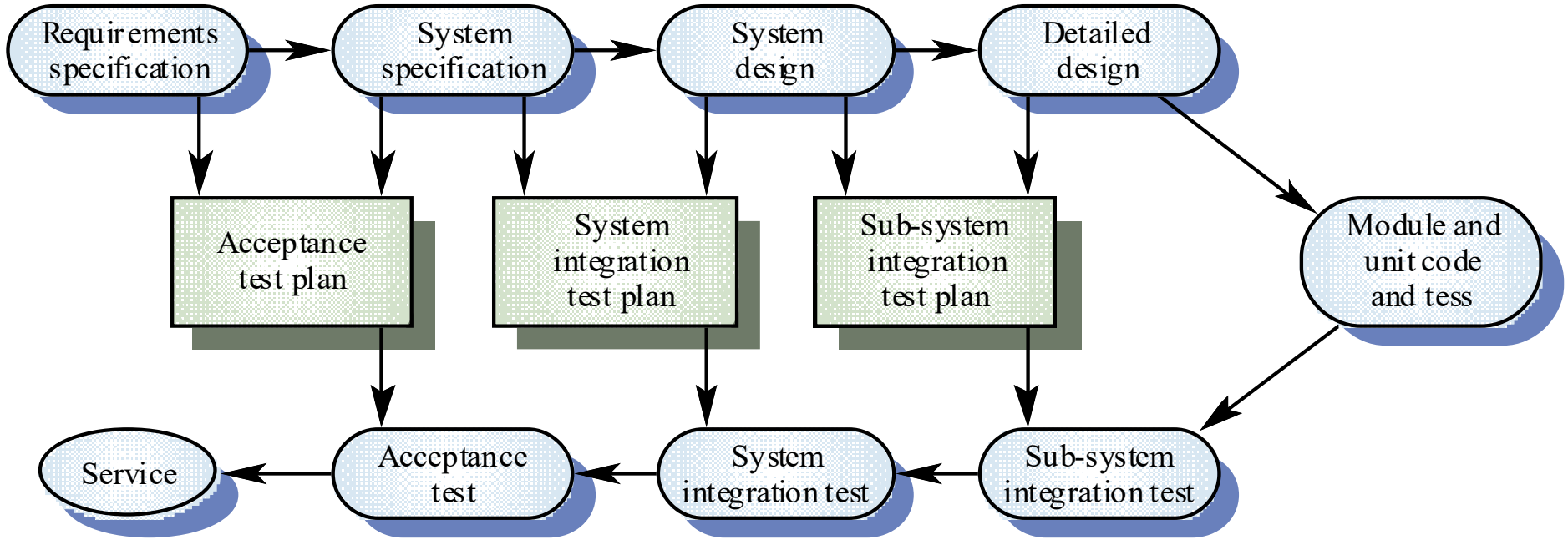
When to start testing

Software quality and testing is a life-cycle process

When to start testing...

- At the time of starting the projects
- How we start the projects ??
- Do we have any formal way ??

The V-model of development



Fact of testing

Testing does not guarantee the absence of defects

What is test case

- A test case is a document that describes an input, action, or event and an expected response, to determine if a feature of an application is working correctly

Good test case design

- An good test case satisfies the following criteria:
  - Reasonable probability of catching an error
  - Does interesting things
  - Doesn’t do unnecessary things
  - Neither too simple nor too complex
  - Not redundant with other tests
  - Makes failures obvious

Test case design technique

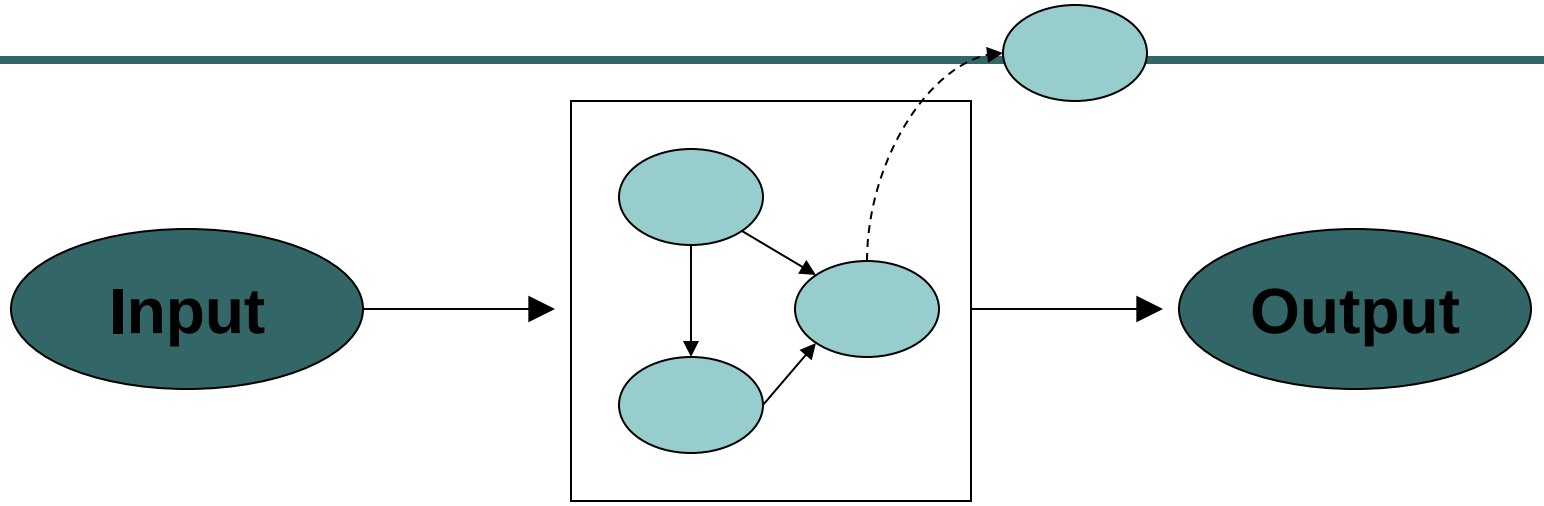
- Test case design techniques can be broadly split into two main categories
  - **Black box (functional)**
  - **White box (structural)**

Black Box tests



- Targeted at the apparent simplicity of the software
  - Makes assumptions about implementation
  - Good for testing component interactions
- Tests the interfaces and behavior

White Box tests



- Targeted at the underlying complexity of the software
  - Intimate knowledge of implementation
  - Good for testing individual functions
- Tests the implementation and design

Test case writing example

- Suppose we have two parameters we want to cover in a set of tests. Parameters are as follows..

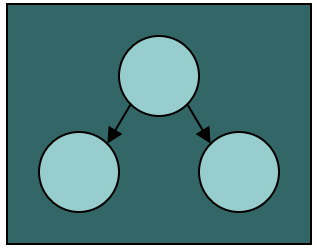
- Operating system
  - Win98
  - Win2k
  - Winxp
- Printers
  - HP 4100
  - HP 4200

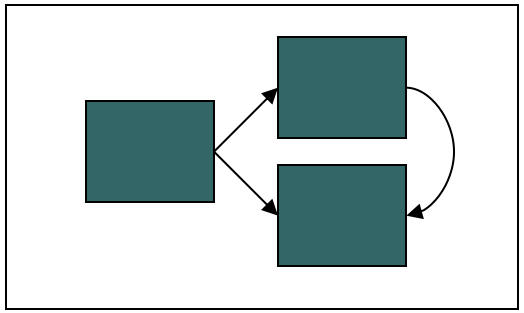
How We should write test case for this ??

Types of Tests

- Unit
  - Individual classes or types
- Component
  - Group of related classes or types
- Integration
  - Interaction between classes









What is a testing framework?

- A test framework provides reusable test functionality which:
  - Is easier to use (e.g. don’t have to write the same code for each class)
  - Is standardized and reusable
  - Provides a base for regression tests

Why use a testing framework?

- Each class must be tested when it is developed
- Each class needs a regression test
- Regression tests need to have standard interfaces
- Thus, we can build the regression test when building the class and have a better, more stable product for less work

Testing tools

Tools are part of the quality equation, but not the entire equation

JUnit

- JUnit is a framework for writing unit tests
  - A unit test is a test of a *single* class
    - A test case is a single test of a single method
    - A test suite is a collection of test cases
- Unit testing is particularly important when software requirements change frequently
  - Code often has to be refactored to incorporate the changes
  - Unit testing helps ensure that the refactored code continues to work

JUnit..

- JUnit helps the programmer:
  - Define and execute tests and test suites
  - Formalize requirements and clarify architecture
  - Write and debug code
  - Integrate code and always be ready to release a working version

What JUnit does

- JUnit runs a suite of tests and reports results
- For *each* test in the test suite:
  - JUnit calls setUp()
    - This method should create any objects you may need for testing

What JUnit does…

- JUnit calls *one* test method
  - The test method may comprise multiple test cases; that is, it may make multiple calls to the method you are testing
  - In fact, since it's your code, the test method can do anything you want
  - The setUp() method ensures you *entered* the test method with a virgin set of objects; what you do with them is up to you
- JUnit calls tearDown()
  - This method should remove any objects you created

Creating a test class in JUnit

- Define a subclass of TestCase
- Override the setUp() method to initialize object(s) under test.
- Override the tearDown() method to release object(s) under test.
- Define one or more public testXXX() methods that exercise the object(s) under test and assert expected results.
- Define a static suite() factory method that creates a TestSuite containing all the testXXX() methods of the TestCase.
- Optionally define a main() method that runs the TestCase in batch mode.

Fixtures

- A fixture is just a some code you want run before every test
- You get a fixture by overriding the method
  - protected void setUp() { …}
- The general rule for running a test is:
  - protected void runTest() {
    setUp(); <run the test> tearDown();
    }
  - so we can override setUp and/or tearDown, and that code will be run prior to or after every test case

The structure of a test method

- A test method doesn’t return a result
- If the tests run correctly, a test method does nothing
- If a test fails, it throws an AssertionFailedError
- The JUnit framework catches the error and deals with it; you don’t have to do anything

Why JUnit

- **Allow you to write code faster while increasing quality**
- **Elegantly simple**
- **Check their own results and provide immediate feedback**
- **Tests is inexpensive**
- **Increase the stability of software**
- **Developer tests**
- **Written in Java**
- **Free**
- **Gives proper uniderstanding of unit testing**

Problems with unit testing

- JUnit is designed to call methods and compare the results they return against expected results
  - This ignores:
    - Programs that do work in response to GUI commands
    - Methods that are used primary to produce output

Problems with unit testing…

- I think heavy use of JUnit encourages a "functional" style, where most methods are called to compute a value, rather than to have side effects
  - This can actually be a good thing
  - Methods that *just* return results, without side effects (such as printing), are simpler, more general, and easier to reuse

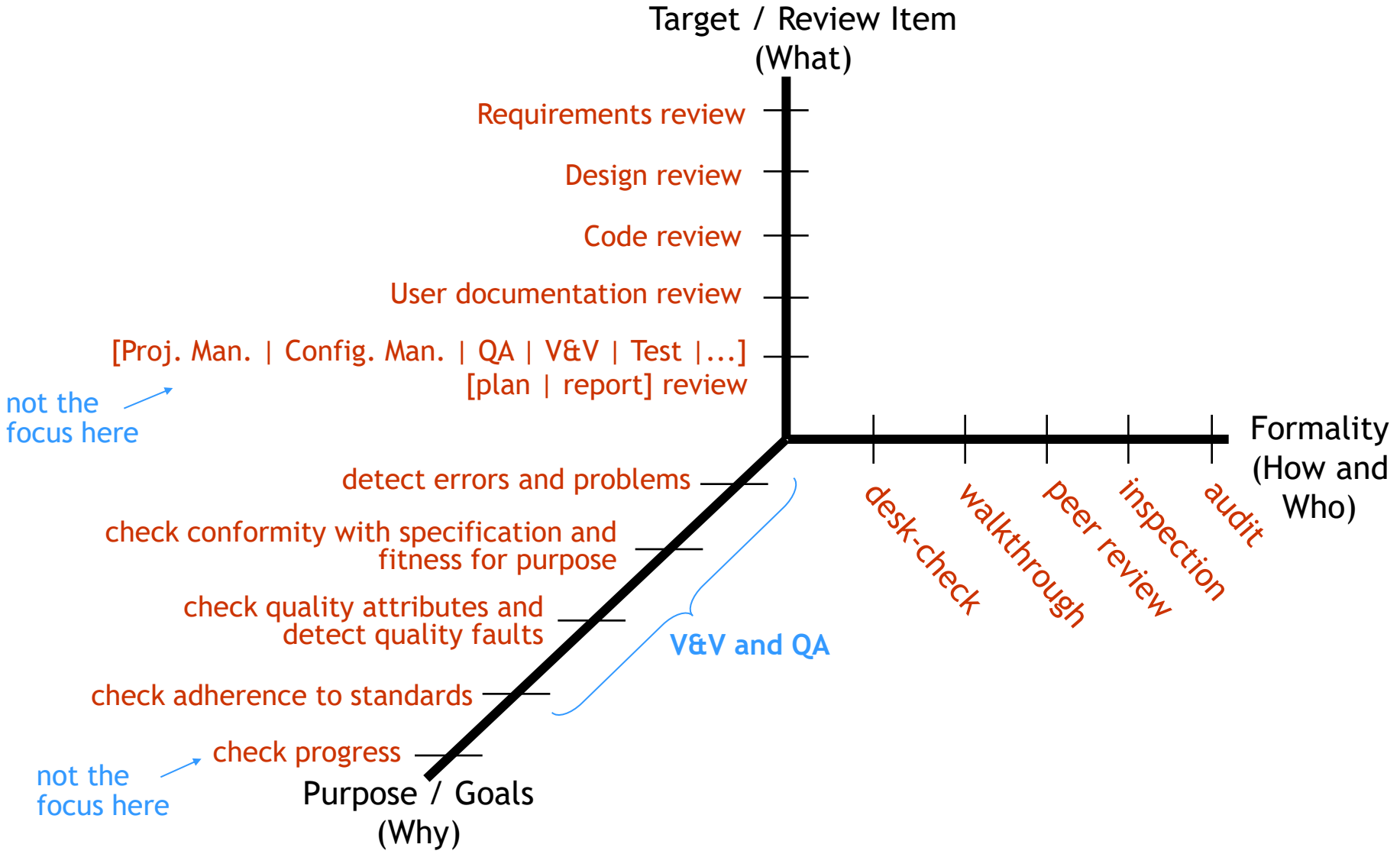
Code Reviews & Static Software Analysis

Week-15

Topics

- Introduction
  - Types of Reviews along the software life cycle
  - Reviews and testing
  - Review planning
  - Review roles, responsibilities and attendance
- Types of reviews according to formality
- Checklists
- Reporting and follow-up
- Other static software analysis techniques

Types of reviews



Desk check

- Also called self check
- Informal review performed by the author of the artifact

Peer reviews

- “I show you mine and you show me yours”
- The author of the reviewed item does not participate in the review
- Effective technique that can be applied when there is a team (with two or more persons) for each role (analyst, designer, programmer, technical writer, etc.)
- The peer may be a senior colleague (senior/chief analyst, senior/chief architect, senior/chief programmer, senior/chief technical writer, etc.)

Walkthroughs

- Type of technical review where the producer of the reviewed material serves as the review leader and actually guides the progression of the review (as a review reader)
- Traditionally applied to design and code
- In the case of code walkthrough, test inputs may be selected and review participants then literally walk through the design or code
- Checklist and preparation steps may be eliminated

Inspections

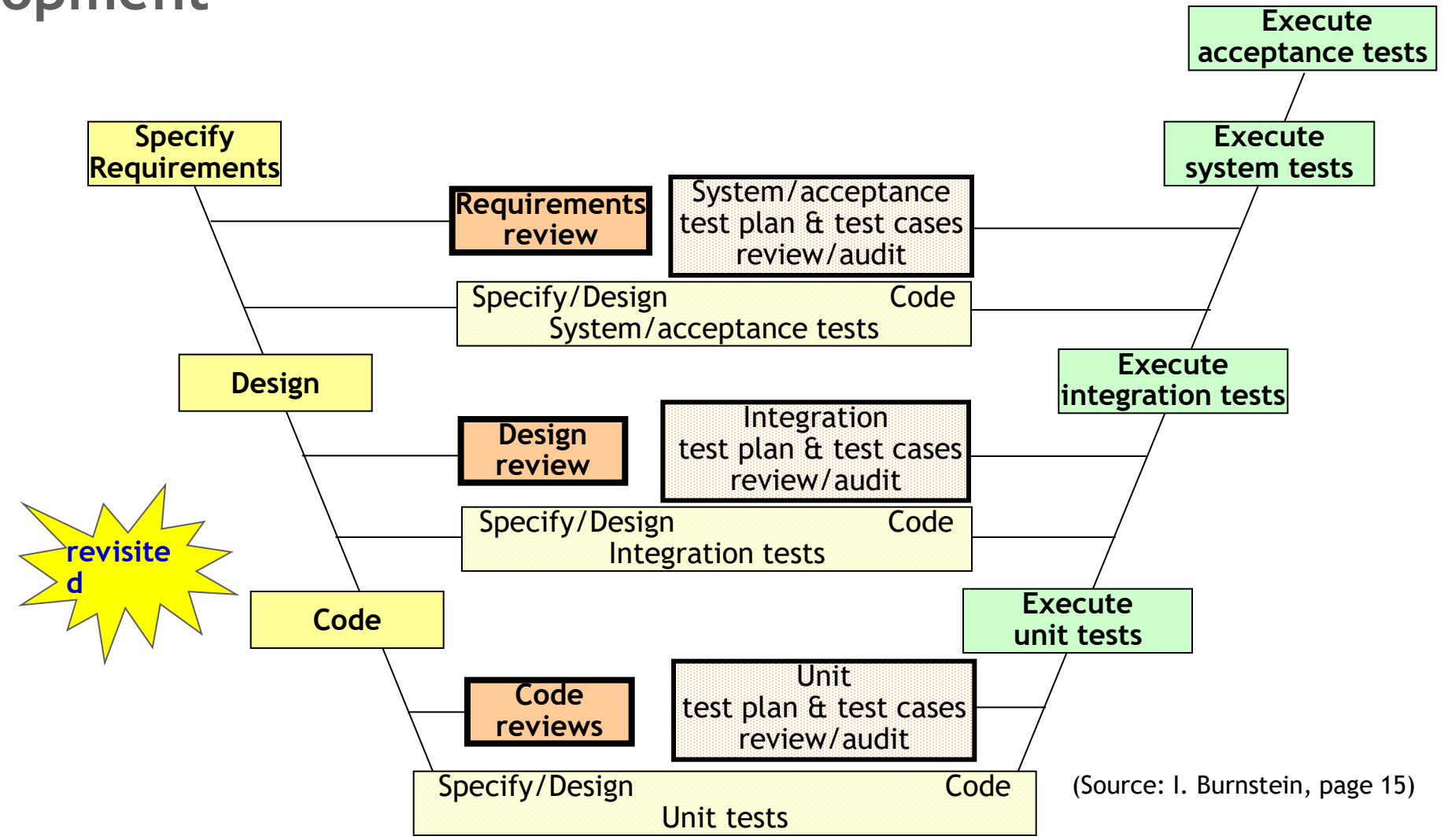
- A **formal evaluation** technique in which software requirements, design, or code are examined in detail by a person or group other than the author to detect faults, violations of development standards, and other problems
- Generally involve the author of a product
- The inspector team may consist of different expertise, such as domain expertise, or design method expertise, or language expertise, etc. Inspections are usually conducted on a relatively small section of the product.
- Often the inspection team may have had a few hours to prepare, perhaps by applying an analytic technique to a small section of the product, or to the entire product with a focus only on one aspect, e.g., interfaces.
- A checklist, with questions germane to the issues of interest, is a common tool used in inspections.
- Inspection sessions can last a couple of hours or less, whereas reviews and audits are usually broader in scope and take longer.
- (source : SWEBOK)

Audits

- An audit is an **independent evaluation of conformance** of software products and processes to applicable regulations, standards, plans, and procedures
- An audit is a formally organized activity, with participants having specific roles, such as lead auditor, other auditors, a recorder, an initiator, and a representative of the audited organization
- Audits may examine plans like recovery, SQA, design documentation, etc.
- Audits can occur on almost any product at any stage of the development or maintenance process

(source : SWEBOK)

Software reviews and the extended V-model of software development

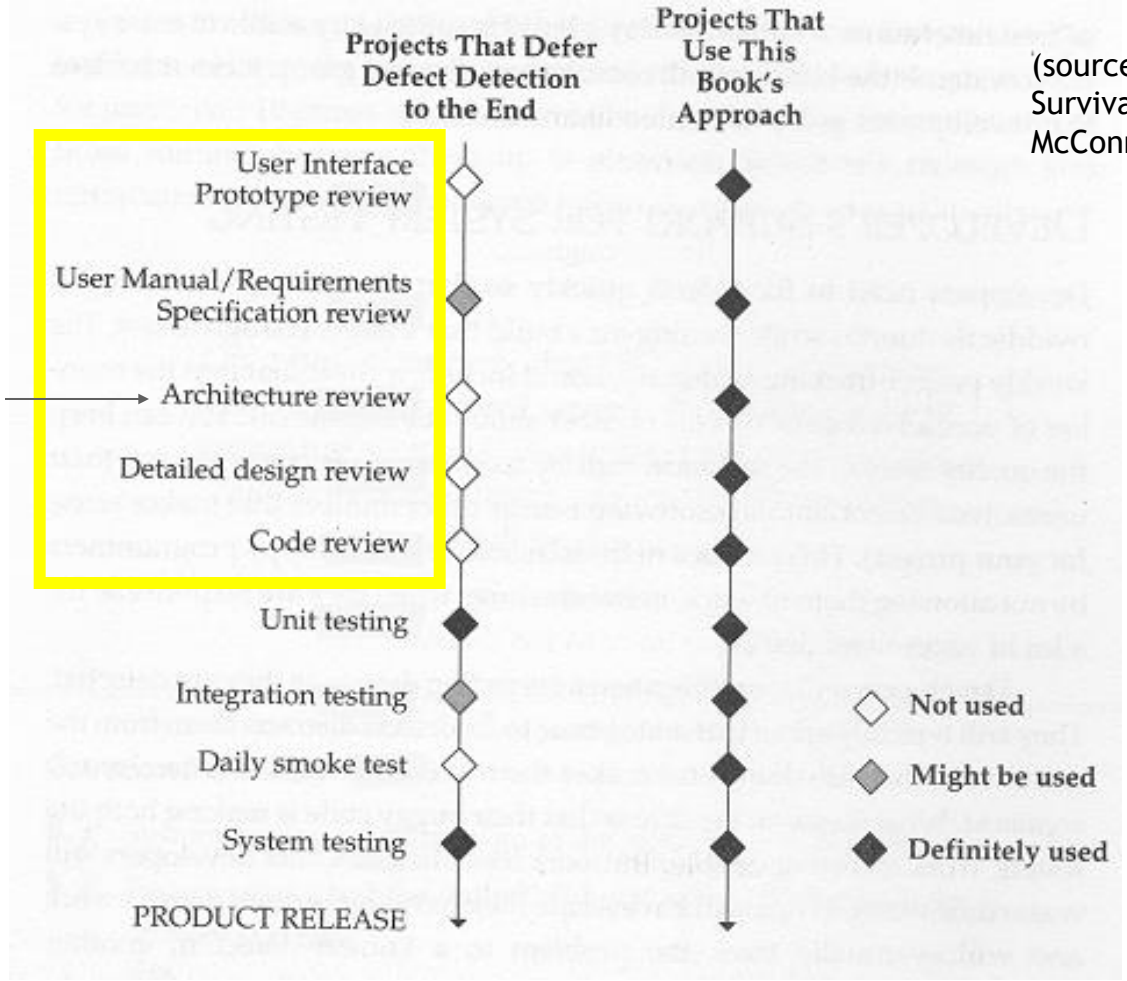


(Source: I. Burnstein, page 15)

Typical tests and reviews

revisited

high-level design →



(source: "Software Project Survival Guide", Steve McConnell)

Reviews and testing

- A software system is more than the code; it is a set of related artifacts; these may contain defects or problem areas that should be reworked or removed; quality-related attributes of these artifacts should be evaluated
- Reviews allow us to detect and eliminate errors/defects *early* in the software life cycle (even before any code is available for testing), where they are less costly to repair
- Most problems have their origin in requirements and design; requirements and design artifacts can be reviewed but not executed and tested
  - Early prototyping is equally important to reveal problems in requirements and high-level architectural design
- A code review usually reveals directly the location of a bug, while testing requires a debugging step to locate the origin of a bug
- Adherence to coding standards cannot be checked by testing

>>>>>>>>>>>>>>>>>>>>> www.softwaretestinggenius.com <<<<<<<<<<<<<<<<<<<<<<

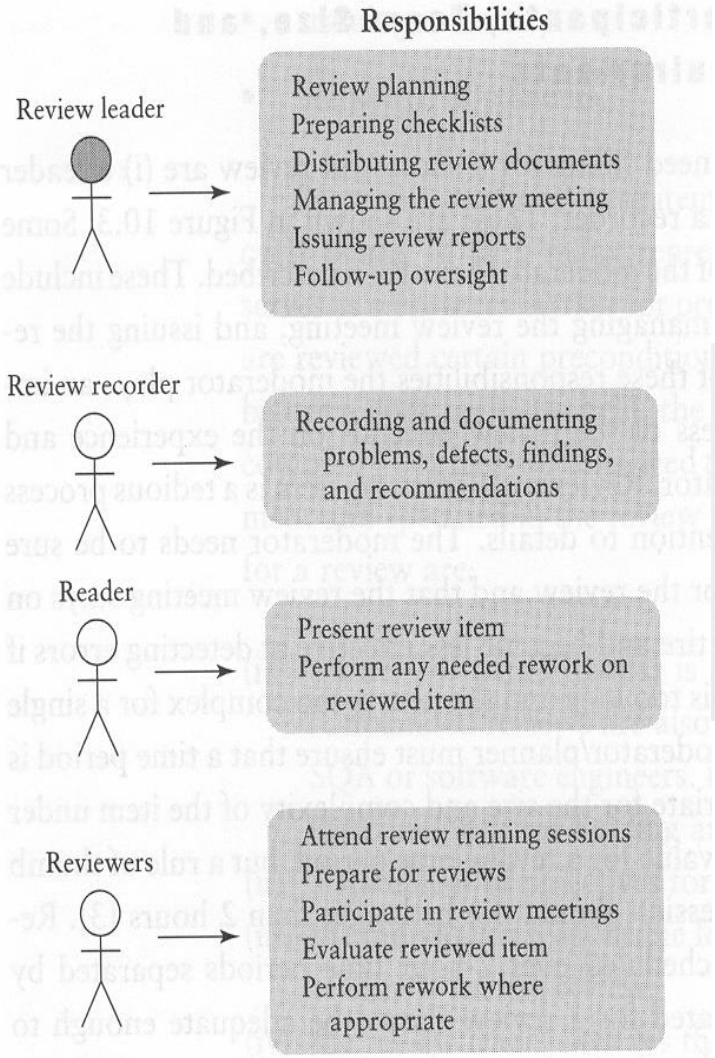
Technical and management reviews

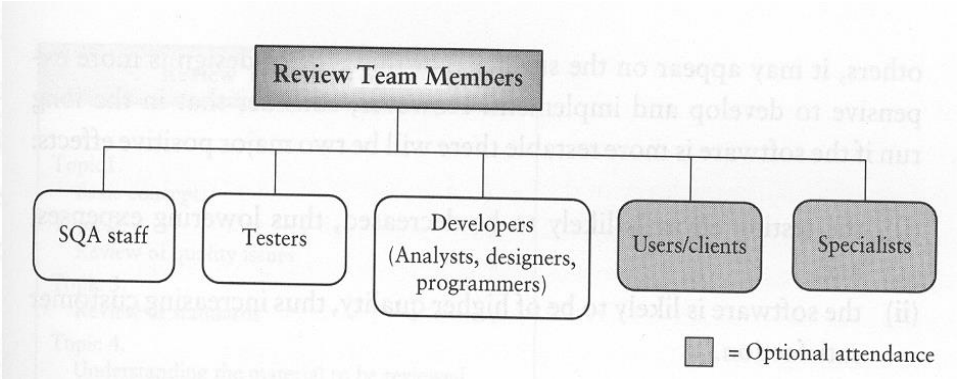
- Technical Reviews - examine work products of the software project (code, requirement specifications, software design documents, test documentation, user documentation, installation procedures) for V&V and QA purposes
  - Multiple forms: Desk checking, Walkthroughs, Inspections, Peer Reviews, Audits
  - Covered here
- Management Reviews - determine adequacy of and monitor progress or inconsistencies against plans and schedules and requirements
  - Includes what Ian Somerville calls **Progress Reviews**
  - May be exercised **on plans and reports** of many types (risk management plans, project management plans, software configuration management plans, audit reports, progress reports, V&V reports, etc.)

Components of a review plan

- Review goals
- Items being reviewed
- Preconditions for the review
- Roles, team size, participants
- Training requirements
- Review steps and procedures
- Checklists and other related documents to be distributed to participants
- Time requirements
- Nature of the review log and summary report
- Rework and follow-up

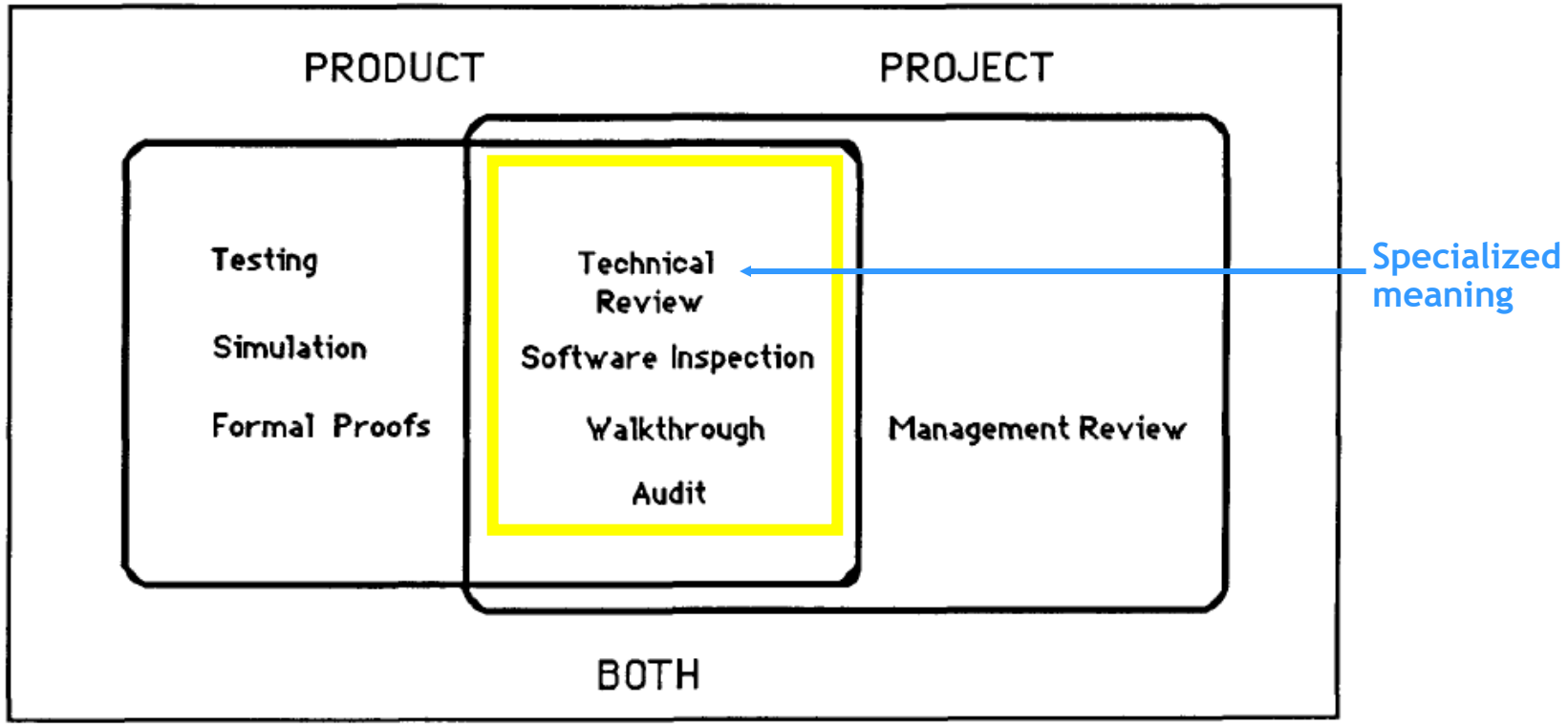
(Source: I. Bursntein)


Responsibilities
Review leader
Review planning
Preparing checklists
Distributing review documents
Managing the review meeting
Issuing review reports
Follow-up oversight
Review recorder
Recording and documenting problems, defects, findings, and recommendations
Reader
Present review item
Perform any needed rework on reviewed item
Reviewers
Attend review training sessions
Prepare for reviews
Participate in review meetings
Evaluate reviewed item
Perform rework where appropriate


Review Team Members
SQA staff
Testers
Developers (Analysts, designers, programmers)
Users/clients
Specialists
= Optional attendance

IEEE Standard for Software Reviews and Audits (IEEE Std 1028-1988)

Fig 1
Relation of Some Quality Assurance Processes to Products and Projects



Types of Reviews in IEEE Std 1028-1988

Category and Attributes	Management Review	Technical Review	Software Inspection	Walkthrough
Objective	Ensure progress. Recommend corrective action. Ensure proper allocation of resources.	Evaluate conformance to specifications and plans. Ensure change integrity.	Detect and identify defects. Verify resolution.	Detect defects. Examine alternatives. Forum for learning.
Delegated Controls				
Decision making	Management team charts course of action. Decisions are made at the meeting or as a result of recommendations.	Review team petitions. Management or technical leadership to act on recommendations.	Team chooses from predefined product dispositions. Defects must be removed.	All decisions made by producer. Change is the prerogative of the producer.
Change verification	Change verification left to other project controls.	Leader verifies as part of review report.	Moderator verifies rework.	Change verification left to other project controls.
Group Dynamics				
Recommended size	Two or more persons.	Three or more persons.	Three to six persons.	Two to seven persons.
Attendance	Management, technical leadership, and peer mix.	Technical leadership and peer mix.	College of peers meet with documented attendance.	Technical leadership and peer mix.
Leadership	Usually the responsible manager.	Usually the lead engineer.	Trained moderator.	Usually producer.

A sample general checklist for reviewing software documents

- Coverage and **completeness**
  - Are all essential items completed?
  - Have all irrelevant items been omitted?
  - Is the technical level of each topic addressed properly for this document?
  - Is there a clear statement of goals for this document?
  - (Don't forget: more documentation does not mean better documentation)
- **Correctness**
  - Are there incorrect items?
  - Are there any contradictions?
  - Are the any ambiguities?
- **Clarity** and **Consistency**
  - Are the material and statements in the document clear?
  - Are the examples clear, useful, relevant and correct?
- **Clarity** and **Consistency** (cont.)
  - Are the diagrams, graphs and illustrations clear, correct, use the proper notation, effective, in the proper place?
  - Is the terminology clear and correct?
  - Is there a glossary of technical terms that is complete and correct?
  - Is the writing style clear (nonambiguous)?
- References and Aids to Document Comprehension
  - Is there an abstract or introduction?
  - Is there a well placed table of contents?
  - Are the topics or items broken down in a manner that is easy to follow and is understandable?
  - Is there a bibliography that is clear, complete and correct?
  - Is there an index that is clear, complete and correct?
  - Is the page and figure numbering correct and consistent?

(Adapted from Ilene Burnstein, Practical Software Testing, page 327)

A sample specification (or requirements) attributes checklist

Attribute	What to consider
Complete	Is anything missing or forgotten? Is it thorough? Does it include everything necessary to make it stand alone?
Accurate	Is the proposed solution correct? Does it properly define the goal? Are there any errors?
Precise, Unambiguous and Clear	Is the description exact and not vague? Is there a single interpretation? Is it easy to read and understandable?
Consistent	Is the description of the feature written so that it doesn't conflict with itself or other items in the specification?
Relevant	Is the statement necessary to specify the feature? Is there extra information that should be left out? Is the feature traceable to an original customer need?
Feasible	Can the feature be implemented with the available personnel, tools, and resources within the specified budget and schedule?
Code-free	Does the specification stick with defining the product and not the underlying software design, architecture, and code?
Testable	Can the feature be tested? Is enough information provided that a tester could create tests to verify its operation?

(Adapted from: Ron Patton, Software Testing)

A sample supplementary checklist for design reviews
(for high-level architectural design and detailed design)

- Are the high-level and detailed design consistent with requirements? Do they address all the functional and quality requirements? Is detailed design consistent with high-level design?
- Are design decisions properly highlighted and justified and traced back to requirements? Are design alternatives identified and evaluated?
- Are design notations (ex: UML), methods (ex: OOD, ATAM) and standards chosen and used adequately?
- Are naming conventions being followed appropriately?
- Is the system structuring (partitioning into sub-systems, modules, layers, etc.) well defined and explained? Are the responsibilities of each module and the relationships between modules well defined and explained? Do modules exhibit strong cohesion and weak coupling?
- Is there a clear and rigorous description of each module interface, both at the syntactic and semantic level? Are dependencies identified?
- Have user interface design issues, including standardization, been addressed properly?
- Is there a clear description of the interfaces between this system and other software and hardware systems?
- Have reuse issues been properly addressed, namely the possible reuse of COTS (commercial off the shelf) components (buy-or-build decision) and in-house reusable components?
- Is the system designed so that it can be tested at various levels (unit, integration and system)?

(Adapted from: Ilene Burnstein, page 328-329)

A sample general code review checklist (1)

- Design Issues
  - Does each unit implement a single function?
  - Are there instances where the unit should he partitioned?
  - Is code consistent with detailed design?
  - Does the code cover detailed design?
- Data Items
  - Is there an input validity check?
  - Arrays-check array dimensions, boundaries, indices.
  - Variables - are they all defined, initiated? have correct types and scopes been checked?
  - Are all variables used?
- Computations
  - Are there computations using variables with inconsistent data types?
  - Are there mixed-mode computations?
  - Is the target value of an assignment smaller than the right-hand expression?
  - Is over- or underflow a possibility (division by zero)?
  - Are there invalid uses of integers or floating point arithmetic?
  - Are there comparisons between floating point numbers?
  - Are there assumptions about the evaluation order in Boolean expressions?
  - Are the comparison operators correct?

A sample general code review checklist (2)

- Control Flow Issues
  - Will the program, module or, unit eventually terminate?
  - Is there a possibility of an infinite loop, a loop with a premature exit, a loop that never executes?
- Interface Issues
  - Do the number and attributes of the parameters used by a caller match those of the called routine? Is the order of parameters also correct and consistent in caller and callee?
  - Does a function or procedure alter a parameter that is only meant as an input parameter?
  - If there are global variables, do they have corresponding definitions and attributes in all the modules that use them?
- Input/output Issues
  - Have all files been opened for use?
  - Are all files properly closed at termination?
  - If files are declared are their attributes correct?
  - Are EOF or I/O errors conditions handed correctly?
  - Is I/O buffer size and record size compatible?

A sample general code review checklist (3)

- Portability Issues
  - Is there an assumed character set, and integer or floating point representation?
  - Are their service calls that mar need to be modified?
- Error Messages
  - Have all warnings and informational messages been checked and used appropriately?
- Comments/Code Documentation
  - Has the code been properly documented? Are there global, procedure, and line comments where appropriate?
  - Is the documentation clear, and correct, and does it support understanding?
- Code Layout and White Space
  - Has white space and indentation been used to support understanding of code logic and code intent?
- Maintenance
  - Does each module have a single exit point?
  - Are the modules easy to change (low coupling and high cohesion)?

(Adapted from: Ilene Burnstein, page 331)

A sample code review checklist for C++ programs (1)

- Data Items
  - Are all variables lowercase?
  - Are all variables initialized?
  - Are variable names consistent, and do they reflect usage?
  - Are all declarations documented (except for those that are very simple to understand)?
  - Is each name used for a singe function (except for loop variable names)?
  - Is the scope of the variable as intended?
- Constants
  - Are all constants in uppercase?
  - Are all constants defined with a "#define"?
  - Are all constants used in multiple files defined in an INCLUDE header file?
- Pointers
  - Are pointers declared properly as pointers?
  - Are the pointers initialized properly?

A sample code review checklist for C programs (2)

- Control
  - Are if/then, else, and switch statements used clearly and properly?
- Strings
  - Strings should have proper pointers.
  - Strings should end with a NULL.
- Brackets
  - All curly brackets should have appropriate indentations and be matched
- Logic Operators
  - Do all initializations use an " = " and not an " = ="?
  - Check to see that all logic operators are correct, for example, use of = / = =, and ||
- Computations
  - Are parentheses used in complex expressions and are they used properly for specifying precedences?
  - Are shifts used properly?

(Adapted from: Ilene Burnstein, page. 331)

Types of (end-user) software documentation(1)

- **Packaging text and graphics**. Box, carton, wrapping, and so on. Might contain screen shots from the software, lists of features, system requirements, and copyright information.
- **Marketing material, ads, and other inserts**. These are all the pieces of paper you usually throw away, but they are important tools used to promote the sale of related software, add-on content, service contracts, and so on. The information for them must be correct for a customer to take them seriously.
- **Warranty/registration**. This is the card that the customer fills out and sends in to register the software. It can also be part of the software and display onscreen for the user to read, acknowledge, and even complete online.
- **EULA**. Pronounced "you-la," it stands for End User License Agreement. This is the legal document that the customer agrees to that says, among other things, that he won't copy the software nor sue the manufacturer if he's harmed by a bug. The EULA is sometimes printed on the envelope containing the media-the floppy or CD. It also may pop up onscreen during the software's installation.
- **Labels and stickers**. These may appear on the media, on the box, or on the printed material. There may also be serial number stickers and labels that seal the EULA envelope. See in a following slide an example of a disk label and all the information that needs to be checked.
- **Installation and setup instructions**. Sometimes this information is printed on the media, but it also can be included as a separate sheet of paper or, if it's complex software, as an entire manual.

Types of (end-user) software documentation (2)

- **User's manual**. The usefulness and flexibility of online manuals has made printed manuals much less common than they once were. Most software now comes with a small, concise "getting started"-type manual with the detailed information moved to online format. The online manuals can be distributed on the software's media, on a Web site, or a combination of both.
- **Online help**. Online help often gets intertwined with the user's manual, sometimes even replacing it. Online help is indexed and searchable, making it much easier for users to find the information they're looking for. Many online help systems allow natural language queries so users can type "Tell me how to copy text from one program to another" and receive an appropriate response.
- **Tutorials, wizards, and CBT (Computer Based Training)**. These tools blend programming code and written documentation. They're often a mixture of both content and high-level, macro-like programming and are often tied in with the online help system. A user can ask a question and the software then guides him through the steps to complete the task. Microsoft's Office Assistant, sometimes referred to as the "paper clip guy" is an example of such a system.
- **Samples, examples, and templates**. An example of these would be a word processor with forms or samples that a user can simply fill in to quickly create professional-looking results. A compiler could have snippets of code that demonstrate how to use certain aspects of the language.
- **Error messages**. Often neglected; ultimately fall under the category of documentation.

(Adapted from: Ron Patton, Software Testing, page 190-192)

Quality attributes (or dimensions) to check in technical information

Quality dimension	Description
Easy to Use	
Task Orientation	Helps users complete tasks related to their work by using the product.
Accuracy	Contains no mistakes or errors, truthful and factual.
Completeness	Includes all essential parts (but only these parts).
Easy to Understand	
Clarity	Contains no ambiguity or obscurity.
Concreteness	Contains no abstractions; including appropriate examples, scenarios, and metaphors.
Style	Uses correct and appropriate writing conventions and word choice.
Easy to Find	
Organization	Organizes material coherently in a way that makes sense to the user.
Retrievability	Presents information in a way that lets users find information quickly and easily.
Visual Effectiveness	Uses layout, illustrations, color, type, icons, and other graphical devices to enhance meaning and attractiveness.

Can be checked by asking probing questions, like:

- Is the information appropriate for the intended audience?
- Is information presented from a user's point of view?
- Is there a focus on real tasks?
- Is the reason for the information evident?
- Do titles and headings reveal real tasks?

Build your own check list!
Adapt to your needs!

Source: **Developing Quality Technical Information** (**DQTI**), Hargis, IBM, 1997

not only for software, not only for end-user documentation
(also documentation for developers and maintainers)

Contents of a formal review report (1)

- Checklist will all items covered (with a check mark) and comments relating to each item
- List of defects found, with
  - description
  - type
  - frequency
  - defect class, e.g.
    - missing
    - incorrect
    - superfluous
  - location
    - cross-reference to the place or places in the reviewed document where the defect occurs
  - severity, e.g.
    - major
    - minor

Contents of a formal review report (2)

- Summary report, with
  - list of attendees
  - review metrics, such as
    - number of participants
    - duration of the meeting
    - size of the item being reviewed (usually LOC or number of pages)
    - number of defects found
    - total preparation time for the review team
    - number of defects found per hour of review time
    - number of defects found per page or LOC
    - LOC or pages reviewed per hour
    - ...
  - status of the reviewed item (requirements document, etc.)
    - **accept** – the item is accepted in its present form or with minor rework required that does not need further verification
    - **conditional accept** – the item needs rework and will be accepted after the moderator has checked and verified the rework
    - **reinspect** – considerable rework must be done to the item. The inspection needs to be repeated when the rework is done.
  - estimate of rework effort and the estimated date for completion of the rework
  - signatures and date

Automated static software analysis (1)

- Static code analysis and audit tools
  - *rule based* - perform checks that result in observations on coding practices; look for constructs that "look dangerous"
  - *metric based* – perform checks that result in observations on code quality metrics values such as Cyclomatic Complexity and Nesting Depth
- Formal proofs
  - based on mathematics
  - may be partially automated (or at least supported by tools that check the internal consistency of the proof)
- Model checking
  - based on a finite state model of the system
  - tools automate proof of properties such as reachability and absence of cycles

Thank You